



Université de Tunis El Manar

Faculté des Sciences de Tunis

Département des Sciences de l'Informatique



ChargeTrackr

Rapport de stage d'été

Développement d'un outil de visualisation et de suivi de la disponibilité des bornes de recharge électrique

Réalisé par

Wajdi Ben Abdeljelil

Encadré en entreprise par

M. Bassem Soua

Formation

Cycle ingénieur en génie
logiciel (IGL3)

Organisme d'accueil



TAC-TIC Tunisie

Du 1^{er} juillet au 31 août 2026

Année universitaire 2025–2026

Remerciements

Je tiens à remercier l'entreprise TAC-TIC Tunisie pour son accueil et pour l'opportunité de réaliser ce stage dans un contexte professionnel lié à la mobilité électrique et aux systèmes logiciels distribués.

J'adresse mes remerciements à M. Bassem Soua, encadrant en entreprise, pour son accompagnement, ses orientations et les échanges consacrés à la compréhension des besoins du projet.

Je remercie également les enseignants du Département des Sciences de l'Informatique de la Faculté des Sciences de Tunis pour les connaissances acquises durant ma première année du cycle ingénieur en génie logiciel.

Enfin, j'exprime ma reconnaissance à toutes les personnes qui ont contribué, par leurs conseils, leurs retours ou leur soutien, à l'avancement de ce projet et à la préparation de ce rapport.

Table des matières

Remerciements	i
Liste des acronymes	xi
Introduction générale	1
1 État de l’art et fondements métiers	3
1.1 Cadre du projet	3
1.1.1 Cadre académique	3
1.1.2 Présentation de l’organisme d’accueil	4
1.1.3 Présentation du domaine d’application	4
1.1.3.1 Mobilité électrique et infrastructure de recharge	4
1.1.3.2 Système central de gestion des bornes	5
1.1.3.3 Interopérabilité avec les bornes	5
1.1.3.4 Disponibilité et qualité de service	6
1.2 Problématique	6
1.3 Étude de l’existant	7
1.3.1 Solutions existantes	7
1.3.1.1 ChargePoint	7
1.3.1.2 Driivz	8
1.3.1.3 Monta	8
1.3.1.4 CitrineOS	8
1.3.2 Synthèse comparative	8
1.4 Solution proposée	9
1.4.1 Vue d’ensemble fonctionnelle	9
1.4.2 Espaces et responsabilités des utilisateurs	10

1.4.3	Périmètre du MVP	11
1.5	Méthodologie de travail	11
1.5.1	Le framework Scrum	11
1.5.2	Justification du choix	12
1.5.3	Adaptation au contexte d'un stage individuel	12
2	Spécification et analyse des besoins	14
2.1	Spécification des besoins	14
2.1.1	Identification des acteurs	14
2.1.2	Besoins fonctionnels	16
2.1.2.1	Fonctions communes aux utilisateurs authentifiés	16
2.1.2.2	Super Administrateur	17
2.1.2.3	Administrateur d'organisation	17
2.1.2.4	Opérateur	18
2.1.2.5	Technicien	19
2.1.2.6	Client	19
2.1.2.7	Visiteur et traitements automatiques	20
2.1.3	Besoins non fonctionnels	20
2.2	Planification du projet	23
2.2.1	Méthode de priorisation et d'estimation	23
2.2.2	Product Backlog révisé	23
2.2.3	Planification des Sprints	26
2.2.4	Matrice de traçabilité fonctionnelle	28
3	Conception	30
3.1	Conception générale	30
3.1.1	Architecture physique	30
3.1.1.1	Architecture client-serveur	30
3.1.1.2	Modèle trois tiers	31
3.1.2	Architecture logique	32

3.1.2.1	Styles et patrons de conception	32
3.1.2.2	Architecture technique	32
3.1.3	Description des composants	34
3.1.3.1	Application web React	34
3.1.3.2	API et coeur métier Laravel	34
3.1.3.3	Persistance, asynchronisme et temps réel	35
3.1.3.4	Passerelle OCPP	35
3.1.3.5	Services externes et adaptateurs	36
3.1.4	Architecture de déploiement	36
3.1.5	Architecture logicielle	38
3.1.5.1	Organisation du frontend	38
3.1.5.2	Organisation du backend	39
3.2	Conception fonctionnelle et scénarios d'utilisation	39
3.2.1	Diagramme global	40
3.2.2	Catalogue des cas majeurs	42
3.2.3	UC-01 – Demander une démonstration et provisionner une organisation	43
3.2.4	UC-02 – Inviter et activer un employé	46
3.2.5	UC-03 – Mettre en service une station OCPP	48
3.2.6	UC-04 – Calculer la disponibilité	51
3.2.7	UC-05 – Effectuer une recharge et payer	53
3.2.8	UC-06 – Traiter une alerte et une intervention	56
3.2.9	UC-07 – Gérer le cycle commercial d'une organisation	58
3.2.10	UC-08 – Générer et échanger un rapport	60
3.3	Conception détaillée	62
3.3.1	Identité, demandes de démonstration et invitations	62
3.3.2	Mise en service d'une station	63
3.3.3	Ingestion OCPP et disponibilité calculée	64
3.3.3.1	Séparation entre preuve brute et projection métier	64

3.3.3.2	Règles de disponibilité	64
3.3.4	Parcours de recharge et transaction OCPP	65
3.3.4.1	Assistant côté client	65
3.3.4.2	Tentative, session et transaction	66
3.3.5	Paieement et reçu	67
3.3.6	Alertes, interventions et maintenance	67
3.3.7	Tarification, plans et gestion commerciale	68
3.3.8	Rapports, documents et exports	69
3.4	Sécurité et isolation multi-tenant	69
3.4.1	Authentification de l'application web	69
3.4.2	Autorisation et périmètre organisationnel	69
3.4.3	Sécurité des frontières OCPP	70
3.4.4	Sécurité du paiement	71
3.4.5	Validation, limitation et gestion des erreurs	71
3.4.6	Secrets, fichiers et journalisation	72
3.4.7	Analyse synthétique des risques	72
3.4.8	Limites de sécurité du prototype	73
3.5	Modèle de données et diagramme de classes	74
3.5.1	Principes de modélisation	74
3.5.2	Domaines fonctionnels	74
3.5.2.1	Identité et séparation organisationnelle	74
3.5.2.2	Réseau et recharge	74
3.5.2.3	Exploitation	75
3.5.2.4	Commerce client et commerce SaaS	75
3.5.3	Contraintes et invariants principaux	75
3.5.4	Diagramme de classes global	78
4	Réalisation et validation	80
4.1	Environnement et organisation de la réalisation	80

4.1.1	Poste, outils et versions	80
4.1.2	Structure du dépôt	82
4.1.3	Processus de développement	82
4.1.4	Suivi des Sprints avec Git	83
4.1.5	État synthétique des objectifs de Sprint	84
4.2	Réalisation des fondations techniques	85
4.2.1	Sprint 2 : entrée, identité et multi-tenant	85
4.2.1.1	Parcours d'entrée	85
4.2.1.2	Isolation organisationnelle	86
4.2.1.3	Onboarding orienté vers la première valeur	86
4.2.2	Sprint 3 : stations, connecteurs et cartographie	86
4.2.2.1	Référentiel et vues adaptées	86
4.2.2.2	Commissioning atomique	87
4.2.3	Sprint 4 : OCPP et disponibilité calculée	89
4.2.3.1	Connexion de la borne simulée	89
4.2.3.2	Règles de projection	90
4.2.3.3	Commandes et télémétrie	91
4.3	Réalisation des parcours métier	92
4.3.1	Sprint 6 : exploitation terrain	92
4.3.1.1	Alertes et qualification	92
4.3.1.2	Interventions, maintenance et SLA	92
4.3.2	Sprint 5 : recharge, transaction et paiement	93
4.3.2.1	Assistant de recharge client	93
4.3.2.2	Cycle de session	93
4.3.2.3	Adaptateur de paiement simulé	94
4.3.3	Sprint 7 : tarification et commerce	94
4.3.3.1	Tarifs et plans de recharge	94
4.3.3.2	Abonnement SaaS de l'organisation	95
4.3.4	Sprint 8 : pilotage, reporting et gouvernance	95

4.3.4.1	Tableaux de bord par rôle	95
4.3.4.2	Rapports et échanges internes	95
4.3.4.3	Administration globale	96
4.3.5	Sprint 9 : exports et évolutions différées	96
4.4	Intégration et validation	96
4.4.1	Intégration bout en bout	96
4.4.2	Stratégie de tests	97
4.4.3	Résultats mesurés	98
4.4.4	Anomalies représentatives et non-régression	99
4.4.5	Performance et robustesse	99
4.4.6	Sécurité appliquée	100
4.4.7	Limites de la validation	100
4.4.8	Traçabilité avec le backlog et les Sprints	101
4.5	Documentation technique produite et restante	102
5	Captures des interfaces	104
5.1	Page d'accueil	104
5.2	Entrée et prise en main	105
5.3	Navigation et adaptation aux rôles	106
5.4	Pilotage de l'organisation	107
5.5	Stations et exploitation opérationnelle	108
5.6	Gestion commerciale et reporting	110
5.7	Principes d'ergonomie retenus	112
	Conclusion générale	114

Table des figures

1.1	Cycle Scrum adopté pour le projet	12
3.1	Synthèse des styles architecturaux et patrons retenus	31
3.2	Architecture technique globale de ChargeTrackr	33
3.3	Architecture de déploiement de l'environnement local	37
3.4	Diagramme global des cas d'utilisation de ChargeTrackr	41
3.5	Provisionnement d'une organisation à partir d'une demande de démonstration	62
3.6	Chaîne de traitement d'un événement OCPP	64
3.8	Cycle opérationnel des alertes, interventions et maintenances	67
3.7	Séquence de recharge distante et paiement simulé	77
3.9	Diagramme de classes conceptuel global de ChargeTrackr	78
4.1	Première étape du commissioning guidé d'une station	88
4.2	Fiche d'une station reliée au simulateur SAP	89
4.3	Télémétrie vérifiée sans génération de mesures fictives	91
4.4	Parcours vertical d'une recharge	93
4.5	Composants traversés par le MVP vertical	97
5.1	Page d'accueil publique de ChargeTrackr	105
5.2	Onboarding personnalisé de l'administrateur	106
5.3	Tableau de bord de l'administrateur d'organisation	108
5.4	Inventaire des stations avec disponibilité calculée	109
5.5	Centre opérationnel des alertes	110
5.6	Suivi de l'abonnement commercial de l'organisation	111
5.7	Analyse et rapports de l'administrateur	112

Liste des tableaux

1.1	Comparaison synthétique des solutions étudiées	9
1.2	Espaces et responsabilités des utilisateurs	10
2.1	Acteurs humains de la plateforme	15
2.2	Acteurs secondaires et systèmes externes	16
2.10	Exigences non fonctionnelles	21
2.11	Product Backlog de ChargeTrackr	23
2.12	Répartition du backlog par priorité	26
2.13	Synthèse des Sprint Backlogs	27
2.14	Traçabilité entre backlog, API et interfaces	28
3.1	Intégrations techniques et responsabilités	36
3.2	Processus nécessaires dans l'environnement de développement	38
3.3	Catalogue des cas d'utilisation majeurs	42
3.4	Cibles de mise en service OCPP	63
3.5	Matrice de décision de disponibilité	64
3.6	Périmètre d'accès des rôles	70
3.7	Principales limites applicatives	72
3.8	Risques principaux et mesures de réduction	73
3.9	Invariants structurants du modèle	75
4.1	Environnement technique effectivement utilisé	81
4.2	Commandes principales de l'environnement local	83
4.3	État de réalisation au 28 juillet 2026	84
4.4	Exemples de règles de disponibilité réalisées	90
4.5	Niveaux de validation appliqués	97

4.6	Résultats de validation du dépôt	98
4.7	Preuves de réalisation par Sprint	101
5.1	Orientation des interfaces selon le rôle	107

Liste des acronymes

Acronyme	Signification
API	Application Programming Interface
CSMS	Charging Station Management System
EV	Electric Vehicle
EVSE	Electric Vehicle Supply Equipment
OCPP	Open Charge Point Protocol
OAuth 2.0	Open Authorization 2.0
REST	Representational State Transfer
RFID	Radio Frequency Identification
SLA	Service Level Agreement
UI/UX	User Interface / User Experience
UML	Unified Modeling Language

Introduction générale

La transition vers la mobilité électrique entraîne le déploiement d'un nombre croissant de bornes de recharge. L'exploitation de ces équipements ne repose pas uniquement sur leur installation physique : elle exige également une supervision continue, une connaissance fiable de leur disponibilité et une coordination efficace entre les utilisateurs, les opérateurs et les techniciens.

Dans ce contexte, le présent stage d'été, réalisé au sein de TAC-TIC Tunisie, porte sur la conception et le développement de ChargeTrackr, une plateforme web consacrée à la visualisation et au suivi de la disponibilité des bornes de recharge électrique. Le projet associe gestion multi-organisation, échanges OCPP, cartographie, sessions de recharge, paiement simulé, alertes, interventions et reporting.

Le rapport conserve l'organisation en cinq chapitres du document de référence. Le premier chapitre introduit le cadre du projet, le domaine métier, la problématique, l'existant et la démarche retenue. Le deuxième formalise les acteurs, les besoins et la planification. Le troisième présente l'architecture et la conception détaillée. Le quatrième décrit la réalisation menée au fil des Sprints, l'intégration et la validation. Enfin, le cinquième illustre les principales interfaces et les parcours adaptés aux différents rôles.

Sur le plan pédagogique, ce stage vise à mobiliser les acquis de la première année du cycle ingénieur : analyse des besoins, modélisation UML, architecture logicielle, développement frontend et backend, sécurité, tests et gestion d'un projet avec Scrum. Sur le plan professionnel, l'objectif est de transformer un sujet fonctionnel en une solution démontrable, structurée et maintenable, tout en justifiant les décisions techniques et les compromis retenus.

Le périmètre validé couvre l'étude métier, le Product Backlog, la conception, la réalisation de l'application multi-rôle, l'intégration d'un simulateur OCPP 1.6J, le calcul de disponibilité, le parcours de recharge, le paiement externe simulé et la validation automatisée. Le travail présenté comprend également l'environnement local, les données de démonstration et la documentation nécessaire à la reproduction du MVP.

Les essais sur une borne physique, le raccordement à un prestataire de paiement agréé, le déploiement de production, OCPP 2.0.1 et la gestion distante du firmware ne sont pas présentés comme des fonctionnalités terminées. Ils constituent des prolongements qui

exigent du matériel, des contrats externes ou des validations de sécurité dépassant le cadre du MVP réalisé pendant le stage.

État de l'art et fondements métiers

Introduction

Ce chapitre présente le cadre académique et professionnel du projet, puis introduit le domaine de la recharge électrique et les principales difficultés liées à la supervision d'un réseau de bornes. L'étude de l'existant met en évidence les services habituellement proposés par les plateformes du secteur et leurs limites au regard du sujet de stage. La solution retenue est ensuite décrite sous un angle fonctionnel, avant de présenter la méthodologie Scrum adoptée et son adaptation à un travail individuel.

1.1 Cadre du projet

1.1.1 Cadre académique

Le présent travail est réalisé dans le cadre d'un stage d'été effectué du 1^{er} juillet au 31 août 2026 au sein de TAC-TIC Tunisie. Il intervient après la première année du cycle ingénieur en génie logiciel (IGL3), au Département des Sciences de l'Informatique de la Faculté des Sciences de Tunis, rattachée à l'Université de Tunis El Manar.

Le stage vise à mettre en pratique les connaissances acquises en analyse des besoins, modélisation, développement d'applications web, conception d'API, gestion de bases de données, sécurité et tests logiciels. Le sujet demande également d'aborder une problématique de système communicant : les informations présentées aux utilisateurs dépendent d'événements produits par des équipements distants et doivent rester cohérentes malgré les erreurs, les retards ou les interruptions de communication.

Le travail est encadré en entreprise par M. Bassem Soua. Le résultat attendu est une application fonctionnelle permettant de visualiser et de suivre la disponibilité de bornes de recharge, tout en couvrant les principaux parcours d'exploitation et d'utilisation associés.

1.1.2 Présentation de l'organisme d'accueil

TAC-TIC Tunisie est une société d'ingénierie indépendante fondée en 2012 et implantée à la Cité El Ghazala, dans le gouvernorat de l'Ariana. Sa présentation institutionnelle la positionne comme une entreprise spécialisée dans les technologies Open Source, les services réseau, le conseil, les solutions outillées et la formation. Elle mentionne également une expertise dans les services IP/MPLS destinés au secteur tunisien des télécommunications, ainsi que des activités de développement web et logiciel [1].

Ce positionnement rend pertinent un sujet associant équipements communicants, interopérabilité et développement d'une plateforme web. Le stage s'inscrit ainsi dans un environnement où la qualité du logiciel dépend autant de la conception de l'application que de sa capacité à échanger de manière fiable avec des systèmes externes.

1.1.3 Présentation du domaine d'application

1.1.3.1 Mobilité électrique et infrastructure de recharge

Le développement de la mobilité électrique augmente la dépendance des conducteurs, des entreprises et des collectivités à des infrastructures de recharge accessibles et fiables. Selon l'Agence internationale de l'énergie, près de 1,8 million de points de recharge publics ont été ajoutés dans le monde en 2025, portant le parc total à plus de sept millions à la fin de cette année [2]. Dans ce contexte, la disponibilité d'une infrastructure devient un indicateur essentiel : une station référencée mais inaccessible, hors service ou entièrement occupée ne répond pas au besoin immédiat du conducteur.

Une infrastructure de recharge s'organise généralement autour des éléments suivants :

- la **station de recharge**, qui représente l'équipement physique installé sur un site et raccordé au réseau électrique ;
- le **connecteur**, qui constitue le point de raccordement utilisé par le véhicule. Une station peut proposer plusieurs connecteurs de types et de puissances différents ;
- la **session de recharge**, qui décrit l'utilisation d'un connecteur entre le début et la fin d'une opération, avec une durée, une quantité d'énergie et un coût éventuel ;
- le **système central de gestion**, chargé de suivre les équipements, d'autoriser certaines opérations et de conserver les informations utiles à l'exploitation ;
- les **acteurs humains**, notamment les conducteurs, les exploitants, les techniciens et les responsables des organisations.

Ces éléments sont interdépendants. L'état global d'une station dépend de sa capacité à communiquer et de l'état de ses connecteurs. Une session produit, quant à elle, des informations utiles à la facturation, aux statistiques, au support et à la maintenance.

1.1.3.2 Système central de gestion des bornes

Le système central chargé de piloter un réseau de recharge est couramment appelé *Charging Station Management System* (CSMS). Il maintient une représentation du réseau, reçoit les informations des stations, conserve les transactions, applique les politiques d'accès et peut transmettre des commandes à distance. Il fournit ensuite des vues adaptées aux besoins des acteurs : carte des stations, tableau de bord opérationnel, alertes, historique des sessions, interventions, tarification, paiements et rapports.

La supervision d'un réseau doit prendre en compte trois temporalités complémentaires :

1. le **temps de communication**, constitué des connexions et messages échangés avec les équipements ;
2. le **temps opérationnel**, durant lequel une anomalie est détectée, qualifiée, assignée puis résolue ;
3. le **temps analytique**, utilisé pour étudier l'énergie, la disponibilité, les revenus et les performances sur une période.

La qualité d'un CSMS dépend donc de sa capacité à distinguer l'état courant, l'historique des événements et les indicateurs agrégés. Une information ancienne ne doit pas être présentée comme une preuve actuelle de fonctionnement.

1.1.3.3 Interopérabilité avec les bornes

Les stations de recharge peuvent provenir de constructeurs différents. Sans protocole commun, chaque intégration nécessiterait un mécanisme propriétaire, ce qui augmenterait les coûts et limiterait l'évolution du réseau. L'*Open Charge Point Protocol* (OCPP), maintenu par l'Open Charge Alliance, standardise les échanges entre une station et un système central [3, 4].

OCPP couvre notamment l'enregistrement d'une station, la preuve périodique de connectivité, la transmission de l'état des connecteurs, l'autorisation d'un utilisateur, le déroulement d'une transaction, les mesures d'énergie et certaines commandes distantes. Il contribue ainsi à l'interopérabilité du réseau, mais ne définit pas toutes les règles fonctionnelles de la plateforme. Par exemple, la durée au-delà de laquelle une station

est considérée comme déconnectée, la priorité donnée à une maintenance ou la manière d'agréger l'état de plusieurs connecteurs restent des décisions du système central.

1.1.3.4 Disponibilité et qualité de service

La disponibilité d'une station ne peut pas être réduite à une valeur saisie manuellement. Elle résulte d'un ensemble de preuves : présence d'une connexion, réception récente d'un message, état des connecteurs, transaction en cours, défaut déclaré, réservation ou maintenance planifiée. L'Open Charge Alliance recommande d'exploiter conjointement la connexion, les messages de présence et les notifications d'état afin d'améliorer la mesure du temps de fonctionnement [5].

Sur le plan métier, plusieurs situations doivent être distinguées :

- une station **disponible** possède au moins un connecteur utilisable ;
- une station **occupée** communique correctement, mais ses connecteurs utilisables sont engagés dans des sessions ;
- une station **réservée** ne peut temporairement pas être proposée à un autre conducteur ;
- une station **en défaut** signale une anomalie nécessitant un diagnostic ;
- une station **en maintenance** est volontairement retirée du service ;
- une station **hors ligne** ne fournit plus de preuve de communication suffisamment récente.

La valeur affichée doit être explicable et accompagnée d'une date de dernière observation. Cette exigence protège le conducteur contre un déplacement inutile et permet à l'exploitant de différencier une occupation normale d'une panne ou d'une perte de communication.

1.2 Problématique

La difficulté principale ne consiste pas à afficher des stations sur une carte ou à stocker une colonne d'état. Une station peut être alimentée mais déconnectée du système, connectée mais sans statut récent, disponible au niveau global mais en défaut sur un connecteur, ou entièrement occupée. Une valeur ancienne conservée dans la base peut alors induire l'utilisateur en erreur.

Cette incertitude affecte l'ensemble des acteurs :

- le client risque de se déplacer vers une borne inutilisable ;
- l'opérateur peut détecter trop tard une perte de communication ;
- le technicien peut recevoir une intervention sans historique ni contexte ;
- l'administrateur manque d'indicateurs fiables pour piloter son organisation ;
- le responsable de la plateforme doit garantir le fonctionnement global et l'isolation des données de chaque organisation.

À ces difficultés s'ajoutent l'hétérogénéité des équipements, la gestion de plusieurs organisations, les autorisations, la tarification, la traçabilité des actions et la nécessité de tester les parcours sans disposer en permanence d'une borne physique ou d'un prestataire financier réel.

Problématique centrale

Comment concevoir une plateforme web multi-rôle capable de visualiser et de suivre de manière fiable la disponibilité d'un réseau de bornes, en transformant les événements techniques en informations et parcours métier compréhensibles, tout en restant sécurisée, traçable, testable et extensible ?

La réponse attendue doit relier les équipements et les usages. Elle doit garantir l'origine et la fraîcheur de chaque état, présenter à chaque acteur les fonctions correspondant à ses responsabilités et couvrir un parcours cohérent allant de la recherche d'une station à l'exploitation des informations produites.

1.3 Étude de l'existant

1.3.1 Solutions existantes

1.3.1.1 ChargePoint

ChargePoint propose une plateforme commerciale de gestion de recharge incluant la visibilité en temps réel, des tableaux de bord, des rapports ainsi que la gestion des accès et des prix. Elle illustre l'intérêt d'unifier la supervision du réseau, les politiques d'utilisation et l'analyse dans un même environnement [6]. Son caractère propriétaire ne permet toutefois pas d'étudier ou d'adapter librement ses règles internes.

1.3.1.2 Driivz

Driivz se positionne comme une plateforme de gestion de réseaux de recharge à grande échelle. Sa documentation présente la surveillance en temps réel, les alertes, la gestion des incidents, la facturation, les services destinés aux conducteurs et des capacités multi-tenant [7]. Cette couverture confirme que disponibilité, maintenance et gestion commerciale doivent partager une information cohérente.

1.3.1.3 Monta

Monta Hub fournit aux opérateurs un environnement de gestion des points de recharge avec suivi d'état, contrôle d'accès, configuration des prix, automatisation opérationnelle et rapports [8]. La solution met particulièrement l'accent sur la simplicité des parcours proposés aux équipes et aux conducteurs. Elle reste cependant une offre commerciale dont le fonctionnement interne n'est pas destiné à être adapté dans un contexte pédagogique.

1.3.1.4 CitrineOS

CitrineOS est un système central Open Source porté par LF Energy. Il fournit une base modulaire pour le provisionnement, le contrôle, les transactions et le suivi des équipements [9]. Il constitue une référence pertinente pour comprendre la structure d'un backend de recharge extensible. En revanche, il ne définit pas à lui seul les espaces utilisateurs, les règles organisationnelles ni l'expérience métier d'un produit complet.

1.3.2 Synthèse comparative

Le tableau 1.1 synthétise les caractéristiques utiles au sujet de stage à partir des informations publiées par les éditeurs. Les identités visuelles reproduites proviennent de leurs ressources officielles [6, 7, 8, 9].

Table 1.1 – Comparaison synthétique des solutions étudiées

Solution	Nature	Fonctions principales	Limites pour le sujet de stage
 ChargePoint	Plateforme commerciale	Supervision, accès, tarification, tableaux de bord et rapports.	Fonctionnement interne propriétaire et personnalisation limitée au cadre proposé par l'éditeur.
 Driivz	Plateforme commerciale	Monitoring, alertes, incidents, conducteurs, facturation et analytique.	Solution destinée aux réseaux industriels, non modifiable pour l'étude des règles internes.
 Monta Hub	Plateforme commerciale	État des points, accès, prix, automatisation, parcours conducteur et rapports.	Logique propriétaire et dépendance au modèle de service de l'éditeur.
 CitrineOS	CSMS Open Source	Provisionnement, contrôle, transactions et suivi des équipements.	Briques techniques à intégrer dans un produit métier et une expérience utilisateur complète.

L'étude montre qu'une solution de gestion de recharge doit réunir deux qualités. Premièrement, l'état opérationnel doit provenir de preuves techniques récentes et rester explicable. Deuxièmement, cette information doit être transformée en services adaptés au conducteur, à l'exploitation, à la maintenance et au pilotage. Les solutions commerciales couvrent largement ces besoins, tandis que CitrineOS apporte une référence ouverte pour le système central. Aucune ne constitue toutefois, à elle seule, un support directement adapté au périmètre pédagogique et organisationnel du stage.

1.4 Solution proposée

1.4.1 Vue d'ensemble fonctionnelle

La solution proposée, nommée ChargeTrackr, est une plateforme web de visualisation, de suivi et de gestion d'infrastructures de recharge électrique. Elle met en relation les informations provenant des stations avec les besoins des différents utilisateurs. Son objectif principal est de présenter une disponibilité fiable tout en assurant la continuité entre supervision, recharge, paiement, maintenance et pilotage.

La plateforme couvre les fonctions suivantes :

- présenter le service au public et permettre la consultation des stations ;
- gérer les organisations, les comptes, les rôles et les invitations ;
- administrer les stations, leurs connecteurs et leurs informations de localisation ;

- recevoir les événements des équipements et en déduire leur état opérationnel ;
- permettre au client de rechercher une station, démarrer une recharge, suivre sa session et obtenir une preuve de paiement ;
- traiter les alertes, interventions et opérations de maintenance ;
- configurer les tarifs, les abonnements et le cycle commercial des organisations ;
- produire des indicateurs, rapports et exports adaptés aux responsabilités de chaque utilisateur.

1.4.2 Espaces et responsabilités des utilisateurs

L'application ne propose pas un tableau de bord générique. Chaque espace est limité aux responsabilités de son utilisateur, comme le résume le tableau 1.2.

Table 1.2 – Espaces et responsabilités des utilisateurs

Acteur	Périmètre	Responsabilités principales
Visiteur	Espace public	Découvrir la plateforme, consulter les stations publiques, contacter le service, demander une démonstration ou créer un compte client.
Super administrateur	Ensemble de la plateforme	Superviser les organisations, traiter les demandes de démonstration, contrôler les paramètres globaux, les intégrations et l'audit.
Administrateur	Une organisation	Piloter les employés, les clients liés à l'organisation, les stations, la tarification, l'abonnement commercial et les rapports.
Opérateur	Réseau de son organisation	Surveiller les stations, traiter les alertes, exécuter les commandes autorisées et assigner les interventions.
Technicien	Interventions de son organisation	Consulter les stations concernées, réaliser les opérations affectées, documenter le travail et mettre à jour son avancement.
Client	Compte personnel multi-réseau	Rechercher une station, suivre ses sessions, payer, consulter ses reçus et gérer ses adhésions aux organisations.

Les employés, à savoir les administrateurs, opérateurs et techniciens, appartiennent à une seule organisation. Un client possède en revanche une identité indépendante et peut

utiliser plusieurs réseaux ou souscrire à plusieurs offres sans créer plusieurs comptes. Cette séparation évite de confondre la propriété d'un actif, l'appartenance d'un employé et la relation commerciale d'un conducteur.

1.4.3 Périmètre du MVP

Le produit minimum viable doit démontrer un parcours complet et cohérent :

Équipement simulé → événements de terrain → disponibilité
→ carte et alertes → session de recharge → tarification
→ paiement simulé → reçu

Le MVP inclut également les fonctions indispensables à l'exploitation de ce parcours : authentification, rôles, organisations, invitations, interventions, maintenance, notifications, tableaux de bord et rapports. Il ne prétend pas certifier un matériel physique ni traiter immédiatement de l'argent réel. La connexion à une borne réelle, l'intégration d'un prestataire financier en production, la gestion distante du firmware et les fonctions prédictives constituent des perspectives d'évolution.

1.5 Méthodologie de travail

1.5.1 Le framework Scrum

Scrum est un framework agile fondé sur des cycles courts appelés *Sprints*. Il organise le travail autour d'un *Product Backlog* ordonné, d'un objectif de Sprint, d'un *Sprint Backlog* et d'un résultat utilisable inspecté à la fin de chaque cycle. Ses piliers sont la transparence, l'inspection et l'adaptation [10].

Le fonctionnement général adopté est représenté par la figure 1.1. Les retours obtenus lors de la revue alimentent le Product Backlog, tandis que la rétrospective permet d'améliorer la manière de travailler pour le Sprint suivant.

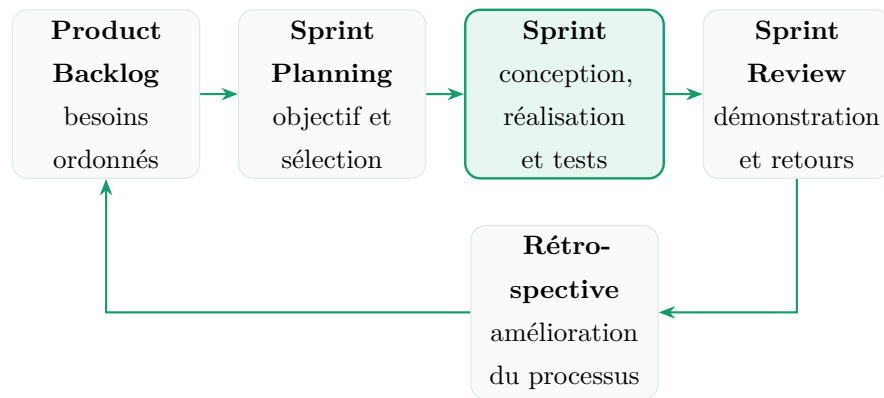


Figure 1.1 – Cycle Scrum adopté pour le projet

1.5.2 Justification du choix

Une démarche strictement séquentielle aurait nécessité de figer très tôt les règles de disponibilité, les échanges avec les stations et les parcours utilisateurs. Or ces éléments ont besoin d'être confrontés rapidement à des scénarios exécutables. Scrum a été retenu afin de :

- construire progressivement une chaîne fonctionnelle démontrable ;
- intégrer les remarques de l'encadrant sans attendre la fin du stage ;
- réordonner le backlog lorsqu'une dépendance métier ou technique apparaît ;
- associer conception, implémentation et validation dans un même cycle ;
- conserver une visibilité régulière sur les résultats obtenus et le travail restant.

Les dépendances entre fonctionnalités ne s'opposent pas à Scrum. Un Sprint peut s'appuyer sur une fondation produite précédemment, à condition que son objectif soit clair et que le résultat obtenu soit vérifiable. Le détail des Sprints et des user stories associées est présenté au chapitre suivant.

1.5.3 Adaptation au contexte d'un stage individuel

Le projet est développé par un seul stagiaire. Scrum est donc adapté sans créer artificiellement une équipe ou attribuer des titres qui ne correspondent pas à l'organisation réelle :

- le stagiaire assure l'analyse, la conception, le développement, les tests et la tenue du Product Backlog ;

- l'encadrant d'entreprise joue le rôle de **validateur métier** : il clarifie les attentes, examine les résultats et fournit les retours nécessaires ;
- le Sprint Planning prend la forme d'une sélection d'objectifs réalistes et de critères d'acceptation avant le démarrage du travail ;
- la Sprint Review correspond à la démonstration du scénario réalisé ;
- la rétrospective est une analyse courte des difficultés, des écarts et des améliorations à appliquer au cycle suivant.

La cadence retenue est hebdomadaire. Avant chaque Sprint, les règles métier, les sources externes et les critères d'acceptation sont précisés. Pendant le Sprint, les contrats d'API permettent de faire progresser le frontend et le backend de manière coordonnée. La fin du Sprint comprend les tests automatisés ciblés, une vérification visuelle, une démonstration et le versionnement des changements dans Git.

Cette adaptation conserve les mécanismes utiles de Scrum — backlog ordonné, objectif court, inspection et adaptation — tout en restant fidèle au contexte réel du stage.

Conclusion

Ce chapitre a présenté le contexte du stage et les fondements d'un système de gestion des bornes de recharge. L'étude du domaine montre que la disponibilité doit être déduite de preuves récentes et interprétée selon l'état des équipements. L'analyse de ChargePoint, Driivz, Monta et CitrineOS confirme également la nécessité de relier supervision, expérience client, maintenance et pilotage. En réponse à cette problématique, ChargeTrackr propose des espaces spécialisés et un parcours métier complet. Le projet est conduit selon Scrum, adapté à un stage individuel et organisé en Sprints courts. Le chapitre suivant formalise les besoins, le Product Backlog et la planification de ces Sprints.

Spécification et analyse des besoins

Introduction

Ce chapitre transforme le cahier des charges en un périmètre vérifiable. Il identifie les acteurs humains et externes, formalise les besoins fonctionnels et non fonctionnels, puis présente le Product Backlog révisé et sa planification en Sprints. Les interactions, scénarios détaillés et diagrammes associés seront étudiés dans le chapitre de conception afin de séparer clairement la spécification de la modélisation de la solution.

2.1 Spécification des besoins

2.1.1 Identification des acteurs

Un acteur représente une entité extérieure au système qui poursuit un objectif au moyen de celui-ci. Une base de données, un worker de file ou le scheduler Laravel ne sont donc pas des acteurs : ils appartiennent à l'implémentation interne. En revanche, une borne, Google OAuth ou un prestataire de paiement sont des acteurs secondaires, car ils échangent avec la plateforme par une interface clairement délimitée.

La notion d'**utilisateur authentifié** est utilisée comme acteur abstrait pour regrouper les capacités communes : connexion, profil, paramètres personnels, onboarding et notifications. Les cinq rôles authentifiés spécialisés restent le Super Administrateur, l'Administrateur, l'Opérateur, le Technicien et le Client.

Table 2.1 – Acteurs humains de la plateforme

Acteur	Catégorie	Responsabilités et périmètre
Visiteur	Principal	Consulte la présentation publique, les informations publiques des bornes, contacte la plateforme, crée un compte client ou dépose une demande de démonstration.
Client	Principal	Utilisateur global sans organisation fixe. Il peut utiliser les bornes de plusieurs organisations, gérer ses abonnements, lancer ses propres sessions, payer et consulter uniquement ses données.
Opérateur	Principal	Employé d’une seule organisation. Il met en service les stations, supervise OCPP, traite les alertes, exécute les commandes distantes, assigne les interventions et planifie les maintenances dans son périmètre.
Technicien	Principal	Employé d’une seule organisation. Il consulte les actifs en lecture, traite uniquement les interventions et maintenances qui lui sont assignées, puis produit les preuves et rapports techniques.
Administrateur	Principal	Responsable d’une seule organisation. Il gère ses opérateurs et techniciens, ses clients observés, ses actifs, tarifs, documents, rapports et son abonnement commercial, sans administrer la plateforme globale.
Super Administrateur	Principal	Gère les organisations et leurs administrateurs, les demandes de démonstration, les offres SaaS, l’audit, les rôles, les paramètres globaux et l’état des intégrations.

Table 2.2 – Acteurs secondaires et systèmes externes

Acteur	Nature	Interaction avec ChargeTrackr
Borne ou simulateur OCPP	Système externe	Ouvre une connexion WebSocket authentifiée, émet les messages OCPP, reçoit les commandes distantes et retourne leur résultat.
Google OAuth	Fournisseur d'identité	Authentifie un compte Google vérifié sans transmettre son mot de passe à la plateforme.
Service de paiement	Service externe	Autorise, capture ou libère un montant et envoie des webhooks signés. Dans le MVP, ce service est simulé mais reste hors du domaine métier.
Service email	Service externe	Distribue les vérifications, invitations, réinitialisations et notifications transactionnelles placées dans la file.
Service cartographique	Service externe	Fournit les tuiles et données nécessaires à l'affichage de la carte et à l'ouverture d'un itinéraire.

Clarification du rôle financier

La *finance* n'est pas modélisée comme un acteur autonome. Les décisions commerciales sont prises par le Super Administrateur ou l'Administrateur selon leur périmètre. Le seul acteur externe du paiement est le prestataire technique qui traite une transaction. Cette distinction évite de confondre une fonction métier, un rôle humain et un système tiers.

2.1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit permettre de réaliser. Ils sont regroupés par acteur, mais les contrôles d'autorisation restent appliqués côté API même lorsqu'une action n'est pas affichée dans l'interface.

2.1.2.1 Fonctions communes aux utilisateurs authentifiés

Code	Besoin fonctionnel
BF-01	Se connecter par email et mot de passe ou, lorsque le compte le permet, par Google OAuth ; se déconnecter et récupérer son accès.
BF-02	Consulter et modifier les données personnelles autorisées, l'adresse, l'avatar et les liens professionnels pertinents.
BF-03	Configurer le fuseau horaire, le rayon de recherche, les préférences de notification et le mot de passe lorsqu'un mot de passe local existe.
BF-04	Suivre un onboarding court adapté au rôle, puis retrouver des indications contextuelles sans être bloqué dans une visite générique.
BF-05	Consulter les notifications personnelles, leur contexte, leur statut lu ou non lu et marquer un ensemble cohérent comme lu.
BF-06	Utiliser une recherche globale limitée aux objets réellement accessibles.

2.1.2.2 Super Administrateur

Code	Besoin fonctionnel
BF-07	Qualifier, rejeter ou convertir une demande de démonstration en organisation d'essai et invitation d'administrateur.
BF-08	Créer, consulter, modifier, activer ou suspendre les organisations et gérer leurs administrateurs.
BF-09	Consulter les indicateurs globaux, l'audit filtrable et la matrice des rôles et permissions.
BF-10	Consulter l'état des intégrations sans exposer leurs secrets et modifier uniquement les paramètres globaux autorisés.
BF-11	Gérer les offres SaaS, essais, capacités, abonnements et factures des organisations, avec historique des transitions.
BF-12	Produire les analyses et exports globaux de la plateforme.

2.1.2.3 Administrateur d'organisation

Code	Besoin fonctionnel
------	--------------------

BF-13	Inviter, modifier, désactiver ou supprimer les opérateurs et techniciens de sa seule organisation, sans connaître leur mot de passe.
BF-14	Consulter les clients ayant réellement utilisé les bornes de l'organisation, sans modifier leur compte global ni leurs activités chez d'autres opérateurs.
BF-15	Administrer les stations, connecteurs, commandes, alertes, interventions, maintenances et documents de son organisation.
BF-16	Définir les tarifs, plans clients et affectations applicables aux stations ou connecteurs de son organisation.
BF-17	Consulter les indicateurs métier, produire et échanger des rapports et exporter les données autorisées.
BF-18	Suivre l'offre commerciale, les quotas, les échéances et factures de son organisation et demander un changement d'offre.

2.1.2.4 Opérateur

Code	Besoin fonctionnel
BF-19	Mettre en service une station et ses connecteurs de manière atomique, la positionner sur la carte et préparer sa liaison à une borne physique ou simulée.
BF-20	Consulter les stations en vue carte, liste ou tableau avec disponibilité calculée, connexion, dernière preuve de vie et alertes.
BF-21	Examiner une station, sa télémétrie vérifiée, ses connecteurs, événements, commandes, sessions, maintenances, alertes et documents.
BF-22	Exécuter les commandes OCPP autorisées : redémarrage logiciel, déverrouillage et changement de disponibilité pour maintenance.
BF-23	Filtrer, acquitter, assigner, suivre ou annuler une alerte dans le respect de son cycle de vie et de son SLA.
BF-24	Créer et suivre les interventions et maintenances préventives ou correctives, y compris leur récurrence et leur calendrier.

BF-25	Superviser les sessions et paiements de l'organisation sans agir à la place d'un client sur ses données privées.
BF-26	Consulter et échanger des rapports opérationnels adaptés à son rôle.

2.1.2.5 Technicien

Code	Besoin fonctionnel
BF-27	Consulter en lecture les stations, connecteurs, cartes et informations techniques de son organisation.
BF-28	Consulter les alertes, interventions et maintenances qui lui sont assignées, avec priorité, échéance et historique.
BF-29	Faire progresser une intervention selon les transitions autorisées, ajouter des notes et téléverser des preuves avant/après privées.
BF-30	Produire un rapport final comprenant diagnostic, actions, pièces, contrôles terrain et résultat, puis demander un suivi lorsque nécessaire.
BF-31	Consulter ses propres indicateurs de terrain et échanger des rapports avec les destinataires autorisés de son organisation.

2.1.2.6 Client

Code	Besoin fonctionnel
BF-32	Rechercher les stations publiques des organisations actives, utiliser la géolocalisation selon un rayon choisi et filtrer les résultats.
BF-33	Consulter les connecteurs et le tarif effectif, copier les coordonnées ou ouvrir un itinéraire vers la station sélectionnée.
BF-34	Démarrer une recharge guidée : choisir station et connecteur, brancher le câble, attendre sa détection OCPP, autoriser le paiement puis lancer la commande.
BF-35	Suivre en temps réel la durée, l'énergie, la puissance et le coût provisoire, puis arrêter la session manuellement ou par une limite configurée.

BF-36	Consulter l'historique multi-organisations de ses sessions et paiements, prévisualiser ou télécharger un reçu.
BF-37	Consulter les plans proposés par les organisations et gérer plusieurs abonnements indépendants avec leurs factures.
BF-38	Utiliser un identifiant OCPP virtuel ; préparer l'évolution vers une carte RFID ou la lecture d'un QR code physique sans exposer de secret.

2.1.2.7 Visiteur et traitements automatiques

Code	Besoin fonctionnel
BF-39	Présenter la solution et les moyens de contact, permettre l'inscription client et la demande de démonstration sans exposer les données protégées.
BF-40	Recevoir, authentifier et rendre idempotents les événements OCPP, puis projeter les états métier et leurs transitions.
BF-41	Détecter les absences de communication, défauts, échéances SLA ou maintenances proches et produire les alertes et notifications correspondantes.
BF-42	Traiter en tâche de fond les emails, occurrences récurrentes, contrôles de SLA, exports volumineux et événements de paiement.
BF-43	Vérifier les signatures et l'idempotence des webhooks de paiement, puis réconcilier le paiement, la session et le reçu.

2.1.3 Besoins non fonctionnels

Les objectifs du cahier des charges sont transformés en exigences vérifiables. Les valeurs de production qui ne peuvent pas être garanties par un environnement local sont explicitement présentées comme des objectifs à valider lors du déploiement.

Table 2.10 – Exigences non fonctionnelles

Code	Qualité	Exigence et critère de vérification
ENF-01	Performance	Sous charge nominale et avec pagination, une page courante doit devenir utilisable en moins de deux secondes ; les opérations longues sont asynchrones et signalent leur progression.
ENF-02	Temps réel	Un événement OCPP accepté doit mettre à jour sa projection et être diffusé sans rechargement manuel. Une perte de communication est détectée après 90 secondes sans message, conformément à la règle métier retenue.
ENF-03	Disponibilité du service	La cible de production est de 99,9 % hors maintenance planifiée. Cette cible doit être mesurée par supervision après déploiement ; elle n'est pas déclarée atteinte par le seul MVP local.
ENF-04	Sécurité des accès	Toute route protégée utilise une session Sanctum, la protection CSRF, RBAC et une policy sur la ressource. Un identifiant deviné d'une autre organisation ne doit retourner aucune donnée.
ENF-05	Isolation multi-tenant	Un administrateur, opérateur ou technicien appartient à exactement une organisation. Toutes les lectures, écritures, diffusions et exports sont limités à ce périmètre.
ENF-06	Protection des secrets	Aucun secret OAuth, email, paiement ou OCPP n'est inclus dans le frontend, les logs applicatifs ou une réponse API. Les mots de passe et jetons persistés sont condensés ou chiffrés selon leur usage.
ENF-07	Intégrité et idempotence	Une station et ses connecteurs sont créés atomiquement. Les événements OCPP, webhooks, commandes et tâches récurrentes supportent la répétition sans doublon métier.

Code	Qualité	Exigence et critère de vérification
ENF-08	Traçabilité	Les changements sensibles conservent acteur, date, ancienne et nouvelle valeur ou événement métier. Les rapports finaux et reçus restent reproductibles.
ENF-09	Maintenabilité	Les contrôleurs délèguent les règles aux services et polices ; les contrats frontend sont centralisés ; les modules critiques sont couverts par des tests automatisés et documentés.
ENF-10	Extensibilité	Les fournisseurs de paiement, email, cartographie et identité sont intégrés derrière des contrats ou configurations remplaçables. OCPP reste séparé de l'API métier.
ENF-11	Ergonomie	Les parcours critiques utilisent des étapes courtes, des états de chargement et des erreurs actionnables. L'interface reste cohérente entre rôles sans rendre leurs tableaux de bord identiques.
ENF-12	Responsive et compatibilité	Les parcours principaux sont utilisables sur ordinateur et tablette, et les parcours client essentiels sur mobile. Les dernières versions de Chrome, Edge et Firefox sont ciblées.
ENF-13	Confidentialité	La position courante du client reste dans l'état transitoire du navigateur et n'est pas persistée. Les pièces d'intervention privées sont servies après autorisation.
ENF-14	Portabilité	L'environnement peut être reproduit avec Git, PHP, Node.js, PostgreSQL et Docker sous WSL2 pour les services simulés, sans dépendre d'un poste unique.
ENF-15	Interopérabilité	Les échanges métier utilisent une API REST JSON documentable par OpenAPI ; les stations utilisent OCPP 1.6 JSON sur WebSocket et le paiement suit un contrat fournisseur indépendant.

2.2 Planification du projet

2.2.1 Méthode de priorisation et d'estimation

Deux informations distinctes sont associées à chaque user story :

- la **priorité** exprime la valeur métier et l'urgence. Les niveaux Haute, Moyenne et Faible correspondent respectivement à une adaptation des catégories *Must*, *Should* et *Could* de MoSCoW ;
- les **story points** estiment l'effort relatif, l'incertitude et le risque selon la suite de Fibonacci : 1, 2, 3, 5, 8 et 13.

Les points ne représentent donc ni des jours ni une priorité. Une fonctionnalité hautement prioritaire peut recevoir peu de points si elle est simple, tandis qu'une évolution facultative comme le firmware peut recevoir 13 points en raison de son risque technique.

2.2.2 Product Backlog révisé

Le backlog initial de 30 user stories a été enrichi à mesure que les relations multi-organisations, la disponibilité calculée et les parcours réels ont été clarifiés. La gestion des véhicules, retirée du produit après validation UX, est remplacée par un onboarding adapté au rôle. Deux user stories distinguent désormais la gestion commerciale globale de la consultation de l'abonnement par l'administrateur. Le backlog révisé contient 42 user stories pour 301 points.

Table 2.11 – Product Backlog de ChargeTrackr

ID	User Story	Priorité	Pts
US-01	En tant que visiteur, je veux créer un compte client ou me connecter de manière sécurisée par email ou Google.	Haute	8
US-02	En tant qu'utilisateur authentifié, je veux consulter et mettre à jour mon profil ainsi que les paramètres de sécurité de mon compte.	Moyenne	3
US-03	En tant que super administrateur, je veux gérer les organisations, leurs administrateurs et les accès globaux de la plateforme.	Haute	8
US-04	En tant qu'administrateur, je veux créer, modifier et désactiver les comptes des opérateurs et techniciens de mon organisation.	Haute	5
US-05	En tant que super administrateur, je veux consulter l'audit global, l'état des intégrations et les statistiques d'utilisation de la plateforme.	Moyenne	5

ID	User Story	Priorité	Pts
US-06	En tant qu'administrateur, je veux consulter les clients ayant utilisé les bornes de mon organisation avec leurs sessions et statistiques.	Moyenne	5
US-07	En tant qu'opérateur, je veux ajouter, modifier et désactiver les bornes de mon organisation avec leurs informations techniques et géographiques.	Haute	8
US-08	En tant qu'opérateur, je veux gérer les connecteurs associés aux bornes de mon organisation avec leur type, puissance et état.	Haute	5
US-09	En tant qu'opérateur, je veux consulter la liste des bornes avec leur disponibilité calculée, leur connexion et leur dernière activité.	Haute	8
US-10	En tant qu'opérateur, je veux visualiser les bornes sur une carte et ajouter une borne en sélectionnant son emplacement.	Haute	8
US-11	En tant que technicien, je veux consulter sans modification les bornes et leur emplacement sur la carte de mon organisation.	Haute	5
US-12	En tant que client, je veux rechercher et filtrer les bornes publiques selon leur disponibilité, connecteur et tarif.	Haute	8
US-13	En tant qu'opérateur, je veux consulter la fiche détaillée d'une borne avec ses connecteurs, sessions, historique et alertes.	Haute	8
US-14	En tant que plateforme, je veux recevoir et valider automatiquement les messages OCPP 1.6 JSON des bornes physiques ou simulées.	Haute	13
US-15	En tant que plateforme, je veux calculer l'état des bornes à partir des événements OCPP, transactions, heartbeats et délais de non-réponse.	Haute	13
US-16	En tant qu'opérateur, je veux superviser en temps réel les états disponible, occupée, non disponible, maintenance, déconnectée ou en défaut.	Haute	13
US-17	En tant qu'opérateur, je veux consulter un tableau de bord avec disponibilité, énergie, sessions actives et taux d'indisponibilité.	Haute	8
US-18	En tant qu'administrateur, je veux consulter le tableau de bord de mon organisation avec utilisateurs, clients, revenus, régions et bornes.	Haute	8
US-19	En tant qu'opérateur, je veux recevoir des alertes lorsqu'une borne est déconnectée, en panne, en surchauffe ou en erreur de communication.	Haute	8
US-20	En tant qu'opérateur, je veux filtrer, suivre et assigner les alertes selon leur gravité, statut et borne.	Moyenne	5
US-21	En tant que technicien, je veux consulter les alertes et interventions qui me sont assignées.	Haute	5
US-22	En tant que technicien, je veux créer un rapport d'intervention avec diagnostic, actions, pièces et photos.	Moyenne	8

ID	User Story	Priorité	Pts
US-23	En tant qu'opérateur, je veux planifier et suivre les maintenances préventives et correctives.	Moyenne	8
US-24	En tant que client, je veux démarrer une session sur une borne disponible, quelle que soit son organisation.	Haute	8
US-25	En tant que client, je veux arrêter ma session et consulter le résumé de durée, énergie et coût.	Haute	5
US-26	En tant qu'utilisateur autorisé, je veux consulter l'historique des sessions de mon périmètre avec durée, énergie, coût et borne.	Moyenne	5
US-27	En tant que nouvel utilisateur, je veux un onboarding adapté à mon rôle afin de configurer mon espace et atteindre une première action utile.	Moyenne	5
US-28	En tant que client, je veux utiliser une carte RFID ou un QR code pour m'identifier et lancer une recharge.	Moyenne	8
US-29	En tant que client, je veux payer mes sessions, suivre le paiement et prévisualiser ou télécharger mes reçus.	Moyenne	8
US-30	En tant que plateforme, je veux traiter les paiements au moyen d'un adaptateur simulé et remplaçable.	Moyenne	8
US-31	En tant qu'administrateur, je veux configurer les tarifs selon le kWh, la durée ou les plages horaires et les affecter aux actifs.	Moyenne	8
US-32	En tant que client, je veux choisir les plans des organisations et gérer simultanément plusieurs abonnements.	Moyenne	8
US-33	En tant qu'utilisateur autorisé, je veux consulter des analyses adaptées à mon rôle et échanger des rapports opérationnels.	Moyenne	8
US-34	En tant qu'utilisateur autorisé, je veux exporter les données et rapports de mon périmètre en PDF, JSON ou CSV.	Faible	5
US-35	En tant qu'administrateur, je veux gérer les paramètres locaux de mon organisation : TVA, logo, langue, fuseau et notifications.	Faible	5
US-36	En tant que super administrateur, je veux configurer les intégrations globales : paiement, notifications, cartographie, OAuth2, OCPP et sécurité.	Faible	8
US-37	En tant qu'administrateur, je veux gérer les contrats, garanties, notices et plans techniques liés aux bornes.	Faible	5
US-38	En tant qu'administrateur, je veux gérer les mises à jour firmware des bornes à distance.	Faible	13
US-39	En tant que visiteur, je veux consulter la présentation, contacter la plateforme et envoyer une demande de démonstration.	Moyenne	3

ID	User Story	Priorité	Pts
US-40	En tant qu'utilisateur authentifié, je veux recevoir des notifications in-app et email et gérer mes préférences.	Moyenne	5
US-41	En tant que super administrateur, je veux gérer les offres SaaS, essais, abonnements et factures des organisations.	Haute	8
US-42	En tant qu'administrateur, je veux suivre l'abonnement commercial, les limites et factures de mon organisation et demander un changement d'offre.	Moyenne	5
		Total	301 points

Table 2.12 – Répartition du backlog par priorité

Priorité	User stories	Points	Part des points
Haute	20	160	53,2 %
Moyenne	17	105	34,9 %
Faible	5	36	12,0 %
Total	42	301	100 %

2.2.3 Planification des Sprints

Le Product Backlog est réalisé au moyen de Sprints courts, chacun associé à un objectif vérifiable. La cadence prévue est hebdomadaire ; le dernier Sprint est légèrement raccourci afin de respecter la date de fin du stage. Les périodes du tableau 2.13 représentent la planification de référence. Le contenu détaillé d'un Sprint peut être ajusté lors du Sprint Planning ou réordonné après une Sprint Review sans modifier l'objectif global du produit. Les story points ne sont pas convertis en heures. Ils servent à comparer l'effort et le risque des user stories. La capacité est appréciée progressivement à partir des Sprints précédents, ce qui évite de présenter une précision artificielle au début du stage.

Table 2.13 – Synthèse des Sprint Backlogs

Sprint	Période prévue	Objectif	User stories principales	Résultat inspecté en revue
S01	1–7 juillet	Cadrer le produit et valider l'expérience cible	US-27, US-39 et travaux de conception	Backlog initial, acteurs, cas d'utilisation global, architecture de référence et prototype navigable.
S02	8–14 juillet	Sécuriser l'entrée et l'organisation des comptes	US-01 à US-06	Authentification, création de compte, invitation, activation, rôles et isolation d'une organisation.
S03	15–21 juillet	Constituer et localiser le réseau de recharge	US-07 à US-13, US-37	Stations, connecteurs, cartographie, fiche détaillée et documents accessibles selon le rôle.
S04	22–28 juillet	Produire une disponibilité issue du terrain	US-14 à US-16, US-19	Connexion d'une station simulée, réception des événements, projection des états, historique et alerte de connectivité.
S05	29 juillet–4 août	Réaliser une recharge complète	US-24 à US-30	Sélection d'une station, connexion, autorisation, mesures, arrêt, paiement simulé et reçu.
S06	5–11 août	Traiter un incident jusqu'à sa résolution	US-20 à US-23, US-40	Alerte qualifiée, intervention assignée, maintenance, preuves de travail et notifications.
S07	12–18 août	Structurer les offres et la tarification	US-31, US-32, US-41, US-42	Tarifs de recharge, adhésions des clients, essais et abonnements commerciaux des organisations.

Sprint	Période prévue	Objectif	User stories principales	Résultat inspecté en revue
S08	19–25 août	Fournir les outils de pilotage	US-05, US-17, US-18, US-33 à US-36	Tableaux de bord spécialisés, rapports, exports, audit, paramètres et état des intégrations.
S09	26–31 août	Stabiliser et préparer la recette	Tests transversaux et documentation	Corrections finales, sécurité, tests de recette, API documentée, manuels, rapport et préparation du déploiement.

La gestion distante du firmware (US-38) reste volontairement dans le Product Backlog sans être engagée dans un Sprint du MVP. Elle nécessite une validation matérielle et sécuritaire qui dépasse le parcours principal du stage. Cette transparence évite de présenter comme réalisée une évolution seulement envisagée.

2.2.4 Matrice de traçabilité fonctionnelle

La matrice suivante relie les grands groupes de besoins à leurs principaux contrats API et écrans. Les routes détaillées et la spécification OpenAPI seront présentées dans les livrables techniques.

Table 2.14 – Traçabilité entre backlog, API et interfaces

Domaine	User stories	Contrats API principaux	Interfaces principales
Identité et entrée	US-01–06, 27, 39	Auth, profil, onboarding, utilisateurs, organisations, demandes de démo	Landing, connexion, activation, bienvenue, profil, utilisateurs, organisations
Stations et carte	US-07–13, 37	Stations, commissioning, connecteurs, carte, télé-métrie, documents	Stations, recherche de station, détail station, cartes par rôle

Domaine	User stories	Contrats API principaux	Interfaces principales
OCPP et disponibilité	US-14–16, 19	API interne OCPP, commandes, transitions, alertes	Détail station, supervision, carte, centre d’alertes
Exploitation	US-20–23, 40	Alertes, interventions, maintenances, notifications	Alertes, interventions, maintenance, calendrier, cloche de notifications
Recharge et paiement	US-24–30	Tentatives, sessions, paiements, reçu, webhook fournisseur	Assistant de recharge, sessions, vue live, paiements et reçus
Tarification et commerce	US-31, 32, 41, 42	Tarifs, plans, abonnements clients, portefeuille commercial, factures	Tarifs, abonnements, portefeuille commercial, facturation organisationnelle
Pilotage et rapports	US-05, 17–18, 33–36	Dashboards, reporting, rapports internes, exports, audit, intégrations, paramètres	Tableaux de bord par rôle, analyses, rapports, audit, intégrations, paramètres

Conclusion

La spécification distingue désormais clairement les responsabilités humaines, les systèmes externes et les composants internes. Le backlog de 42 user stories couvre l’entrée dans la plateforme, la supervision OCPP, l’exploitation terrain, la recharge, le commerce et le pilotage, tout en conservant le firmware comme évolution à haut risque. Les exigences non fonctionnelles rendent les objectifs de sécurité, performance, isolation et traçabilité contrôlables. Le chapitre suivant traduira ces besoins en architecture, cas d’utilisation détaillés, séquences et modèles de données.

Conception

Introduction

Le chapitre précédent a défini ce que la plateforme doit accomplir. Le présent chapitre décrit comment ces besoins sont traduits en architecture, interactions, composants, contrats, processus et données. Les choix présentés ne constituent pas une architecture théorique : ils sont alignés sur les dépendances, les services, les routes et les modèles présents dans le dépôt au moment de la rédaction.

La conception poursuit cinq objectifs : rendre les scénarios métier explicites, séparer les responsabilités métier des adaptateurs techniques, conserver une source de vérité unique, isoler strictement les organisations et rendre les intégrations remplaçables. Cette démarche conduit à des cas d'utilisation vérifiables et à un monolithe Laravel modulaire complété par une application React, une passerelle OCPP spécialisée et des services d'infrastructure indépendants.

3.1 Conception générale

La conception générale décrit l'organisation physique du système, ses couches logiques et les responsabilités de ses principaux composants. Elle privilégie une séparation nette des responsabilités sans introduire la complexité d'une architecture distribuée complète, peu adaptée à la durée du stage et à une réalisation individuelle.

3.1.1 Architecture physique

3.1.1.1 Architecture client–serveur

ChargeTrackr adopte une architecture client–serveur. Le navigateur constitue le client et exécute l'application React. Il communique avec l'API Laravel au moyen de requêtes HTTPS et de réponses JSON. Les mises à jour temps réel sont diffusées par WebSocket

au moyen de Laravel Reverb. Le serveur applicatif applique les règles métier, contrôle les autorisations et reste l'unique composant autorisé à modifier les données persistantes.

La communication avec les bornes constitue un second échange client–serveur. Les bornes ou leur simulateur se connectent à une passerelle WebSocket au moyen du protocole OCPP 1.6 JSON. La passerelle traduit les messages du protocole en événements normalisés, puis les transmet à Laravel par une API interne signée. Elle n'accède jamais directement à la base de données.

3.1.1.2 Modèle trois tiers

L'application est organisée selon un modèle trois tiers :

Présentation pages, composants, formulaires et visualisations exécutés dans le navigateur ;

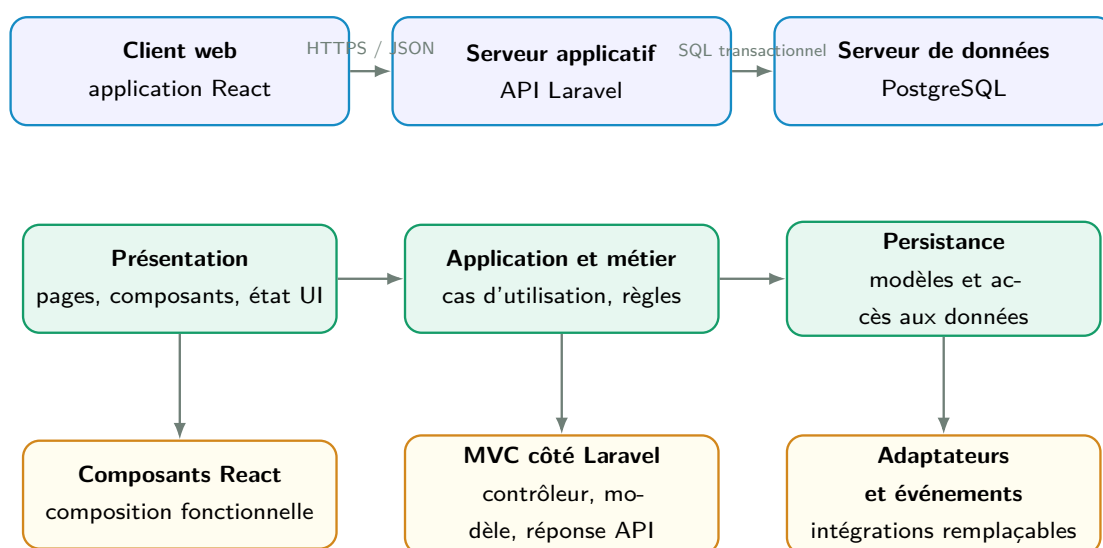
Application et métier

cas d'utilisation, validation, autorisation, tarification, disponibilité, recharge, maintenance et facturation ;

Persistance

modèles relationnels et transactions assurant la conservation cohérente des informations dans PostgreSQL.

Cette séparation limite les dépendances entre l'interface et les règles métier. Une modification visuelle n'affecte pas la persistance, tandis qu'une nouvelle règle de disponibilité ou de tarification peut être testée indépendamment de l'affichage.



Architecture client–serveur et trois tiers, réalisée sous la forme d'un monolithe modulaire complété par une passerelle OCPP spécialisée.

Figure 3.1 – Synthèse des styles architecturaux et patrons retenus

3.1.2 Architecture logique

3.1.2.1 Styles et patrons de conception

Le coeur applicatif est réalisé sous la forme d'un **monolithe modulaire**. Les fonctionnalités sont déployées dans une même application Laravel et partagent une base PostgreSQL, mais elles sont séparées par domaines : identité, organisations, stations, OCPP, recharge, exploitation, commerce et rapports. Ce choix conserve la simplicité des transactions locales tout en évitant de concentrer les règles dans les contrôleurs.

Le backend applique une **architecture en couches**. Les contrôleurs adaptent HTTP, les requêtes dédiées valident les entrées, les polices autorisent l'action, les services exécutent le cas d'utilisation, puis les modèles Eloquent assurent la persistance. Laravel repose bien sur le patron **MVC**, mais, dans ce projet, la vue HTML n'est pas rendue par Laravel : React prend en charge la présentation et les contrôleurs Laravel renvoient des ressources JSON. Le terme MVC décrit donc principalement l'organisation du backend.

Le frontend utilise une architecture **orientée composants**. Les pages composent des composants réutilisables, tandis que les accès API, les types et les requêtes sont regroupés par fonctionnalité. Enfin, deux patrons complètent cette organisation :

- le patron **Adapter** isole Google OAuth, Resend, la cartographie et le paiement derrière des contrats remplaçables ;
- une approche **orientée événements** traite les messages OCPP, les changements de disponibilité, les alertes et les notifications sans confondre l'événement reçu avec l'état métier calculé.

3.1.2.2 Architecture technique

La figure 3.2 regroupe les technologies dans leurs environnements d'exécution. Les cadres en pointillés représentent des systèmes externes ou simulés. PostgreSQL est la source de vérité ; ni le frontend, ni la passerelle OCPP, ni les simulateurs ne modifient directement les données métier.

Principes structurants

- PostgreSQL constitue la source de vérité de la plateforme ;
- Laravel centralise l'autorisation et les décisions métier ;
- la passerelle OCPP transporte et normalise le protocole sans calculer la dispo-

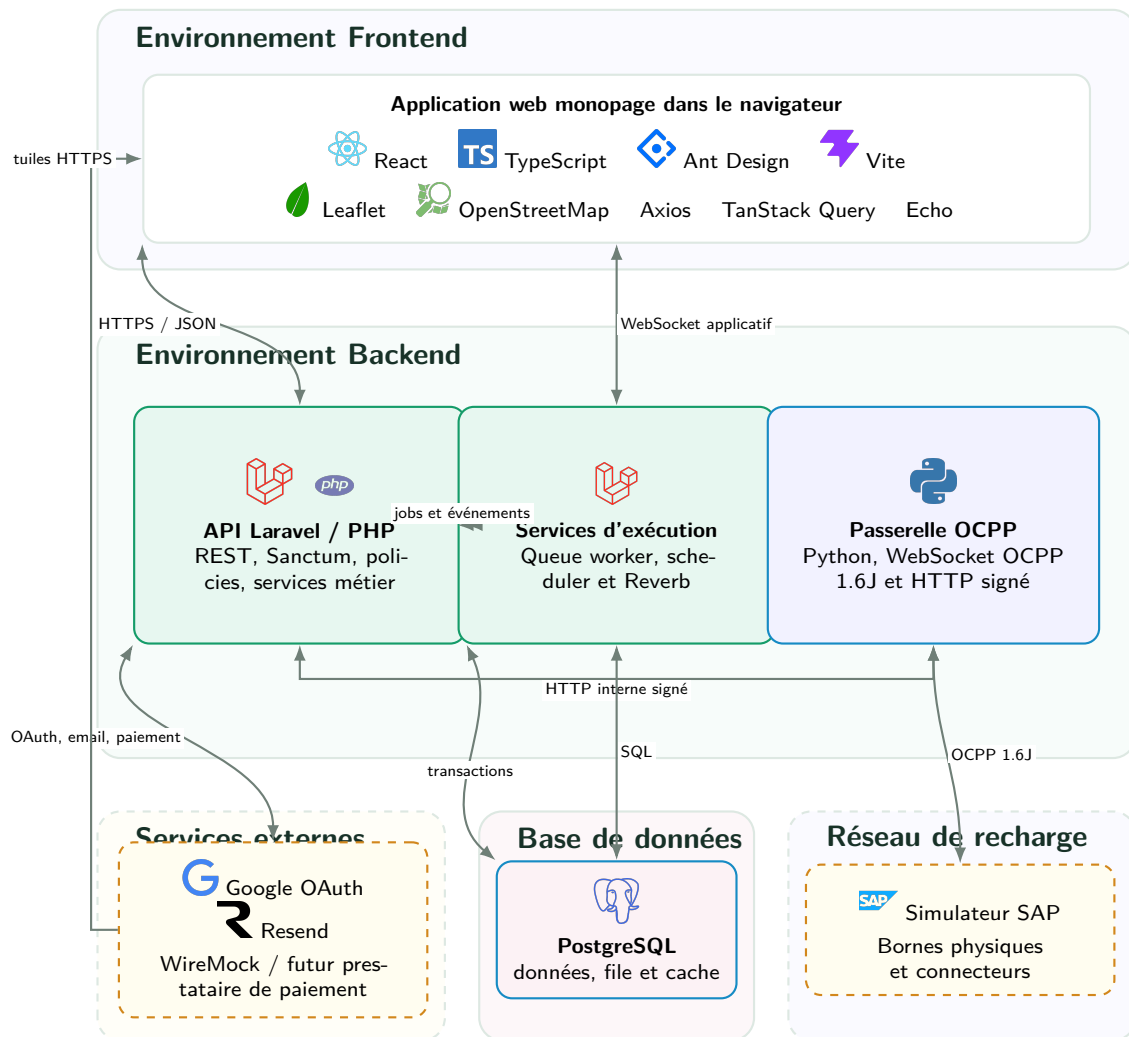


Figure 3.2 – Architecture technique globale de ChargeTrackr

nibilité ;

- le frontend adapte l'expérience au rôle, mais ne remplace jamais les contrôles du backend ;
- les intégrations externes sont encapsulées dans des adaptateurs ;
- les traitements différés et périodiques sont exécutés par le worker et le scheduler.

3.1.3 Description des composants

3.1.3.1 Application web React

L'interface est une application monopage développée avec React et TypeScript [11]. Vite assure le développement et la compilation. Ant Design fournit les contrôles d'interface ; TanStack Query gère l'état provenant du serveur ; Axios centralise les appels HTTP. React Router sépare les routes publiques et protégées. Leaflet et OpenStreetMap affichent les stations et permettent de sélectionner une position [12, 13]. Recharts produit les graphiques et Laravel Echo consomme les événements diffusés par Reverb.

Les appels API sont regroupés par fonctionnalité hors des composants visuels. L'état serveur reste géré par TanStack Query, tandis que les états temporaires d'interface, comme l'étape d'un assistant, restent locaux au frontend.

3.1.3.2 API et coeur métier Laravel

Laravel 13 expose l'API REST, valide les entrées, applique les autorisations et orchestre les cas d'utilisation [14]. Une requête protégée traverse les responsabilités suivantes :

1. Sanctum identifie la session et contrôle le jeton CSRF ;
2. le middleware organisationnel vérifie le statut et le périmètre du compte ;
3. une *Form Request* valide et normalise les données ;
4. une policy vérifie l'action sur la ressource concernée ;
5. le contrôleur délègue l'opération à un service métier ;
6. le service applique les invariants, transactions et événements ;
7. une API Resource structure la réponse exposée.

Les services couvrent notamment le commissioning, la disponibilité, les transactions OCPP, les sessions, le paiement, la maintenance, les notifications, les abonnements et les

exports. Les contrôleurs demeurent ainsi des adaptateurs HTTP et non des conteneurs de logique métier.

3.1.3.3 Persistance, asynchronisme et temps réel

PostgreSQL 16 stocke les comptes, actifs, événements techniques, projections, transactions et journaux [15]. Les opérations atomiques, telles que la création d'une station avec ses connecteurs ou l'enregistrement d'une transaction OCPP, sont protégées par des transactions de base de données. La file d'attente et le cache de l'environnement de référence utilisent également PostgreSQL.

Le worker exécute les emails, notifications et opérations différées. Le scheduler recalcule la disponibilité, expire les commandes OCPP, génère les maintenances récurrentes, vérifie les SLA et synchronise les abonnements. Reverb diffuse ensuite les changements sur des canaux privés associés à un utilisateur, une organisation ou une session.

3.1.3.4 Passerelle OCPP

La passerelle est développée en Python avec les bibliothèques `ocpp`, `websockets` et `httpx`. Elle accepte les connexions `ocpp1.6`, authentifie l'identité de la station auprès de Laravel, publie chaque message normalisé et récupère les commandes distantes. Elle supporte notamment `BootNotification`, `Heartbeat`, `StatusNotification`, `Authorize`, `StartTransaction`, `MeterValues` et `StopTransaction`.

La passerelle prend également en charge les commandes distantes suivantes :

- `RemoteStartTransaction` et `RemoteStopTransaction` pour piloter une recharge ;
- `Reset`, `UnlockConnector` et `ChangeAvailability` pour l'exploitation de la station.

OCPP 1.6 JSON correspond au simulateur et au périmètre du stage [3, 16]. OCPP 2.0.1 demeure une perspective.

3.1.3.5 Services externes et adaptateurs

Table 3.1 – Intégrations techniques et responsabilités

Intégration	Technologie		Responsabilité	Frontière
Identité sociale	Google	OAuth 2.0	Vérifier l'identité Google et l'email	Les rôles et organisations restent locaux.
Email transactionnel	Resend		Envoyer vérifications, invitations et alertes	Les secrets et l'envoi restent côté serveur.
Cartographie	Leaflet / OpenStreetMap		Tuiles, marqueurs, position et itinéraire	La position courante du client n'est pas persistée.
Paieement	Adaptateur de paiement		Autoriser, capturer, libérer ou débiter	Le fournisseur ne modifie jamais directement une session.
Simulation du paiement	WireMock 3		Reproduire succès, refus, délai, erreur et webhook	Aucun moyen de paiement réel n'est utilisé.
Simulation des bornes	Simulateur SAP		Produire des connexions et messages OCPP	Le simulateur ne lit ni l'API métier ni la base.

3.1.4 Architecture de déploiement

Le développement local associe des processus exécutés sur l'hôte et des conteneurs Docker. Les ports ci-dessous rendent les interactions reproductibles sur la machine de développement ; ils ne constituent pas une contrainte de production.

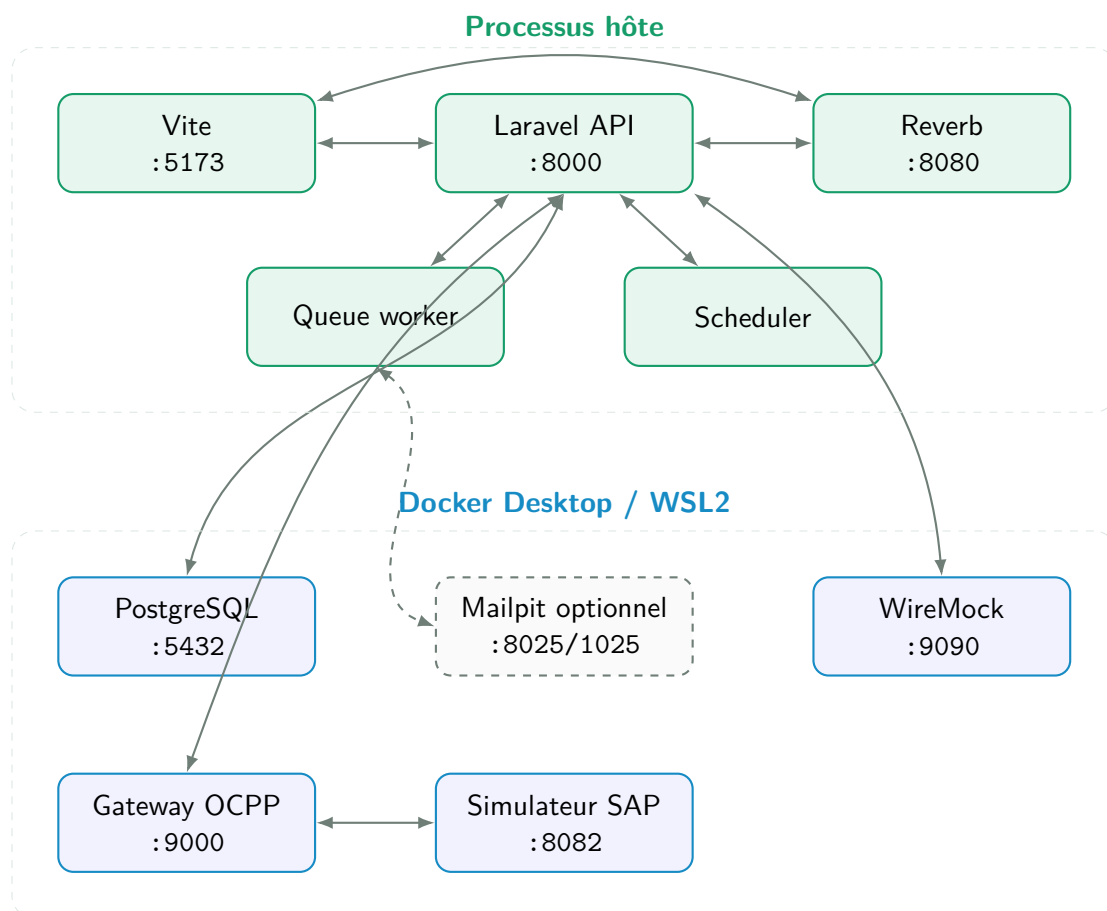


Figure 3.3 – Architecture de déploiement de l'environnement local

Table 3.2 – Processus nécessaires dans l’environnement de développement

Processus	Port	Exécution	Rôle
Frontend Vite	5173	Hôte	Interface React et rechargement à chaud.
API Laravel	8000	Hôte	API REST, OAuth, webhooks et API interne OCPP.
Queue worker	—	Hôte	Emails, notifications et traitements différés.
Scheduler	—	Hôte	Disponibilité, SLA, maintenance et synchronisations.
Reverb	8080	Hôte	WebSocket temps réel de l’application.
PostgreSQL	5432	Local ou Docker	Source de vérité, file et cache.
Passerelle OCPP	9000	Docker	WebSocket OCPP et adaptateur vers Laravel.
Simulateur SAP	8082	Docker	Flotte de bornes et interface de simulation.
WireMock	9090	Docker	API de paiement externe simulée.
Mailpit	8025/1025	Docker, optionnel	Boîte de réception locale pour les tests SMTP.

En production, Vite est remplacé par des fichiers statiques servis derrière HTTPS. Laravel est exécuté par un serveur PHP supervisé et les connexions OCPP utilisent `wss://`. Les workers, le scheduler, Reverb et la passerelle OCPP doivent être supervisés. Les secrets restent injectés par l’environnement et ne sont jamais incorporés aux images ni au dépôt.

3.1.5 Architecture logicielle

3.1.5.1 Organisation du frontend

Le frontend suit une organisation par fonctionnalités :

- `features/auth` regroupe la session et les gardes de routes ;

- `features/stations`, `maps` et `charging` portent les parcours de station et de recharge ;
- `features/operations` couvre alertes, interventions et maintenance ;
- `features/commercial`, `subscriptions` et `tariffs` séparent les deux niveaux de commerce ;
- `features/realtime` encapsule Echo et Reverb ;
- `pages` compose ces modules sans réimplémenter leurs contrats API.

L'interface construit ses menus et tableaux de bord à partir du rôle retourné par le backend. Cette adaptation améliore l'expérience, mais ne constitue pas une barrière de sécurité : toute route masquée reste protégée par Laravel.

3.1.5.2 Organisation du backend

Le backend respecte les responsabilités natives de Laravel :

Http/Controllers

adaptation des requêtes et réponses ;

Http/Requests validation, normalisation et préconditions ;

Policies autorisation par action, rôle et organisation ;

Services orchestration transactionnelle et règles métier ;

Models relations, attributs et invariants locaux ;

Jobs/Events traitements asynchrones et diffusion temps réel ;

Contracts interfaces des intégrations remplaçables.

L'interface `PaymentGateway` illustre cette séparation. Les services de recharge appellent les opérations d'autorisation, de capture, de libération et de débit sans dépendre de WireMock. Un futur adaptateur `ClicToPay`, `e-DINAR/D17`, `GPGCheckout`, `Konnect` ou `Flouci` pourra respecter le même contrat.

3.2 Conception fonctionnelle et scénarios d'utilisation

Cette section complète la spécification par une conception orientée scénarios. Elle part du diagramme global, identifie les cas majeurs, puis détaille pour chacun le contrat fonctionnel, le déroulement nominal et les exceptions. Deux emplacements UML sont réservés à la suite de chaque fiche : un diagramme de cas d'utilisation raffiné et un

diagramme de séquence système. Leur contenu sera produit à partir des scénarios validés, sans introduire d'opération absente du Product Backlog.

3.2.1 Diagramme global

Le diagramme global de la figure 3.4 regroupe les capacités communes et les fonctions propres à chaque rôle. Il distingue les acteurs humains, la borne de recharge et le service de paiement, puis explicite les dépendances **include**, l'extension liée à l'assignation d'une intervention et les généralisations d'acteurs. Les scénarios détaillés complètent cette vue en précisant les préconditions, les contrôles d'autorisation et les cas d'erreur.

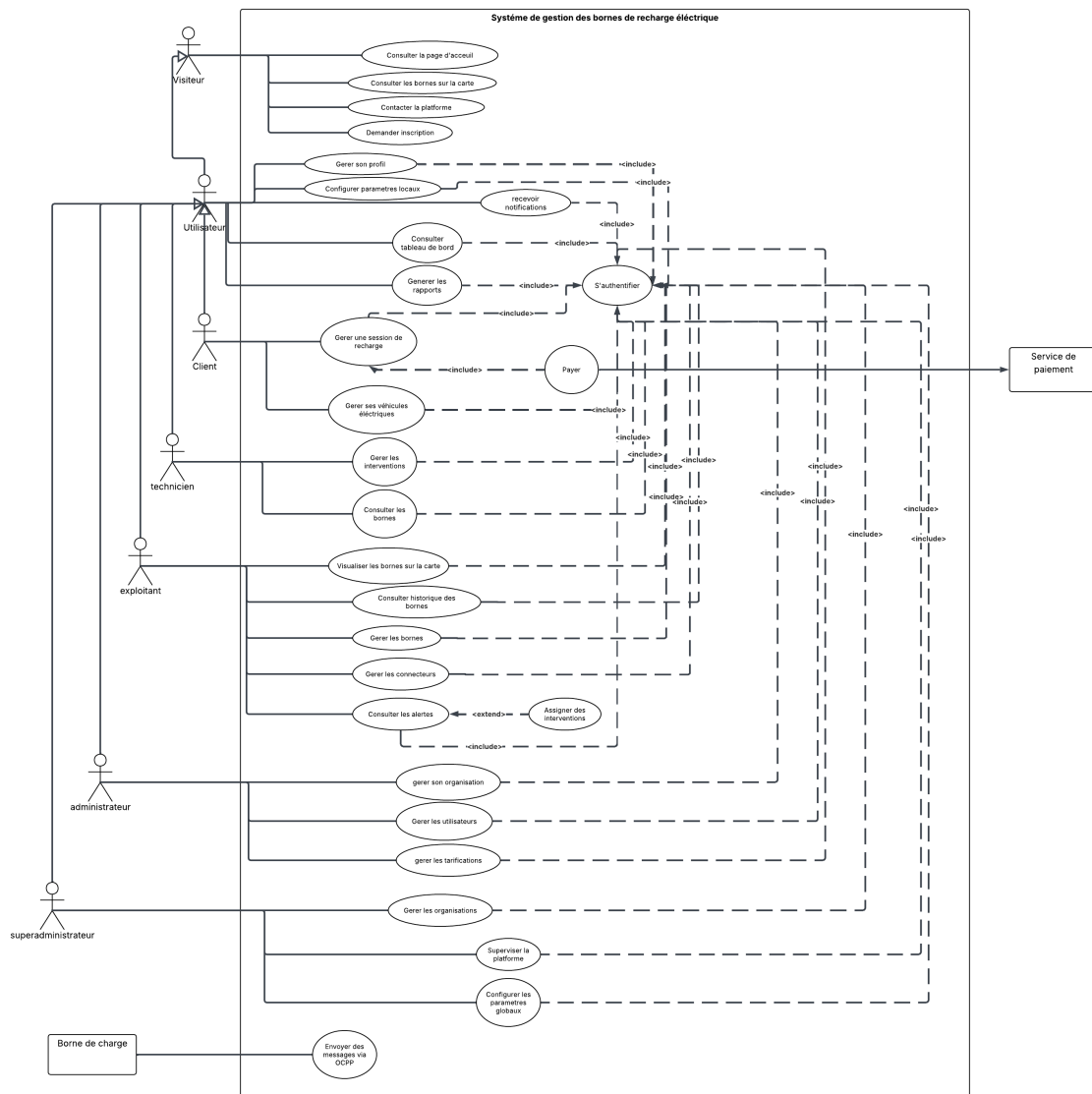


Figure 3.4 – Diagramme global des cas d'utilisation de ChargeTrackr

3.2.2 Catalogue des cas majeurs

Table 3.3 – Catalogue des cas d'utilisation majeurs

ID	Cas d'utilisation	Acteur principal	Résultat métier
UC-01	Demander une démonstration et provisionner une organisation	Visiteur, Super Administrateur	Une organisation d'essai et son administrateur invité sont créés après décision explicite.
UC-02	Inviter et activer un employé	Administrateur	Un opérateur ou technicien définit lui-même son mot de passe et rejoint la bonne organisation.
UC-03	Mettre en service une station OCPP	Opérateur	La station, ses connecteurs et son identité OCPP sont créés atomiquement et prêts à être reliés au simulateur ou au matériel.
UC-04	Calculer la disponibilité	Borne / plate-forme	Les preuves OCPP deviennent un état métier, une transition et, si nécessaire, une alerte.
UC-05	Effectuer une recharge et payer	Client	Une transaction technique produit une session mesurée, un paiement réconcilié et un reçu.
UC-06	Traiter une alerte et une intervention	Opérateur, Technicien	L'incident est qualifié, assigné, documenté et clôturé avec une chronologie.
UC-07	Gérer le cycle commercial d'une organisation	Super Administrateur, Administrateur	L'essai, l'offre, les limites et factures évoluent sans supprimer les données de l'organisation.

ID	Cas d'utilisation	Acteur principal	Résultat métier
UC-08	Générer et échanger un rapport	Utilisateur auto-risé	Une analyse adaptée au rôle est exportée ou envoyée avec pièces jointes et suivi.

3.2.3 UC-01 – Demander une démonstration et provisionner une organisation

Objectif	Convertir une demande publique qualifiée en organisation d'essai et compte administrateur sécurisé.
Acteurs	Visiteur et Super Administrateur ; service email comme acteur secondaire.
Préconditions	Les demandes de démonstration sont activées ; l'adresse n'est pas bloquée par les protections anti-abus.
Déclencheur	Le visiteur valide le formulaire public.
Postconditions	La demande est rejetée ou reliée à une organisation d'essai et à une invitation administrateur traçable.

Scénario nominal

1. Le visiteur renseigne son identité, son organisation, sa taille, ses objectifs multiples, son message et son consentement.
2. Le système valide les champs, le honeypot et la limitation de débit, puis enregistre la demande au statut *nouvelle*.
3. Un accusé de réception est envoyé sans lien vers l'espace Super Administrateur.
4. Le Super Administrateur ouvre la demande et démarre sa revue.
5. Il vérifie le contexte, ajuste la durée d'essai et saisit le nom de l'organisation et de son futur administrateur.
6. Le système crée atomiquement l'organisation, l'abonnement d'essai, le compte administrateur en attente et l'invitation.
7. L'invitation à usage unique est envoyée au demandeur.
8. Le demandeur l'accepte, définit son mot de passe, complète éventuellement le logo de l'organisation et accède à l'onboarding administrateur.

Alternatives et exceptions

- si la demande n'est pas pertinente, le Super Administrateur la rejette avec une raison ;
aucun compte n'est créé ;
- si l'email ou l'organisation provoque un conflit, la transaction est annulée sans
création partielle ;
- si l'invitation expire ou doit être invalidée, elle est révoquée avant qu'une nouvelle
invitation ne soit émise ;
- les transitions intermédiaires sans action métier ne sont pas exposées comme des
statuts sélectionnables manuellement.

Diagramme de cas d'utilisation raffiné

Emplacement réservé pour UC-01 – Demande de démonstration et provisionnement

Acteurs, frontière du système, spécialisations et relations **include/extend**.

Diagramme de séquence système

Emplacement réservé pour UC-01 – Demande de démonstration et provisionnement

Acteur principal, plateforme, systèmes externes, alternatives et réponses observables.

3.2.4 UC-02 – Inviter et activer un employé

Objectif	Donner un accès opérateur ou technicien sans transmettre de mot de passe initial.
Acteurs	Administrateur, employé invité et service email.
Préconditions	L'administrateur et son organisation sont actifs ; la limite d'employés de l'offre commerciale n'est pas atteinte.
Déclencheur	L'administrateur choisit <i>Ajouter un employé</i> .
Postconditions	Le compte activé appartient à exactement une organisation et possède uniquement le rôle choisi.

Scénario nominal

1. L'administrateur saisit le nom, l'email, le rôle opérateur ou technicien, l'équipe et les coordonnées utiles.
2. Le backend ignore tout identifiant d'organisation venant du navigateur et impose celle de l'administrateur.
3. Un compte *pending* sans mot de passe utilisable est créé avec une invitation limitée dans le temps.
4. L'employé reçoit le lien, vérifie les informations non modifiables et choisit son mot de passe.
5. L'invitation est consommée une seule fois, le compte devient actif et l'onboarding du rôle est proposé à la première connexion.

Alternatives et exceptions

- un rôle administrateur, client ou super administrateur est refusé dans ce workflow ;
- un email déjà actif, un quota atteint ou une organisation inactive empêche l'invitation ;
- l'administrateur peut envoyer un rappel tant que le lien est valide, ou renouveler le lien après invalidation de l'ancien ;
- aucun administrateur ne peut afficher, choisir ou réinitialiser le mot de passe de l'employé.

Diagramme de cas d'utilisation raffiné

Emplacement réservé pour UC-02 – Invitation et activation d'un employé
Acteurs, frontière du système, spécialisations et relations **include/extend**.

Diagramme de séquence système

Emplacement réservé pour UC-02 – Invitation et activation d'un employé
Acteur principal, plateforme, systèmes externes, alternatives et réponses observables.

3.2.5 UC-03 – Mettre en service une station OCPP

Objectif	Créer une station exploitable, ses connecteurs physiques et son identité de communication dans une seule opération.
Acteurs	Opérateur ou Administrateur ; borne physique ou simulateur SAP.
Préconditions	L'organisation est autorisée à ajouter une station et n'a pas atteint sa capacité commerciale.
Déclencheur	L'utilisateur ouvre l'assistant de mise en service.
Postconditions	La station est persistée avec ses connecteurs et une cible de commissioning ; son secret en clair n'est affiché qu'une fois si requis.

Scénario nominal

1. L'opérateur renseigne le nom, la référence, le fabricant, le modèle, la puissance, l'adresse et la version OCPP.
2. Il clique sur la carte ou déplace le marqueur ; latitude et longitude sont remplies automatiquement mais restent éditables.
3. Il ajoute chaque connecteur avec son libellé, son identifiant OCPP, son type, son courant et sa puissance maximale.
4. Il choisit la cible *simulateur SAP local* ou *station externe* et confirme l'identité OCPP.
5. Le système vérifie les références, identités et identifiants connecteur, puis crée l'ensemble dans une transaction.
6. Pour le simulateur local, l'interface fournit la commande d'enregistrement du profil ; pour une borne externe, elle affiche une fois les informations de connexion nécessaires.
7. La borne ouvre sa connexion, envoie **BootNotification** et **StatusNotification**, puis la fiche reflète la connexion réelle.

Alternatives et exceptions

- une référence comme CT-TUN-001 déjà utilisée doit être remplacée par une valeur unique ; elle ne peut pas désigner deux stations ;
- une erreur sur un connecteur annule la création complète ;
- tant que le simulateur n'est pas enregistré et redémarré, la station reste en attente de connexion ; la création en base ne simule pas un heartbeat ;

- une rotation de secret invalide l'ancien secret sans modifier l'identité publique de la station.

Diagramme de cas d'utilisation raffiné

Emplacement réservé pour UC-03 – Mise en service d'une station OCPP
Acteurs, frontière du système, spécialisations et relations **include/extend**.

Diagramme de séquence système

Emplacement réservé pour UC-03 – Mise en service d'une station OCPP
Acteur principal, plateforme, systèmes externes, alternatives et réponses observables.

3.2.6 UC-04 – Calculer la disponibilité

Objectif	Transformer les preuves OCPP et les dérogations autorisées en état métier explicable.
Acteurs	Borne ou simulateur ; moteur de projection de la plateforme.
Préconditions	La station est provisionnée pour OCPP et ses connecteurs sont connus.
Déclencheur	Connexion, fermeture, message OCPP ou contrôle périodique de connectivité.
Postconditions	Les états courants, transitions, alertes et diffusions temps réel sont cohérents et audités.

Scénario nominal

1. La passerelle authentifie la station et vérifie la signature de l'appel interne transmis à Laravel.
2. Le message est déduplicué et persisté avant projection.
3. Le moteur applique l'ordre : désactivation manuelle, maintenance planifiée, perte de communication, puis états OCPP des connecteurs.
4. Au moins un connecteur libre rend la station disponible ; sinon les états actifs, réservés ou en défaut déterminent le résultat.
5. Toute modification d'état ou de raison crée une transition et un événement temps réel.
6. Une anomalie de communication ou de connecteur crée ou rouvre une alerte déduplicuée ; sa disparition la résout automatiquement.

Alternatives et exceptions

- après 90 secondes sans aucun message, la station devient hors ligne même si un ancien statut **Available** subsiste ;
- un message tardif ne peut pas écraser une preuve plus récente ;
- une maintenance ou désactivation manuelle reste prioritaire sur un statut disponible envoyé par la borne ;
- une station non provisionnée pour OCPP conserve son statut administratif et n'est pas artificiellement projetée.

Diagramme de cas d'utilisation raffiné

Emplacement réservé pour UC-04 – Calcul de la disponibilité
Acteurs, frontière du système, spécialisations et relations **include/extend**.

Diagramme de séquence système

Emplacement réservé pour UC-04 – Calcul de la disponibilité
Acteur principal, plateforme, systèmes externes, alternatives et réponses observables.

3.2.7 UC-05 – Effectuer une recharge et payer

Objectif	Permettre au client de réaliser une recharge mesurée et de régler le montant final sans créer de session fictive.
Acteurs	Client, borne OCPP et service de paiement.
Préconditions	Client actif et vérifié, station publique connectée, connecteur disponible, tarif effectif et aucune session concurrente.
Déclencheur	Le client choisit <i>Recharger</i> sur une station.
Postconditions	La transaction est arrêtée, le montant final est capturé ou signalé impayé, et un reçu consultable est associé.

Scénario nominal

1. Si la station est déjà choisie, l'assistant commence directement au choix du connecteur ; sinon le client sélectionne d'abord une station.
2. Le client choisit un connecteur libre et consulte son type, sa puissance et le tarif effectif.
3. Il branche physiquement le câble. L'étape progresse uniquement lorsque le message OCPP **StatusNotification** indique la préparation ; aucun bouton ne confirme manuellement le branchement.
4. Le service de paiement préautorise un plafond simulé et conserve une clé d'idempotence.
5. Le backend crée une tentative et envoie **RemoteStartTransaction** avec un identifiant OCPP virtuel à courte portée.
6. La session métier n'est créée qu'après le **StartTransaction** de la borne.
7. Les **MeterValues** mettent à jour énergie, puissance, durée et coût provisoire affichés dans la vue live.
8. Le client arrête la recharge ou une limite d'énergie, de montant ou de durée demande l'arrêt automatique.
9. **RemoteStopTransaction** est envoyé ; la borne retourne **StopTransaction** avec le compteur final.
10. Le montant définitif est calculé à partir du tarif figé, capturé auprès du fournisseur et présenté dans le reçu.

Alternatives et exceptions

-
- si le câble n'est pas détecté, l'assistant reste en attente et permet l'annulation sans session ;
 - un refus de préautorisation empêche l'envoi du démarrage ;
 - un timeout ou refus OCPP annule la tentative et libère la préautorisation ;
 - une coupure pendant la recharge marque la session interrompue ; un événement final tardif peut la réconcilier une seule fois ;
 - un webhook dupliqué ne provoque ni double capture ni double revenu.

Diagramme de cas d'utilisation raffiné

Emplacement réservé pour UC-05 – Recharge et paiement
Acteurs, frontière du système, spécialisations et relations **include/extend**.

Diagramme de séquence système

Emplacement réservé pour UC-05 – Recharge et paiement
Acteur principal, plateforme, systèmes externes, alternatives et réponses observables.

3.2.8 UC-06 – Traiter une alerte et une intervention

Objectif	Transformer une anomalie détectée en action terrain traçable jusqu'à la résolution.
Acteurs	Opérateur et Technicien ; service email pour les événements critiques ou assignations.
Préconditions	L'alerte appartient à l'organisation et l'opérateur dispose du droit de gestion.
Déclencheur	Création automatique d'une alerte ou signalement autorisé.
Postconditions	L'alerte est résolue ou documentée pour suivi ; le rapport et les preuves restent accessibles aux acteurs autorisés.

Scénario nominal

1. Le moteur crée une alerte avec source, gravité, station, connecteur, échéance SLA et clé de déduplication.
2. L'opérateur l'acquitte, examine sa chronologie et choisit un technicien actif de la même organisation.
3. Il crée l'intervention et la priorité est transmise au technicien par notification.
4. Le technicien accepte puis démarre l'intervention.
5. Il ajoute diagnostic, notes, actions, pièces, contrôles et photos avant/après.
6. Il termine le rapport avec un résultat : résolu, suivi requis ou non résolu.
7. L'opérateur vérifie le résultat et clôture l'alerte lorsque la condition technique a disparu.

Alternatives et exceptions

- une alerte identique ouverte est mise à jour au lieu d'être dupliquée ;
- une récurrence après résolution rouvre l'alerte et conserve l'historique ;
- seul le technicien assigné peut produire le rapport terrain ;
- un dépassement de SLA déclenche une notification idempotente ;
- une intervention peut être annulée avec motif, mais son historique n'est pas supprimé.

Diagramme de cas d'utilisation raffiné

Emplacement réservé pour UC-06 – Traitement d'une alerte et d'une intervention

Acteurs, frontière du système, spécialisations et relations **include/extend**.

Diagramme de séquence système

Emplacement réservé pour UC-06 – Traitement d'une alerte et d'une intervention

Acteur principal, plateforme, systèmes externes, alternatives et réponses observables.

3.2.9 UC-07 – Gérer le cycle commercial d'une organisation

Objectif	Maintenir un lien cohérent entre période d'essai, offre SaaS, limites, accès commercial et facturation de l'organisation.
Acteurs	Super Administrateur et Administrateur ; service de paiement simulé pour les règlements futurs.
Préconditions	L'organisation existe et possède un abonnement commercial ou un essai.
Déclencheur	Échéance, demande de changement ou action du Super Administrateur.
Postconditions	Le statut commercial, les capacités et les factures restent cohérents et audités.

Scénario nominal

1. Le Super Administrateur publie une offre avec prix, cycle, limite de stations, limite d'employés et fonctionnalités.
2. L'organisation issue d'une démonstration reçoit une période d'essai et ses capacités initiales.
3. L'administrateur consulte son offre, l'utilisation réelle, l'échéance et ses factures.
4. Il soumet une demande de changement sans modifier lui-même le contrat.
5. Le Super Administrateur accepte la demande, prolonge l'essai ou affecte une nouvelle offre.
6. Une facture organisationnelle est produite, puis acquittée ou annulée avec un événement d'audit.

Alternatives et exceptions

- une tentative de dépasser le quota d'employés ou de stations est refusée avant création ;
- à l'expiration, l'organisation conserve ses données mais les opérations protégées sont limitées jusqu'à régularisation ;
- une suspension manuelle exige un motif et peut être restaurée ;
- les paiements des clients pour leurs recharges restent séparés de la facture SaaS de l'organisation.

Diagramme de cas d'utilisation raffiné

Emplacement réservé pour UC-07 – Cycle commercial d'une organisation
Acteurs, frontière du système, spécialisations et relations **include/extend**.

Diagramme de séquence système

Emplacement réservé pour UC-07 – Cycle commercial d'une organisation
Acteur principal, plateforme, systèmes externes, alternatives et réponses observables.

3.2.10 UC-08 – Générer et échanger un rapport

Objectif	Transformer les données du périmètre en analyse adaptée au rôle, export ou rapport interne traçable.
Acteurs	Super Administrateur, Administrateur, Opérateur ou Technicien.
Préconditions	L'utilisateur possède les permissions de consultation, d'export ou d'échange correspondantes.
Déclencheur	Sélection d'une période ou création d'un brouillon.
Postconditions	L'export est téléchargé ou le rapport est envoyé à un destinataire autorisé avec son historique.

Scénario nominal

1. L'utilisateur choisit une période et consulte les indicateurs propres à son rôle : plateforme, organisation, opérations ou terrain.
2. Les agrégations appliquent le même périmètre que les détails et exposent leur formule.
3. Il exporte la vue en PDF, JSON ou CSV, ou crée un rapport interne.
4. Pour un rapport interne, il choisit catégorie, priorité, destinataire, description et pièces jointes.
5. Le brouillon est envoyé ; le destinataire le consulte, le marque comme lu puis peut l'archiver.
6. L'auteur et le destinataire peuvent prévisualiser le document généré selon leurs permissions.

Alternatives et exceptions

- une période sans données produit des zéros et listes vides explicites ;
- un destinataire d'une autre organisation ou non autorisé n'est pas proposé ;
- une pièce interdite ou trop volumineuse est refusée avant envoi ;
- un export volumineux est traité en file puis notifié lorsqu'il est prêt ;
- le document PDF reprend la période, les filtres, l'émetteur et les indicateurs sans présenter de données étrangères au rôle.

Diagramme de cas d'utilisation raffiné

Emplacement réservé pour UC-08 – Génération et échange d'un rapport
Acteurs, frontière du système, spécialisations et relations **include/extend**.

Diagramme de séquence système

Emplacement réservé pour UC-08 – Génération et échange d'un rapport
Acteur principal, plateforme, systèmes externes, alternatives et réponses observables.

3.3 Conception détaillée

3.3.1 Identité, demandes de démonstration et invitations

Trois parcours créent des comptes, chacun avec une règle différente :

1. un visiteur peut créer directement un compte **client**. L'adresse doit être vérifiée avant la connexion locale ;
2. un client peut utiliser Google OAuth. L'identité Google est liée à un compte local, mais ne fournit ni rôle ni organisation ;
3. un administrateur, opérateur ou technicien est toujours créé par invitation. Le destinataire active lui-même son compte et choisit son mot de passe.

La demande de démonstration constitue l'entrée commerciale d'une organisation. Le visiteur renseigne ses coordonnées, la société, la taille du réseau et plusieurs objectifs opérationnels. Une confirmation est envoyée au demandeur sans lien interne. Le super administrateur ouvre ensuite la demande, la rejette ou lance sa revue. Une acceptation provisionne, dans une transaction, l'organisation, son administrateur en état **pending**, l'essai commercial et une invitation temporaire. Le compte ne devient actif qu'après acceptation du lien à usage unique.

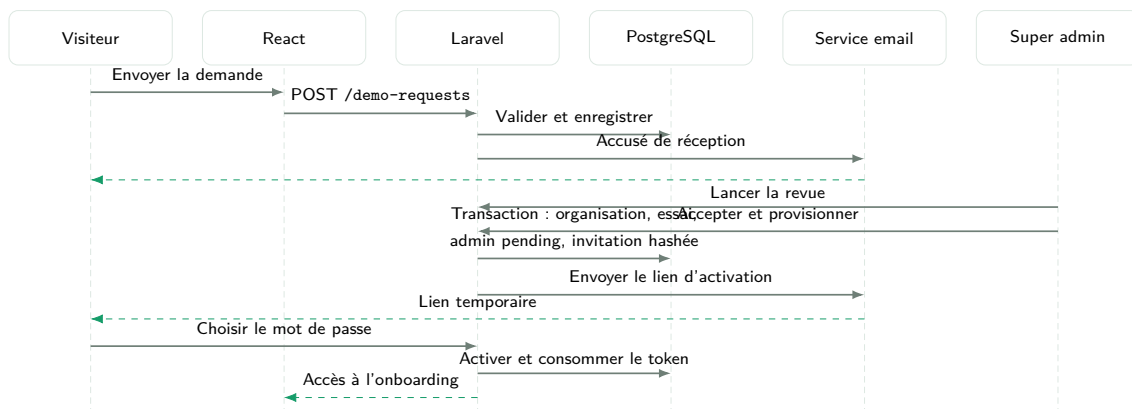


Figure 3.5 – Provisionnement d'une organisation à partir d'une demande de démonstration

Les invitations suivent les invariants suivants :

- le token aléatoire n'est envoyé qu'une fois et seul son condensat SHA-256 est stocké ;
- le lien possède une expiration et devient inutilisable après acceptation, révocation ou remplacement ;
- un rappel renouvelle le token, car sa valeur originale ne peut pas être récupérée ;

- un administrateur ne peut inviter que des opérateurs ou techniciens de son organisation ;
- l'utilisateur invité choisit son secret ; l'administrateur ne connaît jamais son mot de passe.

3.3.2 Mise en service d'une station

Le commissioning rassemble la station, son emplacement, son matériel, ses connecteurs et sa cible OCPP dans un assistant unique. La création est atomique : si une référence, une identité OCPP ou un connecteur est invalide, aucune partie incomplète n'est conservée.

Table 3.4 – Cibles de mise en service OCPP

Cible	Comportement	Usage
Borne physique ou externe	Génération d'un secret Basic Auth individuel de 48 caractères, affiché une seule fois et stocké uniquement sous forme de hash.	Intégration future d'un chargeur réel ou d'un simulateur tiers.
Simulateur SAP local	Création de l'inventaire puis ajout du profil au manifeste local par une commande Artisan dédiée.	Tests reproductibles du stage.
Inventaire seul	Création sans identité ni secret OCPP. L'état reste non disponible jusqu'au provisioning.	Préparation administrative d'un futur actif.

La référence interne et l'identité OCPP peuvent différer, mais leur égalité simplifie l'exploitation. Le libellé A1 d'un connecteur est destiné à l'utilisateur ; `ocpp_connector_id` est l'entier réellement transmis par la borne. Pour le simulateur SAP, ces identifiants doivent être contigus à partir de 1. Après provisionnement, la borne se connecte à `/ocpp/<identité>` avec le sous-protocole `ocpp1.6`.

3.3.3 Ingestion OCPP et disponibilité calculée

3.3.3.1 Séparation entre preuve brute et projection métier

Un statut reçu de la borne n'est pas directement présenté comme vérité métier. L'événement complet est d'abord enregistré dans `ocpp_events`. Les champs `ocpp_*` de la station et du connecteur forment ensuite une projection technique, tandis que les champs `status`, `availability_reason` et `availability_source` constituent la projection métier.

Cette séparation apporte trois garanties :

- un événement tardif ne remplace pas une preuve plus récente ;
- les règles peuvent évoluer sans supprimer l'historique technique ;
- chaque changement visible peut être expliqué par une transition et son événement source.

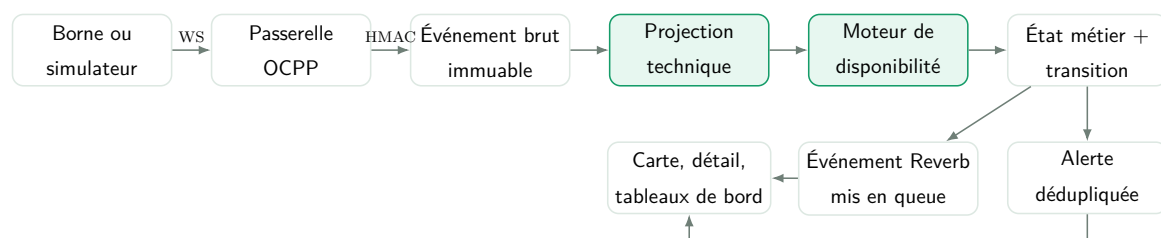


Figure 3.6 – Chaîne de traitement d'un événement OCPP

3.3.3.2 Règles de disponibilité

La résolution applique les règles par priorité. Une décision manuelle ou une maintenance planifiée domine toujours la télémétrie. Ensuite, une fermeture WebSocket explicite ou 90 secondes sans message produit l'état `offline`. Le délai représente trois périodes de heartbeat de 30 secondes. Enfin, les états des connecteurs déterminent la capacité réelle de la station.

Table 3.5 – Matrice de décision de disponibilité

Prio.	Preuve ou règle	État station	Conséquence
1	Désactivation manuelle	<code>unavailable</code>	Recharge interdite jusqu'à levée de l'override.

Prio.	Preuve ou règle	État station	Conséquence
2	Maintenance active	maintenance	Indisponibilité planifiée et commande ChangeAvailability .
3	Déconnexion ou silence supérieur à 90 secondes	offline	Connecteurs hors ligne et alerte de communication.
4	Au moins un connecteur Available	available	Démarrage distant autorisé si l'organisation est active.
5	Aucun libre, au moins un en transaction	charging	Station vivante mais sans place libre correspondante.
6	Aucun libre, au moins un réservé	reserved	Réservation visible ; recharge publique non immédiate.
7	Aucun utilisable, au moins un Faulted	faulted	Alerte de défaut dédoublée.
8	Connectée sans preuve exploitable	unavailable	Attente d'un StatusNotification valide.

Chaque variation de l'état, de la raison ou de la source crée une **AvailabilityTransition**. Les alertes automatiques utilisent une clé de déduplication stable : la même panne ne crée pas une série d'alertes identiques, mais une récurrence après résolution rouvre le suivi. L'événement **StationAvailabilityChanged** invalide les vues React concernées via Reverb.

3.3.4 Parcours de recharge et transaction OCPP

3.3.4.1 Assistant côté client

Le parcours client suit cinq étapes afin de correspondre aux actions physiques :

1. choisir une station publique disponible ou ouvrir directement une station ;

2. sélectionner un connecteur compatible et libre ;
3. brancher le câble ; l'interface attend réellement le statut OCPP **Preparing** et n'utilise pas une confirmation manuelle ;
4. choisir le moyen de paiement et, facultativement, une limite d'énergie, de montant ou de durée ;
5. autoriser le paiement puis demander le démarrage distant.

Le rayon *Near me* est une préférence de compte. La géolocalisation n'est demandée qu'après une action explicite et reste dans le navigateur. Un QR de connecteur contient un lien profond vers la station et le connecteur ; il ne contient ni idTag, ni secret OCPP, ni jeton de paiement.

3.3.4.2 Tentative, session et transaction

Trois objets distincts évitent de facturer une énergie qui n'a pas été délivrée :

ChargingAttempt

intention du client, préautorisation et commande de démarrage ;

OcppTransaction

preuve technique issue de la borne ;

ChargingSession

session métier créée seulement après un **StartTransaction** accepté.

Une préautorisation réussie ne prouve donc pas que la recharge a commencé. Laravel crée une commande **RemoteStartTransaction** chiffrée au repos et limitée dans le temps. La passerelle la réclame, l'envoie à la connexion active, puis retourne le résultat. Seul le **StartTransaction** ultérieur crée la session facturable. Les **MeterValues** actualisent l'énergie, la puissance, l'état de charge et l'estimation de coût. Une limite atteinte produit une commande **RemoteStopTransaction**.

L'arrêt manuel du client suit le même mécanisme distant. Le **StopTransaction** final fournit le compteur et la raison de fin. Si la borne se déconnecte avant ce message, la transaction passe en **awaiting_reconciliation** et la session est interrompue sans inventer de compteur final. Un message tardif peut la réconcilier.

3.3.5 Paiement et reçu

Le paiement applique le patron **Adapter**. Le domaine dépend de **PaymentGateway**, pas d'un fournisseur. Le contrat expose quatre opérations : préautoriser, capturer, libérer et débiter. Deux implémentations existent :

- **SimulatedPaymentAdapter**, rapide et déterministe pour les tests ;
- **WireMockPaymentAdapter**, qui appelle une API HTTP externe simulée.

WireMock reproduit quatre résultats : succès, refus métier, délai dépassé et erreur fournisseur. Les callbacks portent une signature HMAC-SHA256. Laravel vérifie la signature, enregistre l'identifiant fournisseur une seule fois et protège chaque opération par une clé d'idempotence. Une répétition du webhook ne peut donc ni capturer deux fois le paiement ni doubler le chiffre d'affaires.

La préautorisation par défaut est de 30 TND et expire après 15 minutes. Elle est libérée si la station refuse le démarrage ou si la tentative expire. Après **StopTransaction**, le worker capture uniquement le montant final calculé à partir du tarif figé au démarrage. Le reçu PDF est généré côté serveur à partir de la session et du paiement persistés. Il peut être prévisualisé ou téléchargé sans exposer les données d'un autre client.

3.3.6 Alertes, interventions et maintenance

Ces trois notions appartiennent au même cycle opérationnel mais répondent à des questions différentes :

- Alerte** Quel problème a été détecté, avec quelle gravité et quel SLA ?
- Intervention** Qui traite concrètement ce problème, quand et avec quelles preuves ?
- Maintenance** Quelle action préventive ou corrective doit être planifiée, éventuellement de façon récurrente ?

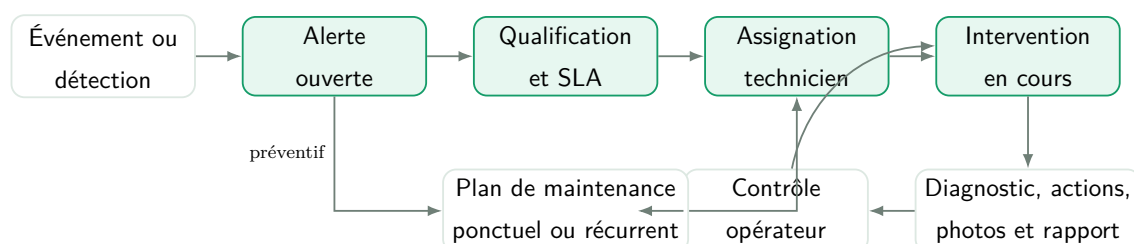


Figure 3.8 – Cycle opérationnel des alertes, interventions et maintenances

L'opérateur qualifie l'alerte, assigne un technicien et peut créer une intervention. Le technicien accepte le travail, ajoute notes, diagnostic, photos avant/après et rapport, puis indique sa résolution. L'opérateur contrôle les preuves et clôture le suivi. Les changements sont conservés dans des journaux d'événements, et les notifications SLA sont dédoublées pour ne pas produire de rappels multiples.

Une maintenance peut exister sans alerte, par exemple un contrôle trimestriel. Le scheduler génère ses occurrences et peut les transformer en interventions assignées. À l'inverse, une intervention corrective peut être créée directement depuis une alerte. Les documents et photos sont conservés sur un disque privé et servis uniquement après vérification de policy.

3.3.7 Tarification, plans et gestion commerciale

La conception distingue deux marchés qui ne doivent pas être mélangés :

- le **tarif de recharge** appartient à l'organisation exploitante et calcule le coût d'une session selon le kWh, les frais de session, l'inactivité et un minimum ;
- le **plan client** est un abonnement du conducteur à une organisation et peut fournir une remise. Un client peut posséder plusieurs abonnements auprès d'organisations différentes ;
- le **plan SaaS** est le contrat entre la plateforme et une organisation. Il fixe notamment les limites d'employés et de stations.

Le tarif effectif est résolu du plus spécifique au plus général : affectation au connecteur, à la station, puis tarif par défaut de l'organisation. Lors du démarrage, la session conserve un instantané du tarif et du plan client. Une modification ultérieure ne change donc pas rétroactivement une recharge en cours.

La demande de démonstration crée un essai commercial. À l'échéance, le cycle de vie peut entrer en période de grâce, être activé sur une offre ou suspendre les opérations dépassant les droits. Le super administrateur gère le catalogue SaaS, les essais et les factures ; l'administrateur d'organisation ne consulte que son contrat, son usage et ses demandes de changement.

3.3.8 Rapports, documents et exports

Les analyses ne sont pas un écran générique dont seules les valeurs changent. Le super administrateur suit l'adoption et le portefeuille d'organisations ; l'administrateur suit les revenus et les capacités ; l'opérateur suit la disponibilité, les incidents et l'énergie ; le technicien suit ses SLA et résultats terrain. Les contrôleurs de reporting exposent donc des contrats distincts **platform**, **organization**, **operations** et **field**.

Les exports CSV et JSON utilisent les mêmes filtres et le même périmètre que la vue. Les PDF sont générés côté serveur afin de garantir une identité visuelle stable et une source vérifiée. Les rapports internes possèdent un émetteur, un destinataire, un statut, des pièces jointes privées et une version PDF. Ils permettent les passations entre employés et l'envoi d'un bilan à l'administrateur sans utiliser un canal externe non audité.

3.4 Sécurité et isolation multi-tenant

3.4.1 Authentification de l'application web

L'application React est un client propriétaire de la même plateforme. Elle utilise donc Laravel Sanctum en mode **session stateful** plutôt qu'un JWT conservé dans le stockage du navigateur. Avant une connexion locale, le frontend récupère le cookie CSRF et envoie les requêtes avec les cookies activés. Le cookie de session est **HttpOnly**, ce qui réduit l'exposition du secret de session au code JavaScript.

Google OAuth est orchestré par le backend. Le secret client Google n'est jamais fourni au frontend. Après le callback, Laravel exige un email vérifié, lie ou crée le compte local puis établit la même session Sanctum. Un compte Google ne reçoit aucun rôle privilégié du fournisseur : les rôles et l'organisation sont attribués uniquement par la plateforme. Dans le périmètre actuel, Microsoft OAuth est différé.

Les mots de passe locaux sont hashés par le mécanisme Laravel. Les formulaires de récupération répondent de manière générique afin de limiter l'énumération des comptes. Les comptes exclusivement Google n'affichent pas une action de changement de mot de passe tant qu'une authentification locale n'a pas été activée.

3.4.2 Autorisation et périmètre organisationnel

L'autorisation repose sur trois niveaux complémentaires :

1. les permissions Spatie décrivent les capacités d'un rôle ;
2. les politiques contrôlent l'action sur la ressource concrète ;
3. `canAccessOrganization` et les filtres de requête garantissent le périmètre de données.

Table 3.6 – Périmètre d'accès des rôles

Rôle	Association	Périmètre
Super adminis- trateur	Global, sans orga- nisation	Organisations, intégrations, audit et commerce SaaS selon ses permissions.
Administrateur	Une organisation	Employés, clients, actifs, tarifs, commerce et rapports de son organisation.
Opérateur	Une organisation	Stations, connecteurs, sessions, alertes, maintenances et interventions de son organisation.
Technicien	Une organisation	Consultation des actifs et traitement des alertes, interventions ou maintenances autorisées, sans création de station.
Client	Global, sans orga- nisation	Stations publiques et uniquement ses propres tentatives, sessions, paiements, abonnements et notifications.

Le frontend adapte la navigation, mais l'absence d'un bouton ne constitue jamais une autorisation. Chaque endpoint sensible appelle une policy. L'administrateur ne peut pas gérer les employés d'une autre organisation, changer son propre rôle, désactiver son propre compte ni supprimer le dernier administrateur actif. Les identifiants présents dans l'URL sont donc revalidés afin de prévenir les accès directs de type IDOR.

3.4.3 Sécurité des frontières OCPP

Deux niveaux protègent la communication des bornes :

- la borne utilise une identité OCPP et un secret Basic Auth. Pour une borne externe, le secret aléatoire est affiché une seule fois et seul son hash est stocké ;

- la passerelle signe ses requêtes internes avec HMAC-SHA256 sur le corps exact, un timestamp Unix et un identifiant UUID.

Laravel rejette les signatures invalides, les timestamps trop anciens et les UUID déjà utilisés. L'identifiant métier de l'événement rend également l'ingestion idempotente : une répétition identique retourne l'enregistrement existant, tandis qu'un même identifiant associé à un contenu différent est refusé. Les payloads de commandes sont chiffrés au repos et expirent. Un seul gateway peut réclamer une commande en attente.

HMAC garantit l'intégrité et l'authenticité du service, mais pas la confidentialité. En production, la borne utilise donc `wss://` et le gateway communique avec Laravel par HTTPS ou au sein d'un réseau privé chiffré. Le secret partagé du simulateur n'est acceptable que dans l'environnement local ; chaque borne physique doit recevoir un secret individuel.

3.4.4 Sécurité du paiement

Le simulateur n'accepte aucune carte réelle. L'API externe reçoit une référence, un montant, une devise, un résultat de simulation et une clé d'idempotence. Le webhook retour est signé et limité en fréquence. `PaymentProviderEvent` conserve l'identifiant du fournisseur, la signature vérifiée et le résultat de réconciliation. Les règles suivantes s'appliquent :

- aucune confirmation du frontend ne marque directement un paiement comme réussi ;
- un webhook invalide n'est pas traité ;
- une répétition valide ne produit pas une seconde opération ;
- un délai ou une indisponibilité technique est distingué d'un refus métier ;
- les secrets, numéros de carte et payloads sensibles ne sont jamais journalisés.

L'intégration d'un prestataire réel nécessitera en plus la conformité aux exigences du fournisseur, TLS public, une politique de remboursement, la réconciliation financière et, selon le mode choisi, l'externalisation complète des données de carte.

3.4.5 Validation, limitation et gestion des erreurs

Les écritures utilisent des Form Requests et retournent une erreur métier contrôlée plutôt qu'une trace SQL ou une exception interne. Des buckets de limitation indépendants empêchent un parcours public de bloquer un autre :

Table 3.7 – Principales limites applicatives

Flux	Limite	Clé
Demande de démonstration	3 par heure	Adresse IP
Inspection d’invitation	10 par minute	Adresse IP
Acceptation d’invitation	5 par minute	IP et hash de l’email
Gestion des invitations employé	10 par minute	Utilisateur authentifié
API interne OCPP	600 par minute	Adresse de la passerelle
Webhook de paiement	120 par minute	Adresse du fournisseur
Changement de mot de passe	5 par minute	Session authentifiée

La déduplication métier complète le throttling. Par exemple, une demande récente ayant la même identité n’est pas recréée, et les alertes automatiques utilisent une clé stable. Cette combinaison traite à la fois les abus volumétriques et les doubles clics légitimes.

3.4.6 Secrets, fichiers et journalisation

Les fichiers `.env` contiennent les clés Laravel, Google, Resend, Reverb, OCPP et paiement. Ils sont ignorés par Git ; seuls les `.env.example` sans valeurs privées sont versionnés. Le frontend ne reçoit que les paramètres publics préfixés `VITE_`. Les secrets de bornes, tokens OAuth, mots de passe, en-têtes `Authorization` et signatures ne doivent pas apparaître dans les logs.

Les photos d’intervention, rapports et documents techniques utilisent le disque Laravel privé. Un contrôleur autorisé lit le fichier et le diffuse après policy ; une URL de stockage publique ne contourne donc pas le périmètre organisationnel. Les types MIME, tailles et extensions sont validés avant enregistrement.

`PlatformAuditLog` conserve les actions sensibles : gestion de comptes et rôles, changement de paramètres, commissioning, rotation de credentials, tarifs, abonnements et états financiers. L’auteur, l’organisation, la cible, l’action et un résumé sont enregistrés sans inclure le secret modifié. Une tâche quotidienne applique la politique de rétention.

3.4.7 Analyse synthétique des risques

Table 3.8 – Risques principaux et mesures de réduction

Risque	Exemple	Mesures
Vol de session	Script lisant un token du navigateur	Cookie HttpOnly, Sanctum stateful, CSRF, rotation et TLS.
IDOR multi-tenant	Accès à <code>/stations/11</code> d'une autre organisation	Policies, scoping organisationnel, tests d'isolation.
Rejeu OCPP	Réémission d'un événement signé	Timestamp, UUID de requête, HMAC et identifiant d'événement idempotent.
Faux paiement	Callback frontend ou webhook forgé	Confirmation serveur, signature HMAC et événements fournisseur uniques.
Double facturation	Répétition d'une capture après timeout	Clés d'idempotence et réconciliation persistée.
Exposition documentaire	URL publique d'une photo technique	Stockage privé, validation et téléchargement soumis à policy.
Secret dans Git	Commit d'un client secret OAuth	<code>.gitignore</code> , exemples vides et injection par environnement.
Déni de service logique	Soumission répétée de demandes ou invitations	Rate limiting indépendant, honeypot et déduplication.

3.4.8 Limites de sécurité du prototype

Le socle protège les parcours implémentés, mais ne remplace pas un audit de production. Il reste notamment à automatiser un scan des dépendances, renforcer les en-têtes HTTP, exécuter des tests de charge OCPP, définir la rotation des secrets, déployer une terminaison TLS, superviser les processus et tester les sauvegardes. Une borne physique et un fournisseur de paiement réel devront être homologués séparément. Ces limites sont documentées afin de ne pas confondre un MVP sécurisé par conception avec une certification d'exploitation.

3.5 Modèle de données et diagramme de classes

3.5.1 Principes de modélisation

Le modèle conceptuel est organisé autour de quatre notions centrales : l'organisation qui exploite le réseau, la station qui fournit le service, le client qui utilise ce service et la session qui matérialise la recharge. Les autres classes complètent ce noyau pour couvrir l'exploitation et les deux relations commerciales de la plateforme.

La conception respecte les principes de normalisation relationnelle. Une information métier n'est portée que par l'entité qui en est responsable : les connecteurs sont séparés des stations, les paiements des sessions et les offres des adhésions. Les relations plusieurs-à-plusieurs sont résolues par des entités d'association, notamment l'adhésion d'un client à un plan et l'abonnement d'une organisation à une offre SaaS. Cette organisation évite les groupes répétitifs et les dépendances partielles.

Le diagramme reste volontairement conceptuel. Les tables de journalisation, les événements OCPP, les échantillons de mesure, les fichiers et les notifications ne sont pas représentés afin de préserver sa lisibilité. Ces éléments techniques conservent l'historique, mais ne modifient pas la structure des concepts métier principaux.

3.5.2 Domaines fonctionnels

3.5.2.1 Identité et séparation organisationnelle

Une organisation possède ses employés et son patrimoine de recharge. Les administrateurs, opérateurs et techniciens sont rattachés à une seule organisation. Le super administrateur et le client sont des acteurs globaux. Un même client peut utiliser les stations de plusieurs organisations ; son compte n'est donc pas rattaché à l'une d'elles. La relation avec une organisation apparaît au travers des adhésions et des sessions.

3.5.2.2 Réseau et recharge

Une organisation exploite plusieurs stations. Chaque station possède au moins un connecteur identifié dans le protocole OCPP. Une session associe un client à un connecteur pendant une période déterminée et conserve l'énergie délivrée, le coût calculé et son état. Elle référence le tarif appliqué et, lorsqu'il existe, l'avantage provenant d'une adhésion. Une session finalisée possède au plus un paiement principal.

L'état de disponibilité n'est pas une simple saisie. Il est projeté à partir des messages OCPP récents, des connecteurs, des opérations de maintenance et des éventuelles dérogations. Les traces techniques sont conservées séparément pour expliquer chaque transition.

3.5.2.3 Exploitation

Une station peut produire plusieurs alertes et être couverte par des plans de maintenance. Une alerte ou une échéance de maintenance peut conduire à une intervention. Celle-ci porte l'affectation du technicien, la priorité, le délai SLA et l'état de traitement. Les événements, photos, documents et rapports d'intervention constituent des preuves associées sans alourdir le diagramme global.

3.5.2.4 Commerce client et commerce SaaS

Deux cycles commerciaux sont séparés. Le premier relie le client à une organisation au moyen d'un plan de recharge et d'une adhésion ; il peut accorder une remise sur les sessions. Le second relie l'organisation à la plateforme au moyen d'une offre SaaS et d'un abonnement commercial ; il détermine la période d'essai, les capacités et le droit d'utiliser les fonctions de gestion. Cette distinction empêche de confondre le paiement d'une recharge et la facturation du logiciel.

3.5.3 Contraintes et invariants principaux

Table 3.9 – Invariants structurants du modèle

Domaine	Invariant
Identité	Un utilisateur actif possède exactement un rôle ; un rôle organisationnel exige une organisation active, alors qu'un rôle global interdit cette association.
Multi-tenant	Toute lecture ou écriture opérationnelle est limitée à l'organisation de l'utilisateur, sauf permission explicite du super administrateur.
Commissioning	Une station et ses connecteurs sont créés dans une même opération ; la référence et l'identité OCPP sont uniques.

Domaine	Invariant
Disponibilité	Le statut métier est dérivé des preuves OCPP récentes et des décisions opérationnelles ; il n'est pas saisi directement pour une station connectée.
Recharge	Un client et un connecteur ne peuvent participer qu'à une tentative ou une session active à la fois.
Mesure	L'énergie cumulée d'une session ne diminue jamais et chaque valeur reste rattachée à la transaction OCPP correspondante.
Paielement	Une même demande de paiement ne peut être traitée qu'une fois et un événement du fournisseur ne modifie qu'une transaction connue.
Tarification	Le tarif et la remise sont figés au début de la session afin que son coût reste explicable.
Intervention	Le technicien affecté, la station et l'intervention appartiennent à la même organisation.
Abonnement	L'abonnement client et l'abonnement SaaS suivent des cycles indépendants et ne partagent aucune facture.

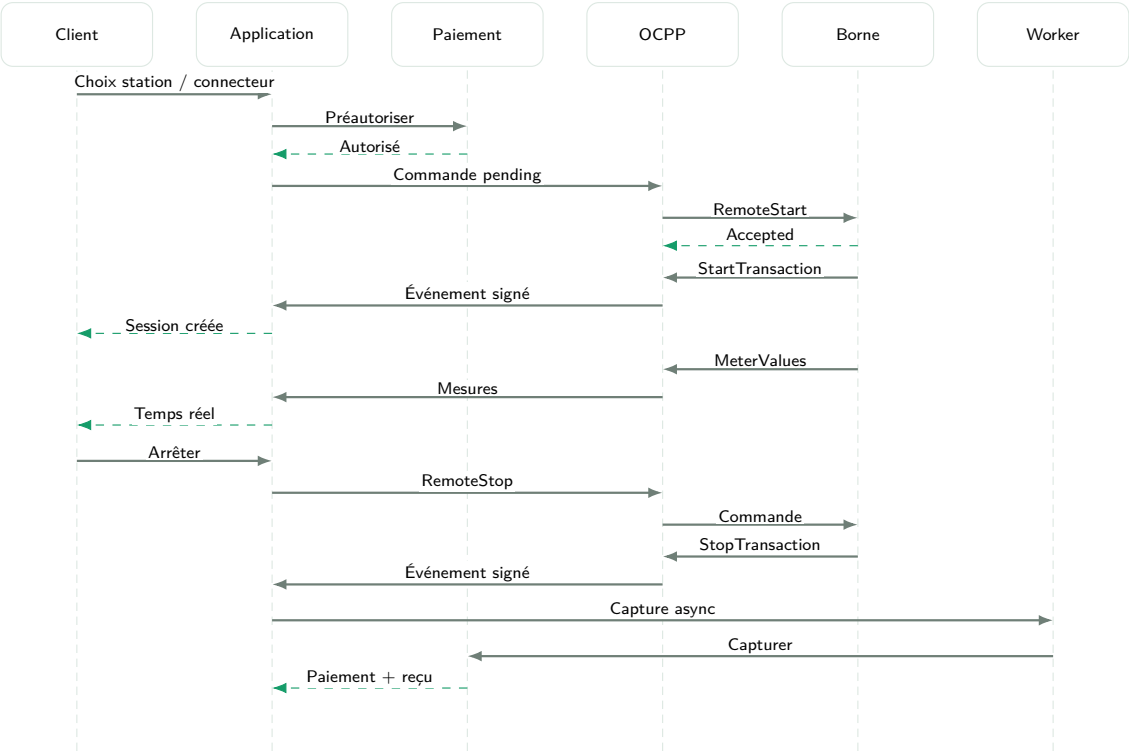


Figure 3.7 – Séquence de recharge distante et paiement simulé

3.5.4 Diagramme de classes global

La figure 3.9 présente une vue unique des classes métier principales. Les attributs sont limités aux informations nécessaires pour comprendre la responsabilité de chaque classe. Les cardinalités décrivent les relations fonctionnelles et non les détails propres à Eloquent ou aux migrations.

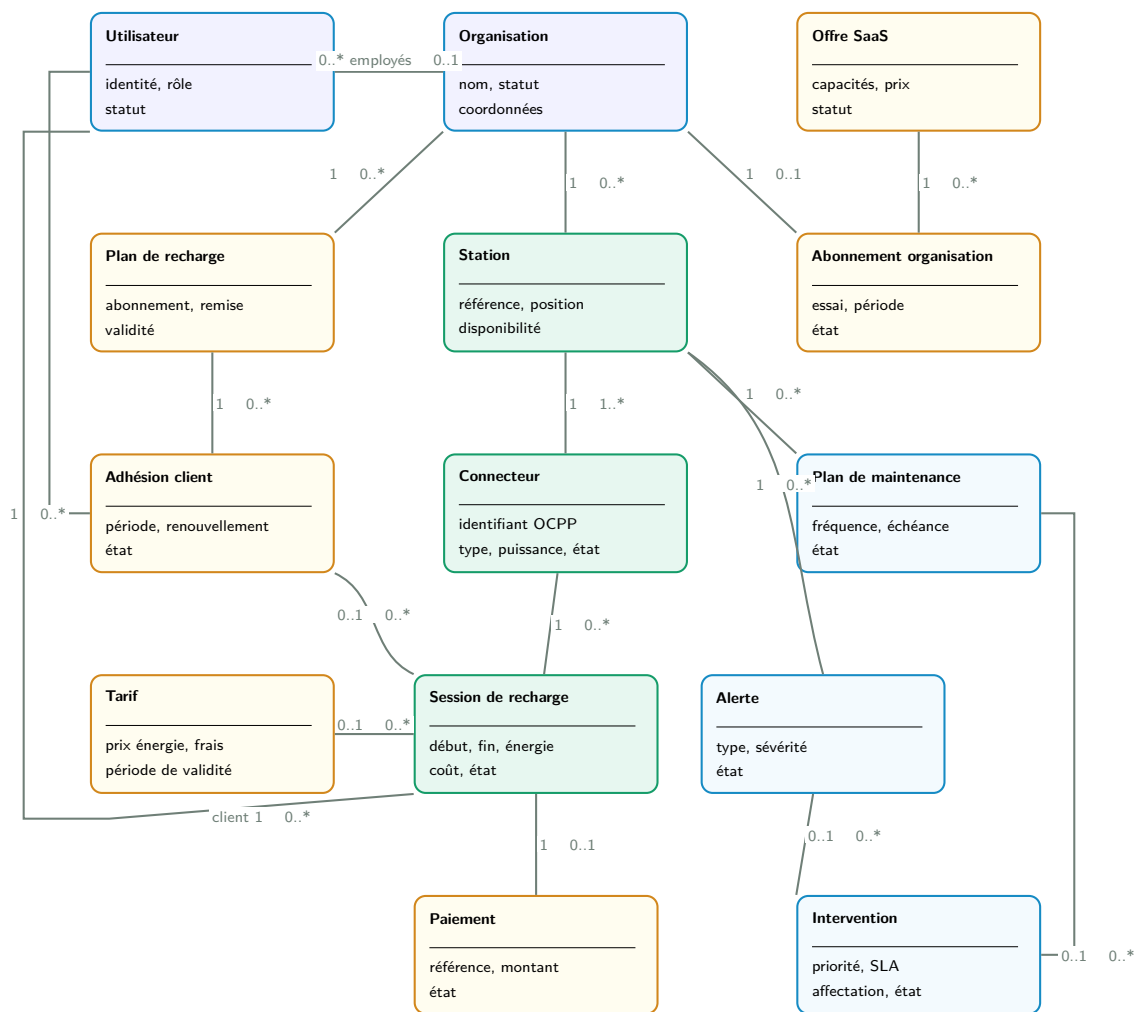


Figure 3.9 – Diagramme de classes conceptuel global de ChargeTrackr

Conclusion

La conception retenue combine la simplicité transactionnelle d'un monolithe modulaire et l'isolation d'une passerelle dédiée aux connexions OCPP longues. Les scénarios fonctionnels relient chaque acteur à un résultat vérifiable. Les états métier ne sont jamais assimilés aux messages bruts des bornes : des services de projection reconstruisent la disponibilité, les sessions et les alertes à partir de preuves horodatées. La séparation par

contrats rend le paiement, la messagerie et les équipements remplaçables, tandis que les politiques et le *scoping* organisationnel protègent les données. Le chapitre suivant montrera comment cette conception a été réalisée au fil des Sprints et validée par des tests.

Réalisation et validation

Introduction

Ce chapitre présente la traduction effective de la conception en une application exécutable. Il ne décrit pas seulement les écrans obtenus : il relie les objectifs des Sprints aux composants frontend, aux services métier, aux données persistées, aux processus d'infrastructure et aux contrôles qui permettent de vérifier leur comportement.

La réalisation a été menée au fil de Sprints dépendants. Chaque Sprint poursuit un objectif démontrable et s'appuie, lorsque nécessaire, sur les résultats du précédent. Cette progression a d'abord sécurisé l'identité et l'isolation des organisations, puis le référentiel des stations, la communication OCPP, les opérations terrain, la recharge, le paiement et enfin le pilotage commercial. Elle aboutit à un premier parcours vertical :

**borne simulée → OCPP → disponibilité → alerte ou carte → session → tarif
→ paiement simulé → reçu.**

Les résultats présentés correspondent à l'état du dépôt au 28 juillet 2026. Une fonctionnalité est qualifiée de *réalisée* lorsqu'elle possède un contrat backend, une interface exploitable et des tests automatisés adaptés. Le terme *validée localement* ne signifie pas qu'elle est déjà certifiée pour la production : les limites relatives à une borne physique, à un prestataire de paiement réel, au déploiement et à la charge sont indiquées explicitement.

4.1 Environnement et organisation de la réalisation

4.1.1 Poste, outils et versions

Le développement est effectué sur un poste Windows avec PHP installé localement et Docker Desktop appuyé sur WSL2 pour les composants conteneurisés. Cette organisation

permet de conserver un cycle rapide pour React et Laravel tout en isolant la passerelle OCPP, le simulateur SAP et le simulateur de paiement. Git assure la traçabilité du code et GitHub centralise le dépôt distant.

Table 4.1 – Environnement technique effectivement utilisé

Domaine	Technologies	Usage dans le projet
Frontend	React 19, TypeScript 6, Vite 8	Application monopage, typage statique et compilation de production.
Interface	Ant Design 6, Recharts, Leaflet, GSAP	Composants, graphiques, cartes et animations ciblées.
État et échanges	TanStack Query, Axios, Zustand	Cache serveur, appels HTTP et états locaux transverses.
Backend	PHP 8.5, Laravel 13	API REST, logique métier, validation, autorisation et tâches planifiées.
Identité	Sanctum, Socialite, Spatie Permission	Session SPA, Google OAuth 2.0, rôles et permissions.
Données	PostgreSQL 16, Eloquent	Persistance transactionnelle, relations et migrations.
Temps réel	Reverb, Laravel Echo	Notifications et mises à jour poussées par WebSocket.
OCPP	Python 3.11, OCPP 1.6J, simulateur SAP	Passerelle WebSocket, messages de borne et scénarios de test.
Paieement	Contrat PaymentGateway, WireMock 3	Fournisseur externe simulé, webhooks et scénarios d'erreur.
Email	Resend, queue Laravel	Vérification d'email, invitations et notifications asynchrones.
Qualité	PHPUnit 12, Oxlint, TypeScript	Tests backend, analyse frontend et validation de compilation.
Documentation	L ^A T _E X, TikZ, OpenAPI prévu	Rapport, schémas et future spécification de l'API.

Le fichier `composer.json` exige PHP 8.3 ou supérieur, ce qui laisse la possibilité d'utiliser une version stable en déploiement même si le poste local exécute PHP 8.5. Les dépendances sont verrouillées par `composer.lock` et `pnpm-lock.yaml`. Les valeurs sensibles sont placées dans les fichiers `.env`, exclus de Git, tandis que les fichiers `.env.example` documentent uniquement les clés attendues.

4.1.2 Structure du dépôt

Le dépôt conserve une séparation physique entre les composants tout en proposant des commandes communes à sa racine :

frontend/	application React, pages, fonctionnalités, composants, styles et ressources visuelles ;
backend/	API Laravel, services métier, migrations, politiques, jobs et tests ;
ocpp-gateway/	passerelle Python entre les bornes et l'API interne Laravel ;
infra/ocpp/	composition Docker et profils du simulateur SAP ;
infra/payment-simulator/	serveur WireMock et contrats du faux fournisseur de paiement ;
scripts/	commandes d'orchestration, de provisioning et de pilotage des scénarios ;
docs/	rapport, diagrammes et futurs livrables d'exploitation.

L'audit statique du dépôt met en évidence 49 contrôleurs, 45 classes de validation HTTP, 24 ressources API, 39 modèles, 11 politiques et 47 migrations. Le frontend contient 39 pages React. Ces chiffres ne constituent pas une métrique de qualité en eux-mêmes ; ils montrent cependant que la logique a été répartie selon les responsabilités Laravel plutôt que concentrée dans quelques contrôleurs.

4.1.3 Processus de développement

Le fichier `package.json` racine sert de façade aux commandes des trois environnements. Une session de développement complète utilise les processus suivants :

Table 4.2 – Commandes principales de l’environnement local

Commande	Responsabilité
<code>pnpm dev:frontend</code>	Lance Vite sur le port 5173.
<code>pnpm dev:backend</code>	Lance l’API Laravel sur le port 8000.
<code>pnpm dev:queue</code>	Traite les emails et travaux asynchrones.
<code>pnpm dev:scheduler</code>	Exécute les règles périodiques.
<code>pnpm dev:reverb</code>	Lance le serveur WebSocket applicatif.
<code>pnpm ocpp:up</code>	Démarre la passerelle et le simulateur OCPP.
<code>pnpm ocpp:status</code>	Affiche l’état d’une borne simulée.
<code>pnpm ocpp:plug</code> / <code>ocpp:unplug</code>	Simule la connexion ou le retrait physique d’un câble.
<code>pnpm payment:up</code>	Démarre le fournisseur WireMock.
<code>pnpm build:frontend</code>	Type-check et compile le frontend.
<code>pnpm test:backend</code>	Exécute la campagne PHPUnit.

Le scheduler recalcule la disponibilité toutes les 30 secondes, expire les commandes OCPP toutes les 10 secondes, contrôle les SLA chaque minute et génère les maintenances récurrentes toutes les heures. Les abonnements clients et organisationnels sont également synchronisés toutes les heures. L’option `withoutOverlapping` empêche qu’une nouvelle occurrence commence alors que la précédente n’est pas terminée.

4.1.4 Suivi des Sprints avec Git

La chronologie des commits reflète les risques du produit. Les premières modifications ont initialisé l’authentification, PostgreSQL et l’isolation multi-tenant. Les stations, tarifs et sessions ont ensuite fourni les premiers objets métier. L’intégration OAuth, les invitations et les demandes de démonstration ont stabilisé l’entrée dans la plateforme. Les blocs OCPP, disponibilité et opérations ont été intégrés avant le paiement et les tableaux de bord. Enfin, le commissioning, la télémétrie vérifiée et les documents ont complété le parcours d’exploitation.

Chaque bloc fonctionnel est suivi d’un commit cohérent puis poussé sur la branche `main`. Cette discipline facilite l’identification d’une régression et conserve un historique

exploitable pour le rapport. Les fichiers d’environnement, les dépendances installées et les sorties de compilation ne sont pas versionnés.

4.1.5 État synthétique des objectifs de Sprint

Table 4.3 – État de réalisation au 28 juillet 2026

Sprint	Capacité démon- trable	Validation actuelle	Limite principale
S01	Cadrage, backlog, architecture et prototype navigable	Documents de conception et parcours de référence validés	Ajustements continus selon les retours.
S02	Comptes, Google OAuth, invitations, démo, onboarding et rôles	API et parcours web testés	Microsoft OAuth reporté.
S03	Stations, connecteurs, cartes, documents et commissioning	CRUD, politiques et création atomique testés	Pas de borne physique.
S04	OCP, heartbeats, disponibilité, commandes et télémétrie	Simulateur SAP et projections testés	OCP 2.0.1 hors périmètre.
S05	Recharge, mesures, arrêt, paiement et reçu	Parcours vertical avec paiement simulé	Aucun débit bancaire réel.
S06	Alertes, interventions, maintenance, SLA et notifications	Cycles de vie et documents testés	Recette terrain à réaliser.
S07	Tarifs, plans clients et contrat SaaS organisationnel	Règles, limites et factures testées	Souscription SaaS non encaissée.

Sprint	Capacité démontable	Validation actuelle	Limite principale
S08	Dashboards, rapports, audit, paramètres et intégrations	APIs par rôle et exports testés	Finitions UX encore itératives.
S09	Exports généralisés et préparation documentaire	CSV, JSON et PDF disponibles par périmètre	Firmware et CI/CD reportés.

4.2 Réalisation des fondations techniques

4.2.1 Sprint 2 : entrée, identité et multi-tenant

4.2.1.1 Parcours d'entrée

L'entrée dans ChargeTrackr dépend du type d'utilisateur. Un client peut créer son propre compte local, vérifier son adresse email, définir son mot de passe ou continuer avec Google. Un employé ne s'inscrit pas librement : l'administrateur de son organisation crée le compte opérateur ou technicien, puis une invitation temporaire permet au destinataire d'activer son accès et de choisir son mot de passe. Enfin, un futur administrateur commence par une demande de démonstration. Le super administrateur étudie la demande, crée l'organisation et émet l'invitation de l'administrateur.

Cette séparation résout deux problèmes métier. Premièrement, un compte employé ne peut pas s'attribuer lui-même une organisation ou un rôle privilégié. Deuxièmement, une demande de démonstration ne crée aucune ressource sensible avant la décision du super administrateur. Les invitations sont révocables, renouvelables et limitées dans le temps.

Le backend s'appuie sur Sanctum pour la session de la SPA et sur Socialite pour Google OAuth. Le service `GoogleAccountService` associe l'identité externe à un compte local sans importer les rôles Google. L'API locale reste propriétaire du rôle, des permissions, de l'organisation et de l'état du compte. Les formulaires d'inscription, de connexion, d'activation et de demande de démonstration possèdent des règles de limitation et des réponses d'erreur structurées.

4.2.1.2 Isolation organisationnelle

Les administrateurs, opérateurs et techniciens appartiennent à une seule organisation. Un client reste indépendant et peut utiliser les stations ou souscrire aux plans de plusieurs organisations. Cette asymétrie est conservée dans le modèle de données : l'appartenance d'un employé détermine son périmètre, alors que les relations d'un client avec les organisations sont dérivées de ses abonnements et de ses sessions.

La protection ne repose pas sur le menu React. Le middleware `EnsureUserOrganizationScope`, les politiques et les requêtes Eloquent filtrent le périmètre côté serveur. Par exemple, un administrateur peut gérer les employés et consulter les clients de son organisation, mais il ne peut pas modifier un utilisateur d'une autre organisation en changeant simplement un identifiant dans l'URL. Les tests d'API couvrent ces refus d'accès.

4.2.1.3 Onboarding orienté vers la première valeur

Après sa première connexion, chaque rôle reçoit un parcours court qui confirme son contexte puis propose une première action utile. L'administrateur prépare l'identité de son organisation, invite un employé puis enregistre une station. L'opérateur rejoint directement la supervision, le technicien consulte son travail assigné et le client recherche une station. Le parcours peut être quitté et repris ; il ne bloque pas l'application.

Le rendu de ce parcours et sa progression visuelle sont présentés dans la figure 5.2 du chapitre consacré aux interfaces.

Les classes `AccountInvitationService` et `DemoProvisioningService`, les contrôleurs d'authentification et les politiques concrétisent l'objectif du Sprint. Les campagnes `AuthApiTest`, `GoogleOAuthTest`, `DemoRequestWorkflowTest`, `OnboardingApiTest`, `OrganizationManagementApiTest` et `UserManagementApiTest` fournissent la preuve automatisée principale.

4.2.2 Sprint 3 : stations, connecteurs et cartographie

4.2.2.1 Référentiel et vues adaptées

Une station possède une référence opérationnelle unique, un nom, une adresse, une position, un constructeur, un modèle, une puissance, une version OCPP et un ensemble de connecteurs physiques. Un connecteur porte un identifiant OCPP unique dans sa

station, un libellé lisible, un standard de prise, un type de courant et une puissance maximale.

Les écrans proposent des vues grille, liste, table et carte à partir du même contrat API. Les filtres exploitent l'état calculé, le type de connecteur et la recherche textuelle. L'opérateur et l'administrateur peuvent gérer les actifs de leur organisation. Le technicien les consulte sans création, tandis que le client ne voit que les stations publiques pertinentes.

La déclinaison de ce référentiel en vues opérationnelles est illustrée par la figure 5.4.

Leaflet et OpenStreetMap permettent de placer un marqueur par clic ou glisser-déposer. Les coordonnées restent éditables pour traiter les cas où la géolocalisation est imprécise. Côté client, la préférence de rayon *Near me* limite la recherche autour de la position autorisée par le navigateur. Le frontend peut également copier les coordonnées ou ouvrir l'itinéraire dans un service cartographique externe.

4.2.2.2 Commissioning atomique

La simple création d'une ligne en base ne suffit pas à rendre une borne exploitable. Un assistant de commissioning regroupe l'identité de la station, son matériel, ses connecteurs et son profil de simulateur. La référence interne et l'identité OCPP sont contrôlées avant la transaction. Pour une cible locale, le backend prépare un profil que le simulateur SAP peut enregistrer.

Commission a station
Create the station, its physical connectors and its local OCPP simulator profile together.

Station | Hardware | Review

01 / STATION IDENTITY

Place the station on the network
Use a unique operational reference and set its exact public location.

Station name: Lac 1 Fast Hub

Internal reference: CT-TUN-101

Location name: Les Berges du Lac 1

City: Tunis

Street address: Rue du Lac Biwa, Tunis

Exact station position
Click the map or drag the marker. Coordinates remain editable when needed. [Enter manually](#)

Cancel | Continue

Figure 4.1 – Première étape du commissioning guidé d’une station

`StationCommissioningService` exécute la création de la station, des connecteurs et des accès OCPP dans une transaction. Si une référence, une identité ou un connecteur est invalide, aucune création partielle ne subsiste. Le secret éventuel d’une borne physique n’est affiché qu’une fois et seul son condensat est conservé.

Les QR codes sont dérivés d’un jeton de connecteur destiné au parcours client. Le code affiché dans l’espace d’exploitation sert à imprimer l’étiquette physique ; le client utilise ensuite le scanner pour identifier le connecteur sans saisir manuellement sa référence. La plateforme ne considère toutefois pas le scan comme une preuve suffisante : la disponibilité et le démarrage restent validés par le backend et par OCPP.

Les documents de station sont persistés avec leur type, leur propriétaire et leur périmètre. Les autorisations distinguent contrats, garanties, notices et plans techniques. L’API ne retourne jamais un chemin arbitraire du système de fichiers ; le téléchargement passe par un contrôleur autorisé.

4.2.3 Sprint 4 : OCPP et disponibilité calculée

4.2.3.1 Connexion de la borne simulée

La passerelle Python reçoit les connexions WebSocket OCPP 1.6 JSON. Au démarrage, le simulateur envoie `BootNotification`, puis des `Heartbeat` périodiques et des `StatusNotification` pour chaque connecteur. La passerelle normalise le message et l'envoie à une API interne Laravel signée. Elle ne calcule pas l'état métier et n'écrit pas dans PostgreSQL.

`OcppEventIngestionService` enregistre l'événement brut utile à l'audit, met à jour les projections techniques et délègue les décisions à `AvailabilityProjectionService` ou `OcppTransactionProjectionService`. L'identifiant de la station présent dans l'URL WebSocket doit correspondre exactement à l'identité provisionnée.

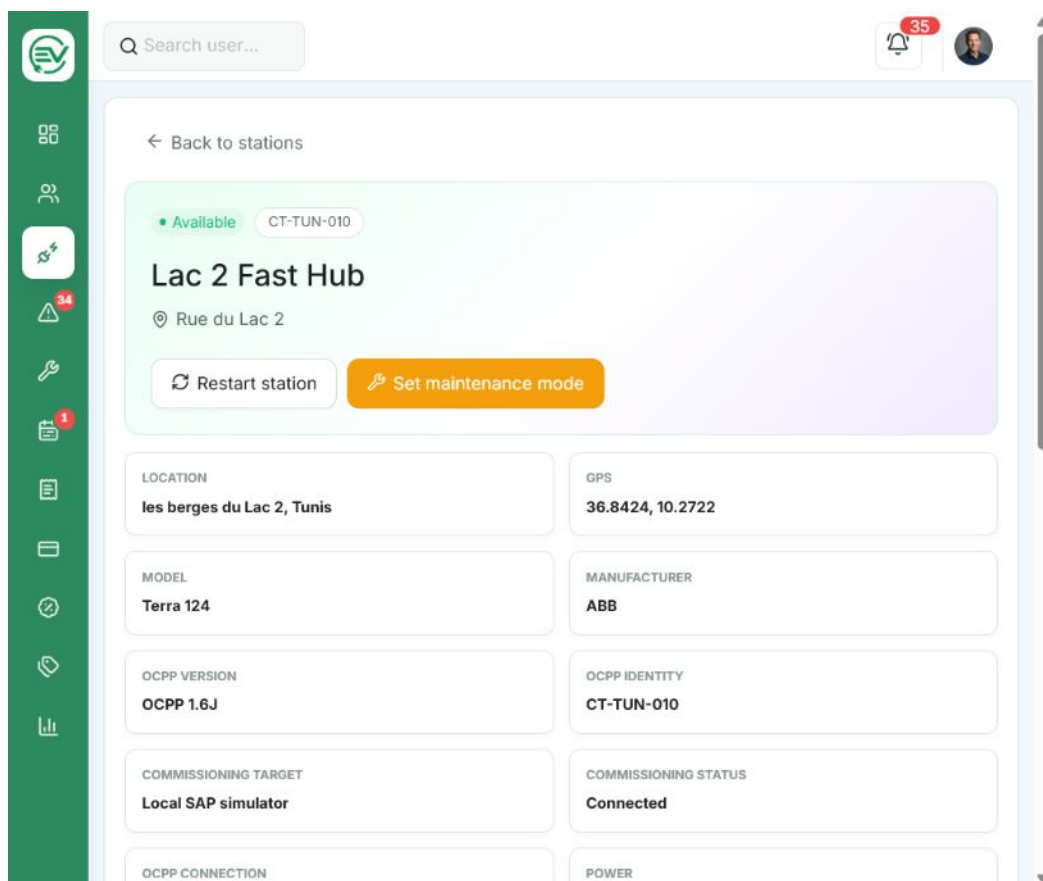


Figure 4.2 – Fiche d'une station reliée au simulateur SAP

4.2.3.2 Règles de projection

L'état affiché n'est pas un champ saisi manuellement. Il est reconstruit à partir des preuves disponibles selon un ordre de priorité. Une maintenance active ou une désactivation administrative peut rendre l'actif indisponible. Une absence de signal au-delà du délai configuré produit l'état déconnecté. Un code d'erreur OCPP actif produit l'état en défaut. Une transaction active rend le connecteur occupé. En l'absence de ces conditions, le dernier `StatusNotification` détermine notamment les états disponible, réservé ou indisponible.

Table 4.4 – Exemples de règles de disponibilité réalisées

État projeté	Preuve dominante	Conséquence
Disponible	Statut OCPP <code>Available</code> , signal récent et aucune contrainte prioritaire	Le connecteur peut être proposé au client.
Occupé	Transaction OCPP active ou statut <code>Charging</code>	Le connecteur n'est plus sélectionnable pour une nouvelle session.
Réservé	Statut <code>Reserved</code> ou réservation valide	L'accès est limité au bénéficiaire attendu.
Maintenance	Fenêtre de maintenance active ou commande de changement de disponibilité	Le connecteur est retiré du service.
En défaut	Code d'erreur ou statut <code>Faulted</code>	Une alerte opérationnelle est créée ou actualisée.
Déconnecté	Aucun heartbeat ou événement dans le délai configuré	La station est indisponible et une alerte de communication est ouverte.

Le recalcul est idempotent : recevoir plusieurs fois la même preuve ne crée pas une suite d'alertes identiques. Lorsque la communication revient, la projection est réévaluée et l'alerte peut être résolue selon sa règle. Les transitions sont horodatées afin de calculer l'uptime sur une période.

4.2.3.3 Commandes et télémétrie

Depuis la fiche station, un utilisateur autorisé peut demander **Reset**, **UnlockConnector** ou **ChangeAvailability**. Laravel crée d'abord une commande avec un état attendu, puis la passerelle la récupère et transmet l'appel à la borne. La réponse de la borne met à jour le résultat ; une tâche planifiée expire les commandes qui ne reçoivent pas de réponse. Le bouton n'affirme donc jamais qu'une action physique a réussi avant l'accusé OCPP.

MeterValues alimente la puissance et l'énergie réellement rapportées. L'interface distingue ces mesures des statistiques calculées à partir des sessions. Lorsqu'aucun **MeterValues** n'a été reçu, elle affiche *No signal* au lieu de générer une série visuelle artificielle.

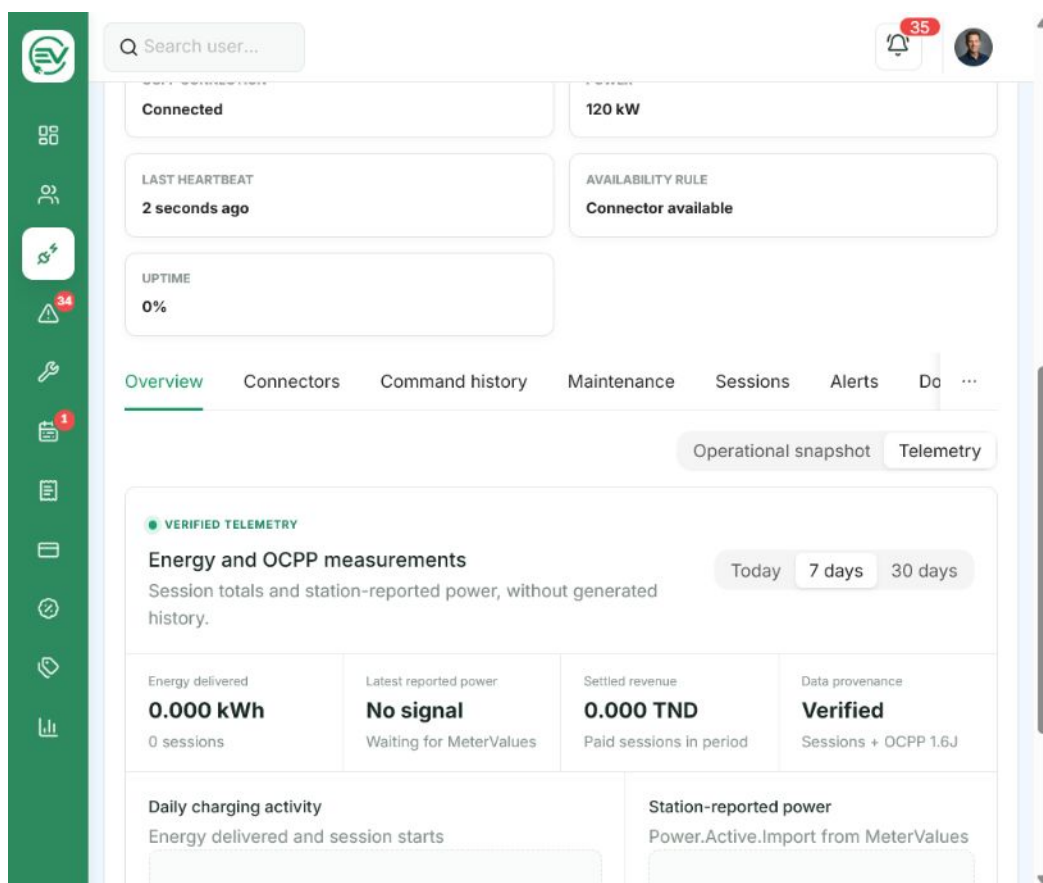


Figure 4.3 – Télémétrie vérifiée sans génération de mesures fictives

Les tests `OcppGatewayApiTest`, `OcppSimulatorFleetTest`, `OcppSupervisionApiTest`, `OcppTransactionProjectionTest`, `AvailabilityProjectionTest` et `StationTelemetryApiTest` vérifient les contrats, la déduplication, les transitions et l'isolation des organisations.

4.3 Réalisation des parcours métier

4.3.1 Sprint 6 : exploitation terrain

4.3.1.1 Alertes et qualification

Une alerte représente un symptôme opérationnel à traiter, et non une intervention. Elle possède une sévérité, un statut, une station, un contexte technique et une chronologie. Elle peut provenir automatiquement d'une perte de communication, d'un défaut OCPP ou d'une règle de maintenance, mais un utilisateur autorisé peut également créer une alerte manuelle.

L'interface utilise une disposition en deux panneaux : la liste filtrable reste visible pendant que le détail, les actions et l'historique de l'alerte sélectionnée sont consultés. Les filtres de sévérité et d'état ne modifient pas la donnée ; ils sont convertis en paramètres API. Les compteurs de la barre latérale sont dérivés des notifications non lues liées à ce contexte. La figure 5.5 présente l'organisation visuelle de ce centre opérationnel dans le chapitre suivant.

4.3.1.2 Interventions, maintenance et SLA

Une intervention matérialise le travail humain déclenché par une alerte ou une décision d'exploitation. L'opérateur la crée, choisit une priorité et l'assigne à un technicien de la même organisation. Le technicien accepte le travail, ajoute ses notes, documente le diagnostic, les actions et les pièces, joint des photos *avant/après*, puis soumet son rapport. L'opérateur contrôle ensuite le résultat et clôt l'intervention.

La maintenance répond à un besoin différent. Elle planifie une action préventive ou corrective sur une période, indépendamment de l'existence immédiate d'une alerte. `MaintenancePlanService` gère le plan, tandis que `MaintenanceLifecycleService` contrôle les transitions. Le job `GenerateMaintenanceOccurrences` matérialise les occurrences récurrentes.

`NotificationSlaService` analyse chaque minute les échéances approchantes ou dépassées. Les notifications sont enregistrées en base, diffusées par Reverb et éventuellement envoyées par email via la queue. L'utilisateur peut marquer une notification lue globalement ou dans le contexte de la page consultée.

Les documents d'alerte, d'intervention et de maintenance sont générés par le backend. Le

rapport PDF reprend la référence, les acteurs, les dates, les preuves et l'historique sans dépendre d'une capture de l'écran. Les pièces jointes passent par des routes autorisées et sont supprimables selon le cycle de vie de la ressource.

4.3.2 Sprint 5 : recharge, transaction et paiement

4.3.2.1 Assistant de recharge client

Le client peut ouvrir l'assistant depuis *Start session*, depuis une fiche station ou depuis la carte. Dans les deux derniers cas, la station est déjà sélectionnée et le parcours commence directement au choix du connecteur. Le workflow se compose de cinq étapes :

1. sélectionner une station disponible ;
2. choisir un connecteur compatible et libre ;
3. brancher le câble et attendre la confirmation OCPP ;
4. autoriser le paiement simulé et vérifier le tarif applicable ;
5. envoyer la demande de démarrage distant et attendre la transaction OCPP.

Le passage de l'étape de branchement n'est pas manuel. Le frontend interroge la tentative de recharge et reçoit la mise à jour du connecteur ; il avance uniquement lorsque le backend confirme une preuve OCPP compatible. Les pictogrammes de prises aident le client à reconnaître CCS2, Type 2, CHAdeMO ou un autre standard.

4.3.2.2 Cycle de session

`ChargingAttemptService` conserve le contexte entre les étapes et refuse une station, un connecteur ou un tarif devenus invalides. Après autorisation, `OcppCommandService` demande un `RemoteStartTransaction`. Le simulateur répond, puis `StartTransaction` devient la preuve de démarrage. Les `MeterValues` actualisent l'énergie, la puissance et l'estimation. Le client peut laisser la vue active en arrière-plan et y revenir depuis la page des sessions.

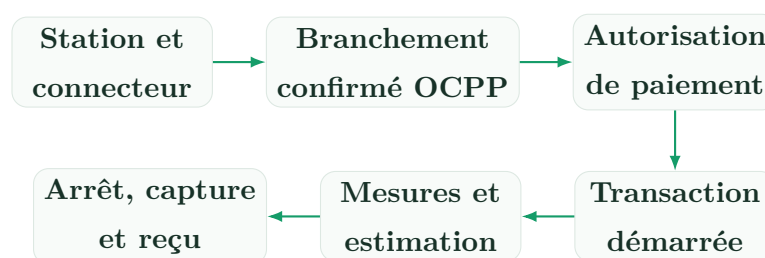


Figure 4.4 – Parcours vertical d'une recharge

L'arrêt peut être demandé par le client, par un opérateur autorisé, par la borne ou par une condition technique. **StopTransaction** clôt la transaction et fige les valeurs finales. Le coût n'est pas calculé dans React : **TariffResolver** sélectionne la règle applicable, puis le backend calcule les composantes énergie, temps, forfait et remise.

4.3.2.3 Adaptateur de paiement simulé

Le domaine dépend de l'interface **PaymentGateway**. Deux implémentations sont disponibles : **SimulatedPaymentAdapter**, utile aux tests unitaires, et **WireMockPaymentAdapter**, qui appelle un serveur HTTP externe local. Les opérations fonctionnelles sont l'autorisation, la capture, la libération et le débit direct.

WireMock reproduit des succès, refus, délais, erreurs serveur et webhooks. Le webhook est signé ; **PaymentWebhookSignature** vérifie son authenticité et **PaymentProviderEventService** assure l'idempotence. Un même événement rejoué ne capture donc pas deux fois une session. Le reçu est consultable dans l'application et exportable en PDF.

Ce choix ne simule pas une banque au niveau réglementaire. Il valide la frontière technique qui permettra d'ajouter ultérieurement ClicToPay, e-DINAR/D17, GPGCheckout, Konnect ou Flouci. L'adaptateur réel devra ajouter la conformité du prestataire, le rapprochement, la gestion des litiges et des remboursements.

4.3.3 Sprint 7 : tarification et commerce

4.3.3.1 Tarifs et plans de recharge

Un tarif appartient à une organisation et peut combiner un prix au kWh, un prix au temps, un coût fixe et des règles horaires. Les affectations définissent son périmètre : organisation, station ou connecteur. Lorsqu'une session commence, **TariffResolver** applique l'affectation la plus spécifique et conserve une référence au tarif retenu afin qu'une modification future ne change pas rétroactivement le calcul.

Les plans clients sont des avantages proposés par une organisation : remise sur la recharge, éventuel abonnement périodique et conditions d'accès. Un client peut cumuler des abonnements auprès de plusieurs réseaux, car son utilisation d'une borne ne le transforme pas automatiquement en client exclusif de son propriétaire. La simple recharge crée une relation transactionnelle ; l'abonnement demeure une décision explicite.

4.3.3.2 Abonnement SaaS de l'organisation

Le contrat commercial de l'organisation est volontairement séparé des paiements des conducteurs. Il détermine la période d'essai, le nombre maximal d'employés et de stations, les fonctions accessibles, la date de renouvellement et les factures adressées à l'organisation. `OrganizationEntitlementService` contrôle les limites avant la création d'un actif, tandis que `OrganizationSubscriptionLifecycleService` traite essai, activation, suspension, expiration et restauration.

L'interface de suivi de l'abonnement est reproduite dans la figure 5.6.

Le super administrateur gère le catalogue SaaS et le portefeuille global. L'administrateur consulte son offre, sa consommation de capacité et ses factures, puis émet une demande de changement. Cette demande évite qu'un administrateur modifie directement un contrat financier. L'encaissement SaaS reste simulé ou manuel dans le MVP.

4.3.4 Sprint 8 : pilotage, reporting et gouvernance

4.3.4.1 Tableaux de bord par rôle

`DashboardService` compose une réponse différente selon le rôle plutôt que de renvoyer le même tableau de bord avec des libellés modifiés. Le super administrateur supervise les organisations et les intégrations. L'administrateur observe les employés, clients, revenus, stations et performances de son organisation. L'opérateur suit disponibilité, incidents et sessions. Le technicien voit sa charge terrain et ses SLA. Le client consulte ses recharges, paiements et abonnements.

La figure 5.3 montre la composition retenue pour le tableau de bord de l'administrateur. Les indicateurs utilisent les mêmes règles et périmètres que les écrans de détail. Par exemple, la disponibilité du tableau de bord passe par `AvailabilityMetrics` et ne moyenne pas des valeurs fictives du frontend. Les sélecteurs de période sont transformés en bornes temporelles côté API.

4.3.4.2 Rapports et échanges internes

Les espaces d'analyse sont adaptés au périmètre du rôle : `/reporting/platform`, `/organization`, `/operations` et `/field`. Les exports PDF, JSON et CSV réutilisent les mêmes filtres et autorisations. Les rapports internes sont des objets distincts des exports : un employé peut composer un rapport, joindre des fichiers, choisir un destinataire de son

organisation, l'envoyer, puis suivre lecture et archivage.

La figure 5.7 illustre cet espace d'analyse et d'échange.

4.3.4.3 Administration globale

Le super administrateur dispose d'un espace cohérent avec la navigation du produit, mais orienté opérations de plateforme : organisations, utilisateurs, rôles, permissions, demandes de démonstration, catalogue commercial, audit, intégrations et paramètres. Les secrets OAuth, OCPP, email ou paiement ne sont jamais renvoyés par les endpoints d'intégration. L'écran expose uniquement l'état de configuration et la date du dernier contrôle.

`PlatformSettingService` limite les paramètres modifiables à une liste validée. Les valeurs sensibles restent dans l'environnement. Toute modification globale est attribuée à son auteur dans le journal d'audit. Les paramètres personnels, tels que fuseau horaire, préférences de notification et rayon de recherche, restent séparés des contrôles globaux.

4.3.5 Sprint 9 : exports et évolutions différées

`ReportExportService` centralise les formats PDF, CSV et JSON. OpenSpout est disponible pour les formats tabulaires plus riches, tandis que DomPDF produit les documents visuels. Les reçus, factures et rapports opérationnels sont générés près de leur ressource afin que l'utilisateur les retrouve dans le workflow concerné, et non uniquement dans une page de rapports générique.

La gestion distante du firmware n'est pas simulée comme si elle était achevée. Elle nécessite une matrice de compatibilité matériel/logiciel, la signature des paquets, un hébergement sécurisé, le suivi de progression, une stratégie de retour arrière et une validation sur borne. Elle reste donc une évolution du Product Backlog non engagée dans le MVP, conformément au risque de 13 points associé à US-38.

4.4 Intégration et validation

4.4.1 Intégration bout en bout

L'intégration verticale est vérifiable sans accès direct du frontend à la borne ou au fournisseur de paiement. React appelle uniquement l'API publique Laravel. Laravel crée

les intentions métier et les commandes. La passerelle OCPP échange avec le simulateur puis renvoie des événements signés. PostgreSQL conserve les preuves et projections. Reverb diffuse les changements utiles à l'interface. WireMock représente un système financier externe indépendant.

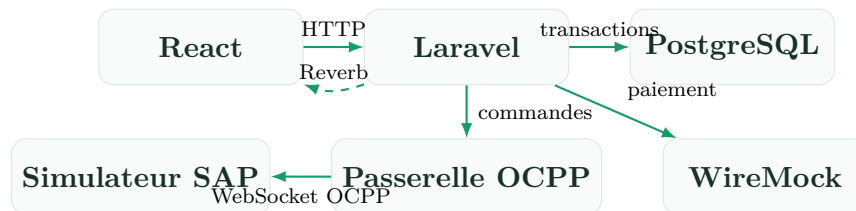


Figure 4.5 – Composants traversés par le MVP vertical

Cette structure a facilité le diagnostic. Un écran n'actualisant pas une donnée peut être analysé successivement au niveau de la requête React, de la réponse API, de la projection PostgreSQL, de l'événement reçu par la passerelle et du message émis par le simulateur. Le fournisseur de paiement peut de la même manière être observé sans modifier les tables depuis WireMock.

4.4.2 Stratégie de tests

La stratégie accorde la priorité aux invariants backend, car le frontend ne doit pas être la seule barrière contre une action interdite. Les tests utilisent une base SQLite en mémoire, une queue synchrone, un mailer en mémoire et un diffuseur nul. Ils restent ainsi reproductibles et n'envoient ni email réel ni événement WebSocket pendant la campagne.

Table 4.5 – Niveaux de validation appliqués

Niveau	Objet	Exemples	Preuve
Tests métier	Invariants, projections et transitions	disponibilité, transactions OCPP, tarifs, cycles commerciaux	Assertions PHPUnit.
Tests API	Validation, autorisation, sérialisation et effets	authentification, stations, alertes, rapports, paiement	Réponses et base vérifiées.

Niveau	Objet	Exemples	Preuve
Tests d'isolation	Accès croisé entre organisations	utilisateurs, actifs, interventions, exports	Réponses 403/404 attendues.
Tests d'adaptateur	Contrat avec un service remplaçable	WireMock, webhook signé, Google OAuth simulé	Doubles et serveur local.
Tests OCPP	Lecture des profils et scénarios du simulateur	connexion, plug/unplug, transactions, projections	Node Test et PHPUnit.
Qualité frontend	Types, imports et règles statiques	compilation Vite et Oxlint	Commandes sans erreur.
Vérification visuelle	Mise en page et parcours représentatifs	dashboards, stations, cartes, formulaires, documents	Inspection navigateur/PDF.

4.4.3 Résultats mesurés

La campagne exécutée le 28 juillet 2026 produit les résultats suivants :

Table 4.6 – Résultats de validation du dépôt

Contrôle	Résultat	Portée
<code>php artisan test -compact</code>	201 tests, 1 489 assertions	37 classes de tests Feature et tests unitaires chargés par PHPUnit.
<code>pnpm test:ocpp-tools</code>	2 tests, 0 échec	Lecture et contrôle des états du simulateur.
<code>pnpm lint:frontend</code>	Succès	Analyse Oxlint du code TypeScript/React.
<code>pnpm build:frontend</code>	Succès	Type-check TypeScript et bundle Vite de production.
Inventaire des routes	186 routes, dont 177 API	75 GET, 67 POST, 23 PATCH/PUT et 12 DELETE.

La réussite de la compilation ne prouve pas seule la qualité UX, et le nombre de tests ne remplace pas une recette. Elle montre néanmoins que les contrats principaux, les migrations et la majorité des cycles de vie restent cohérents après l'intégration des développements successifs.

4.4.4 Anomalies représentatives et non-régression

Plusieurs anomalies rencontrées pendant la réalisation ont renforcé les contrôles :

- une requête depuis l'adresse locale du réseau pouvait recevoir une erreur 419 lorsque les domaines de session et les origines CSRF ne couvraient que `localhost`. La configuration a été rendue cohérente pour l'hôte local et l'adresse LAN de développement ;
- une double soumission involontaire d'une invitation déclenchait une erreur 429. Le bouton est verrouillé pendant la requête et les limites restent actives côté serveur ;
- un tri `created_at` combiné à un `GROUP BY` était accepté par certains moteurs mais refusé par PostgreSQL. La requête d'agrégation des photos d'intervention a été corrigée et couverte par un test ;
- `crypto.randomUUID()` échouait sur un navigateur mobile servi en HTTP non sécurisé. L'identifiant temporaire utilise désormais une solution compatible qui ne modifie pas l'identité métier du backend ;
- des URL WebSocket figées sur `localhost` empêchaient Reverb de fonctionner depuis un téléphone. Le frontend construit l'hôte de développement à partir de sa configuration plutôt que de supposer une machine unique.

Les réponses d'erreur de l'API sont normalisées pour ne pas exposer une trace SQL, un chemin local ou un secret. En environnement de production, `APP_DEBUG=false` reste obligatoire.

4.4.5 Performance et robustesse

Les listes volumineuses utilisent pagination et filtres côté serveur. Les relations nécessaires aux ressources API sont chargées explicitement afin de limiter les requêtes répétitives. Les agrégats de tableaux de bord sont calculés par période et périmètre plutôt que reconstruits dans le navigateur. Les travaux lents, notamment emails et certaines notifications, passent par la queue.

Pour OCPP, la passerelle conserve la connexion WebSocket mais délègue rapidement l'événement à Laravel. Le recalcul de disponibilité est idempotent et protégé contre le

chevauchement. Les événements et mesures restent horodatés, ce qui permet d'appliquer un délai de non-réponse sans dépendre de l'heure du navigateur.

La compilation frontend signale toutefois plusieurs bundles supérieurs à 500 kio ; le principal fichier JavaScript atteint environ 767 kio. Ce résultat n'empêche pas le MVP local, mais justifie une optimisation avant production : découpage par route, chargement différé de PDF.js et du scanner QR, séparation des icônes et graphiques, puis mesure Lighthouse sur réseau mobile.

4.4.6 Sécurité appliquée

Les mesures suivantes sont effectivement présentes dans le code :

- session SPA protégée par Sanctum et cookie CSRF ;
- validation systématique des écritures par *Form Requests* ;
- politiques et scoping organisationnel côté serveur ;
- hachage des mots de passe et des secrets de station ;
- vérification des signatures de webhooks et déduplication des événements ;
- limitation des tentatives de connexion, d'activation et d'invitation ;
- routes internes OCPP séparées des routes publiques ;
- journal d'audit pour les actions sensibles ;
- fichiers et documents servis par contrôleurs autorisés ;
- secrets exclus du dépôt et masqués dans les réponses d'intégration.

Un audit de sécurité complet reste nécessaire avant exposition publique : rotation des clés, TLS/WSS, durcissement des cookies, en-têtes de sécurité, analyse des dépendances, tests IDOR systématiques, sauvegardes chiffrées et supervision des événements anormaux.

4.4.7 Limites de la validation

Frontière des résultats

- Le simulateur SAP valide les messages OCPP 1.6J, mais pas le comportement électrique, le modem ou le firmware d'une borne réelle.
- WireMock valide le contrat HTTP et les webhooks, mais aucune transaction bancaire ni exigence PCI DSS.
- Les tests automatisés n'ont pas encore été complétés par une campagne de

charge OCPP multi-borne ni par des tests de performance formels.

- Le déploiement de production, TLS, sauvegardes, supervision et CI/CD ne sont pas encore qualifiés.
- La spécification OpenAPI/Swagger, les manuels et le procès-verbal de recette constituent des livrables distincts encore à finaliser.
- OCPP 2.0.1, Microsoft OAuth, le paiement réel et le firmware restent hors du MVP validé.

4.4.8 Traçabilité avec le backlog et les Sprints

Table 4.7 – Preuves de réalisation par Sprint

Sprint	User stories	Preuves dans le code	Tests principaux
S01	US-27, 39 et travaux de conception	Backlog, cas d'utilisation global, architecture et prototype	Relectures, validation des parcours et démonstration du prototype.
S02	US-01–04, 06	Auth, Google, invitations, démo, onboarding, policies	Auth, OAuth, onboarding, utilisateurs, organisations.
S03	US-07–13, 37	Stations, connecteurs, cartes, commissioning, documents	Station API, commissioning, documents.
S04	US-14–16, 19	Gateway, ingestion, projection, commandes, télé-métrie	OCPP gateway, flotte, supervision, disponibilité, télé-métrie.
S05	US-24–26, 28–30	Tentatives, sessions, transactions, paiements, reçus	Recharge distante, session/paiement, webhook WireMock.
S06	US-20–23, 40	Alertes, interventions, maintenance, SLA, notifications	Alertes, rapports termin, maintenance, notifications.

Sprint	User stories	Preuves dans le code	Tests principaux
S07	US-31, 32, 41, 42	Tarifs, plans, abonnements clients et SaaS	Tarifs, plans, commercial organisationnel.
S08	US-05, 17, 18, 33-36	Dashboards, reporting, audit, paramètres, intégrations	Dashboard, rapports, gouvernance, recherche.
S09	Stabilisation et livrables	Exports, documentation, cahier de tests et préparation de la recette	Compilation, contrôles transversaux et validation des documents.

4.5 Documentation technique produite et restante

Le dépôt contient déjà les scripts d'installation, les configurations d'exemple, les commandes d'exploitation des simulateurs et le présent rapport. Les noms des routes et les ressources Laravel forment une base vérifiable pour produire une spécification OpenAPI 3.1. Celle-ci sera exposée avec Swagger UI pour l'exploration, mais le fichier OpenAPI versionné restera la source du livrable API.

Les livrables à finaliser sont le manuel utilisateur par rôle, le manuel administrateur, le cahier de tests, la procédure d'installation, la documentation des processus OCPP et le procès-verbal de recette. Ils devront reprendre les mêmes identifiants de user stories et critères d'acceptation afin d'éviter une documentation déconnectée du produit.

Conclusion

La réalisation confirme la faisabilité de l'architecture proposée. Le dépôt contient une application React par rôle, une API Laravel protégée, un modèle PostgreSQL couvrant les domaines principaux, une passerelle OCPP indépendante et des adaptateurs pour les services externes. Le simulateur SAP produit des messages OCPP effectivement ingérés ; les services Laravel calculent ensuite les projections de disponibilité, les sessions, les alertes et les données de pilotage. Le paiement externe reste simulé, mais son contrat permet de remplacer WireMock sans réécrire le parcours de recharge.

La campagne locale atteint 201 tests backend et 1 489 assertions, complétés par deux tests des outils OCPP, le lint frontend et une compilation de production. Ces résultats

valident le MVP vertical dans l'environnement du stage. Les travaux encore ouverts concernent principalement l'industrialisation, la mesure de charge, la documentation OpenAPI, la recette formelle, la connexion à un prestataire financier réel, l'essai sur une borne physique et la gestion sécurisée du firmware.

Captures des interfaces

Introduction

Ce chapitre présente le résultat visuel de ChargeTrackr et complète la description technique du chapitre précédent. Les captures ne servent donc pas à prouver une règle métier ou un échange protocolaire : elles montrent la manière dont les fonctions réalisées sont organisées pour les utilisateurs. Les trois captures strictement techniques du commissioning, de la connexion au simulateur SAP et de la télémétrie OCPP restent volontairement dans le chapitre 4 afin d'éviter toute duplication.

5.1 Page d'accueil

La page d'accueil constitue le point d'entrée public de ChargeTrackr. Elle présente directement le domaine de la plateforme à travers une photographie de bornes de recharge, une proposition de valeur concise et un aperçu du pilotage opérationnel. La navigation conduit aux capacités du produit, aux ressources et au contact, tandis que les actions principales distinguent la connexion, la création d'un compte client et la demande de démonstration destinée à une future organisation.

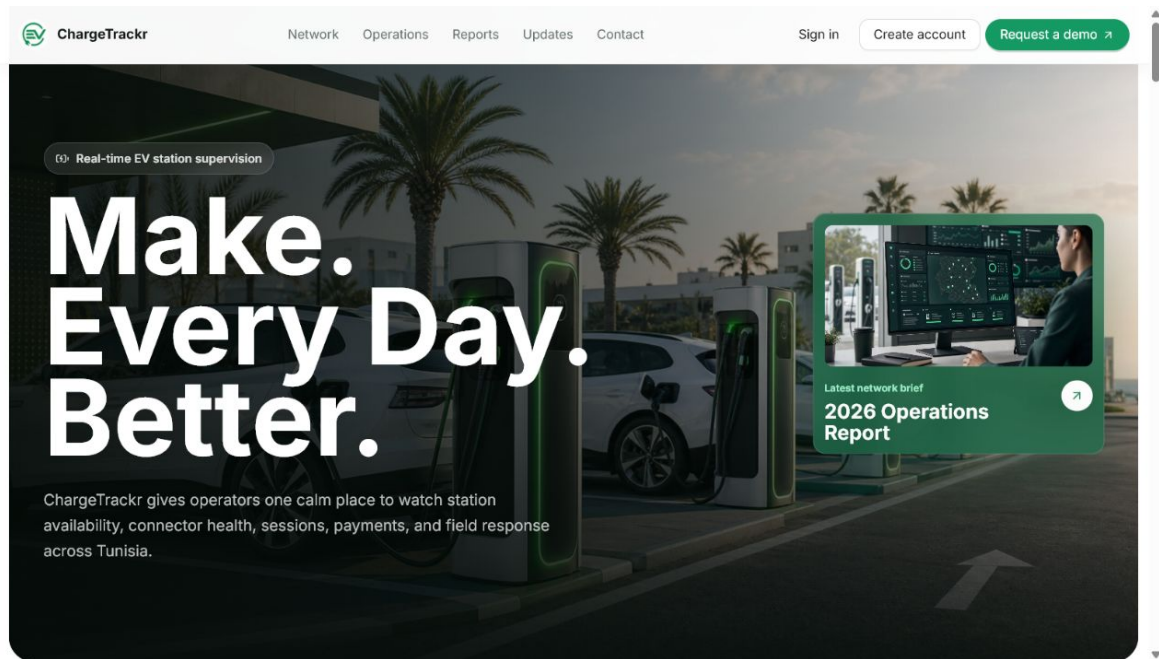


Figure 5.1 – Page d'accueil publique de ChargeTrackr

Le premier écran privilégie la compréhension immédiate du service et conserve les actions essentielles au-dessus de la ligne de flottaison. Les sections suivantes détaillent la supervision temps réel, la gestion des opérations, les indicateurs et le fonctionnement général. Cette entrée publique reste séparée de la coque authentifiée afin de ne pas exposer la navigation métier avant la connexion.

5.2 Entrée et prise en main

Les pages publiques donnent accès à la présentation du produit, à la demande de démonstration, à la connexion locale ou Google et à la création d'un compte client. Après l'authentification, un onboarding court remplace la visite exhaustive des fonctionnalités. Il rappelle le contexte d'accès, demande uniquement les informations utiles au rôle et conduit vers une première action concrète.

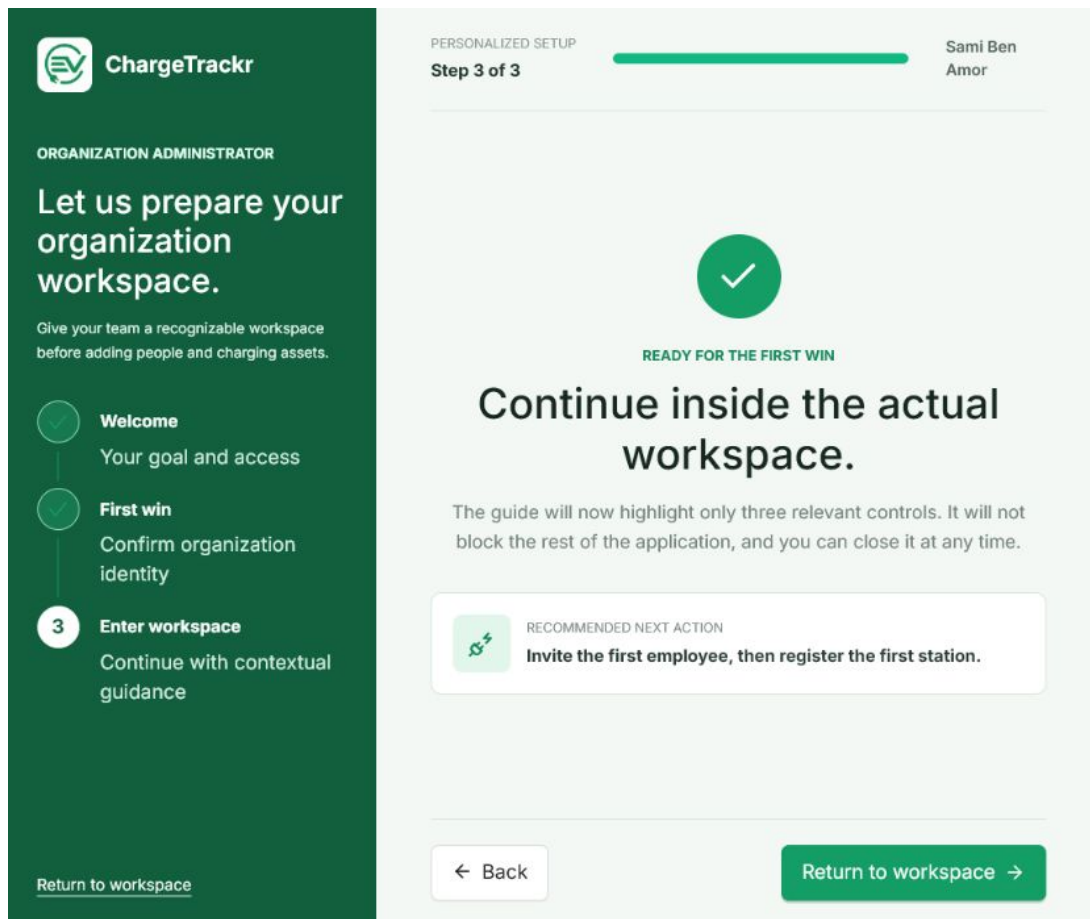


Figure 5.2 – Onboarding personnalisé de l'administrateur

La progression reste visible dans la colonne latérale et les étapes déjà terminées peuvent être identifiées immédiatement. L'utilisateur peut quitter le parcours sans perdre l'accès à son espace. Pour un administrateur, la première valeur consiste à confirmer l'organisation, inviter son équipe et préparer son premier actif ; pour les autres rôles, les contenus sont adaptés à leur mission.

5.3 Navigation et adaptation aux rôles

La coque applicative reste commune afin de réduire l'effort d'apprentissage : logo et menu principal à gauche, recherche et notifications en haut, contenu contextuel au centre. Cette cohérence ne signifie pas que tous les utilisateurs reçoivent le même tableau de bord. Les routes, indicateurs et actions sont composés à partir du rôle et du périmètre autorisé.

Table 5.1 – Orientation des interfaces selon le rôle

Rôle	Priorité de l'interface	Actions mises en avant
Super administrateur	Gouvernance de la plateforme et portefeuille d'organisations	Traiter les demandes, gérer les organisations, contrôler les intégrations et les paramètres globaux.
Administrateur	Pilotage métier d'une organisation	Gérer les employés, les actifs, les tarifs, l'abonnement commercial et les rapports.
Opérateur	Continuité du service	Superviser les stations, qualifier les alertes, commander les bornes et affecter les interventions.
Technicien	Exécution du travail terrain	Consulter les missions, mettre à jour leur avancement, joindre les preuves et soumettre un rapport.
Client	Recharge et dépenses personnelles	Rechercher une station, démarrer une recharge, suivre la session, payer et consulter les reçus.

5.4 Pilotage de l'organisation

Le tableau de bord de l'administrateur associe un bandeau de période, l'identité de l'organisation, des raccourcis opérationnels et des indicateurs vérifiés. L'espace utilise la largeur disponible pour présenter les informations sans empiler inutilement des cartes. Les résultats restent néanmoins séparés par famille afin de faciliter la lecture.

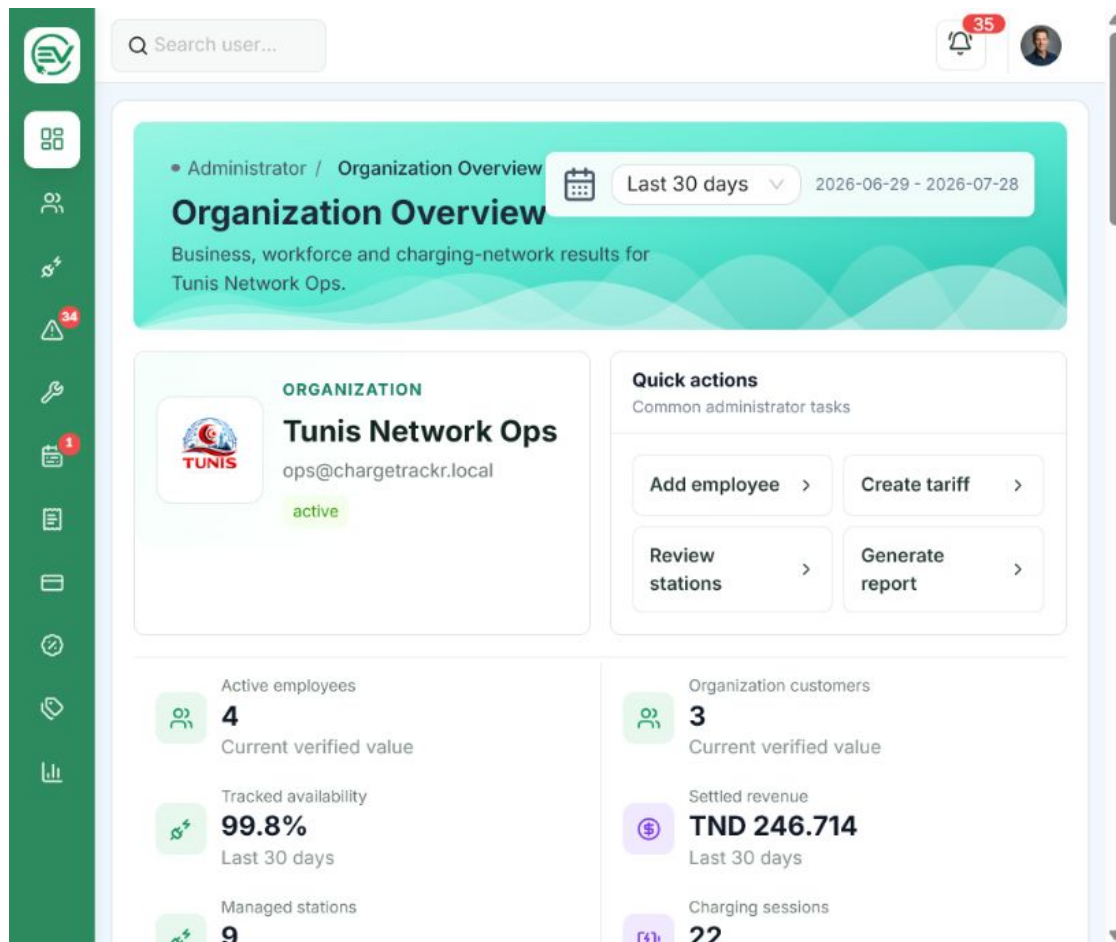


Figure 5.3 – Tableau de bord de l'administrateur d'organisation

La recherche globale, le centre de notifications et le profil restent accessibles sans changer de contexte. Les raccourcis conduisent vers des workflows complets, par exemple la création d'un employé, l'enregistrement d'une station ou la génération d'un rapport.

5.5 Stations et exploitation opérationnelle

L'inventaire des stations permet de basculer entre carte, grille, liste et table. Chaque mode répond à un besoin distinct : repérer géographiquement un actif, comparer rapidement quelques stations ou exploiter un ensemble plus dense. Les filtres et la recherche sont partagés entre les vues.

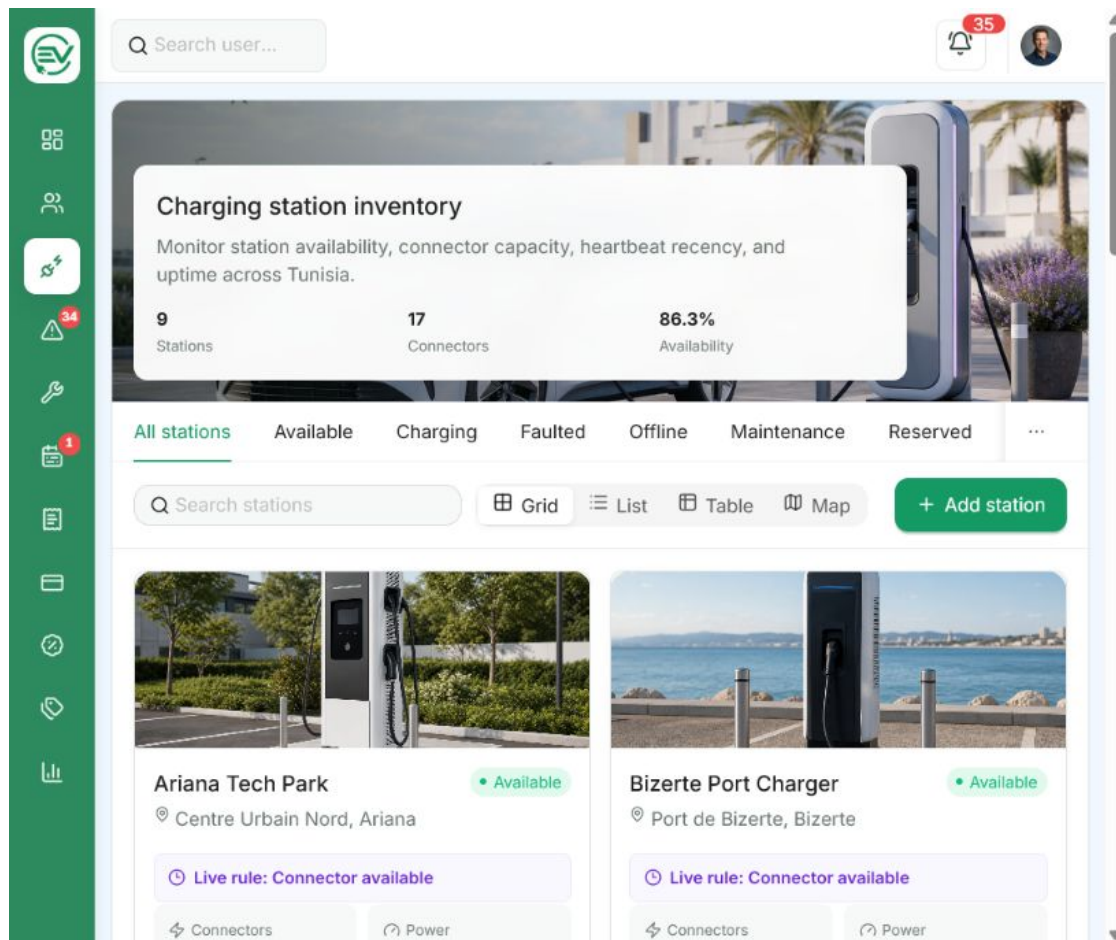


Figure 5.4 – Inventaire des stations avec disponibilité calculée

Les états visibles ne sont pas saisis dans l'interface. Ils proviennent des projections décrites au chapitre précédent. Les actions de création et de commissioning sont proposées à l'administrateur et à l'opérateur, alors que le technicien dispose d'une consultation orientée diagnostic et que le client ne voit que les stations utilisables.

Le centre d'alertes conserve la liste et le détail dans le même écran. Cette disposition permet de qualifier plusieurs événements sans perdre les filtres, la sélection ou la chronologie.

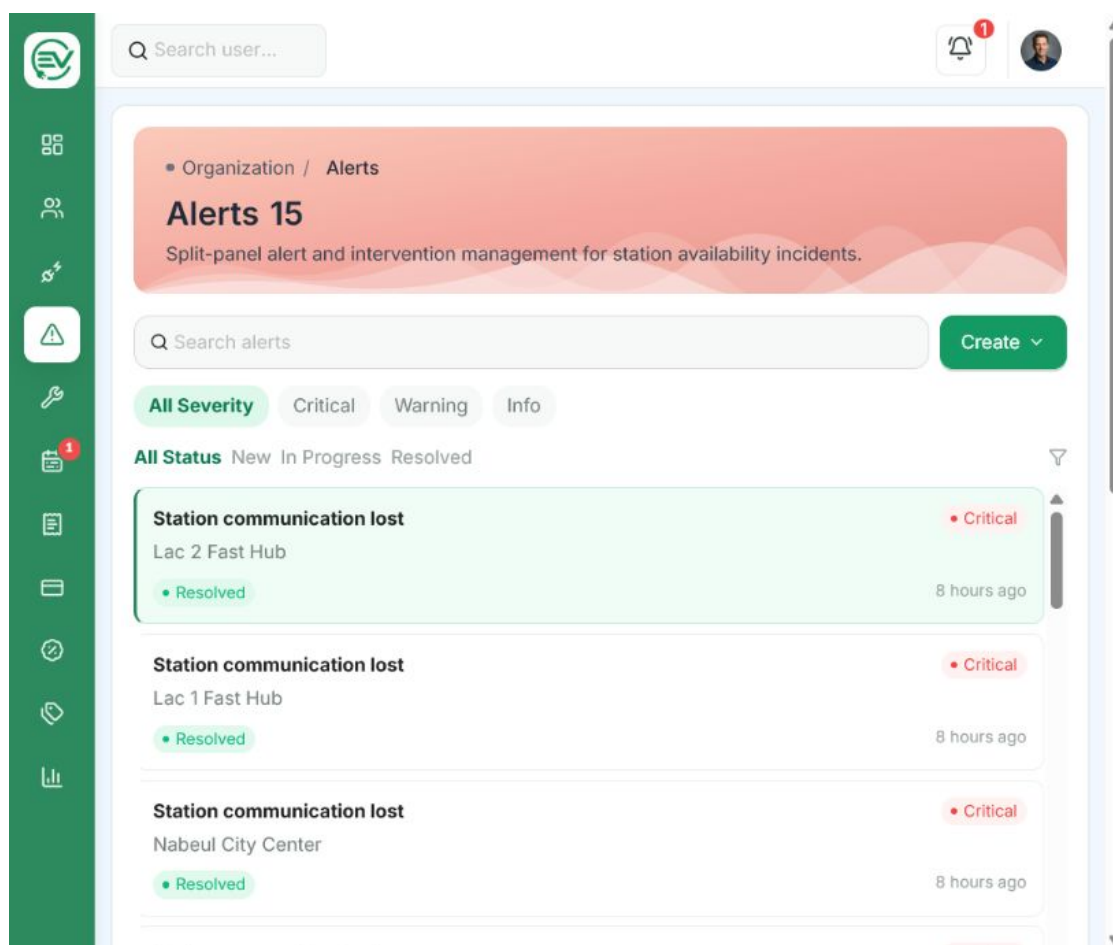


Figure 5.5 – Centre opérationnel des alertes

La couleur indique la sévérité ou l'état, mais elle n'est jamais l'unique information disponible : les libellés, références, dates et actions restent explicites. Les compteurs de navigation attirent l'attention sans remplacer le filtrage de la page.

5.6 Gestion commerciale et reporting

L'abonnement de l'organisation est séparé des paiements des conducteurs. La page commerciale présente le plan courant, la période d'évaluation ou de facturation, les capacités consommées et les demandes de changement. Cette séparation rend le rôle financier de chaque écran explicite.

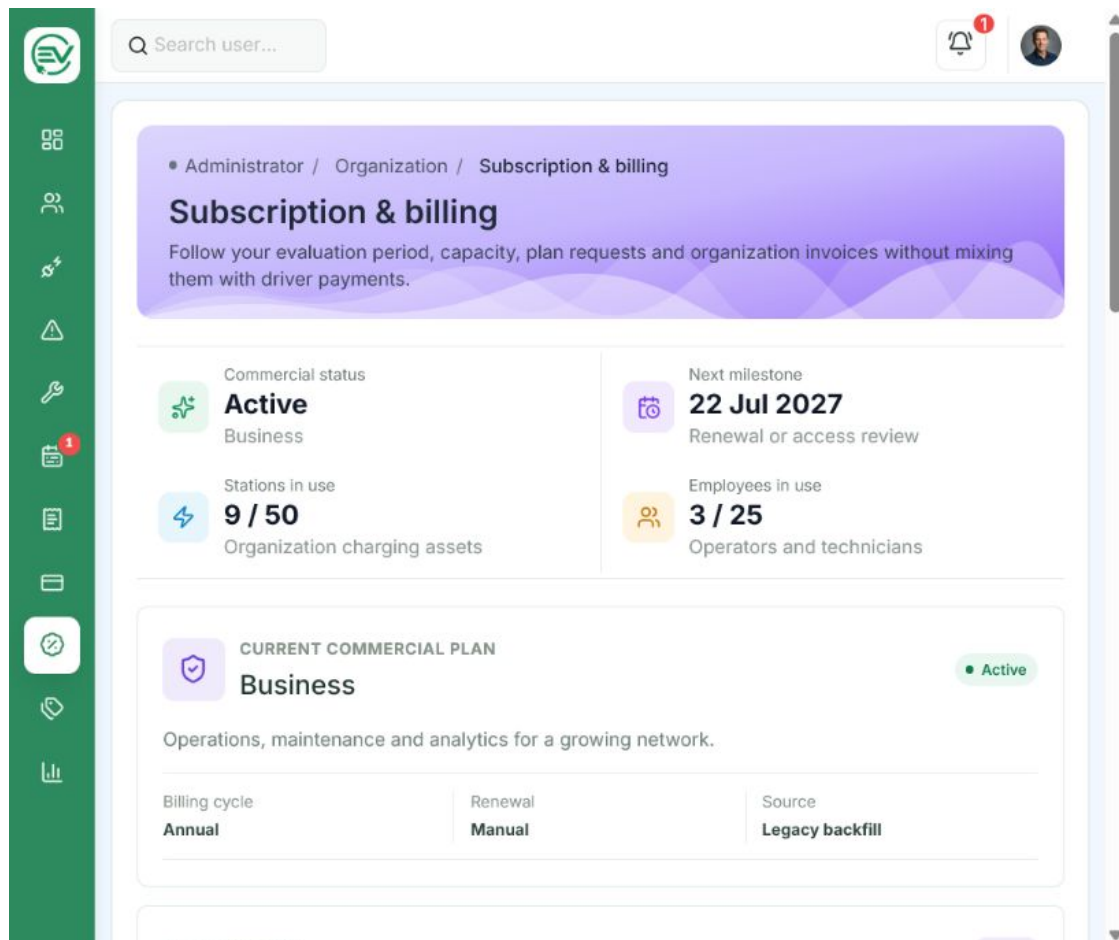


Figure 5.6 – Suivi de l'abonnement commercial de l'organisation

Le studio de performance complète le tableau de bord sans le dupliquer. Il propose des indicateurs plus détaillés, des répartitions, des tendances et des filtres de période. Les rapports internes peuvent ensuite être composés, accompagnés de pièces jointes et transmis dans le périmètre de l'organisation.

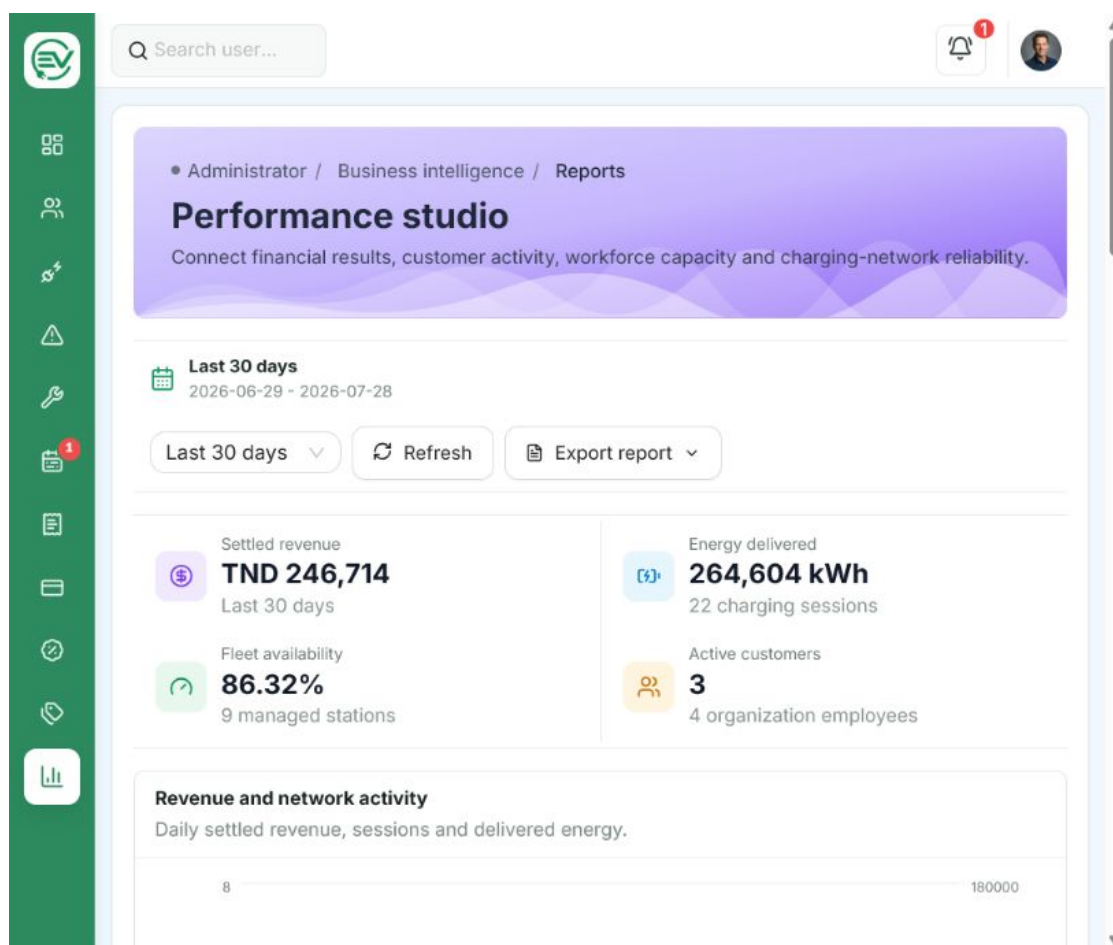


Figure 5.7 – Analyse et rapports de l’administrateur

Les exports PDF, CSV et JSON sont disponibles depuis les ressources auxquelles ils se rapportent. Cette proximité évite de transformer la page d’analyse en entrepôt de documents et permet de retrouver un reçu, une facture ou un rapport d’intervention dans son workflow d’origine.

5.7 Principes d’ergonomie retenus

Les interfaces réalisées appliquent les principes suivants :

- conserver une navigation et une hiérarchie visuelle stables entre les rôles, sans exposer les fonctions non autorisées ;
- proposer des contrôles adaptés aux données : carte et coordonnées pour une position, sélecteur pour un état, calendrier pour une date et formats dédiés pour les exports ;
- afficher les états de chargement, les écrans vides et les erreurs au niveau du contexte concerné ;

- réduire les confirmations manuelles lorsque l'événement réel peut être observé, notamment pendant le branchement OCPP ;
- conserver les actions critiques explicites et demander une confirmation avant une suppression, un arrêt de session ou une commande distante ;
- adapter la densité aux écrans plus étroits sans superposer la carte, la barre latérale ou les panneaux de détail.

Conclusion

Les captures montrent la continuité entre le prototype validé et l'application fonctionnelle. L'identité visuelle demeure commune, tandis que les priorités, les indicateurs et les actions varient réellement selon le rôle. Le chapitre précédent établit la réalité technique des échanges OCPP, du calcul de disponibilité et des services métier ; celui-ci montre comment ces capacités sont rendues accessibles dans des parcours cohérents et exploitables.

Conclusion générale

Le stage réalisé au sein de TAC-TIC Tunisie avait pour objectif de concevoir et développer un outil de visualisation et de suivi de la disponibilité des bornes de recharge électrique. Ce sujet a conduit à dépasser la simple représentation d'un état enregistré en base : une disponibilité exploitable doit être dérivée des messages de la borne, de la fraîcheur de sa communication, de ses connecteurs, de ses transactions et des contraintes d'exploitation.

Bilan du travail réalisé

ChargeTrackr aboutit à un MVP vertical qui relie une borne simulée à une application métier complète. Le simulateur SAP communique en OCPP 1.6J avec une passerelle Python ; les événements sont transmis de manière signée à Laravel, persistés puis projetés en états de disponibilité. Ces états alimentent la cartographie, les alertes, les tableaux de bord et le parcours de recharge. Une session peut être préparée par le client, démarrée à distance, suivie à partir des **MeterValues**, arrêtée puis facturée au moyen d'un prestataire HTTP simulé derrière l'interface **PaymentGateway**.

Le produit couvre également l'identité locale et Google, les invitations, les demandes de démonstration, l'isolation multi-organisation, les stations et connecteurs, les commandes OCPP, les alertes, les interventions, la maintenance, les tarifs, les abonnements clients, le contrat commercial des organisations et les rapports. Les espaces Super Administrateur, Administrateur, Opérateur, Technicien et Client présentent des indicateurs et des actions adaptés à leurs responsabilités, au lieu de réutiliser un tableau de bord générique.

La validation locale confirme la cohérence du socle réalisé : 201 tests backend et 1 489 assertions réussissent, les outils OCPP disposent de leurs propres tests, le lint frontend est valide et la compilation de production aboutit. Ces résultats ne remplacent pas une recette sur infrastructure réelle, mais ils fournissent une base mesurable et reproductible pour le MVP.

Apports du stage

Ce travail a permis de mobiliser les différentes étapes d'un projet de génie logiciel : clarification des acteurs et des règles métier, construction d'un Product Backlog, modélisation UML, choix d'une architecture, développement organisé en Sprints, intégration de services externes, tests et documentation. Il a également renforcé des compétences techniques en React et TypeScript, Laravel, PostgreSQL, WebSocket, OCPP, Docker, sécurité multi-tenant et génération de documents.

L'apport principal ne se limite pas au volume fonctionnel. Le projet met en place des frontières explicites entre l'interface, le domaine et l'infrastructure : Laravel reste propriétaire des autorisations et des règles, la passerelle OCPP ne calcule pas l'état métier, le frontend n'invente pas les mesures et le paiement dépend d'un contrat remplaçable. Cette séparation réduit le risque lors du remplacement futur d'un simulateur par un service ou un équipement réel.

Difficultés et réponses apportées

Les principales difficultés ont concerné les dépendances entre les objectifs des Sprints. Une session de recharge fiable dépend du commissioning, de la connexion OCPP, des états de connecteur, du tarif et du paiement. La progression par Sprints a donc été organisée sans supposer que les fonctionnalités étaient indépendantes : chaque Sprint consolidait les contrats nécessaires au suivant.

La deuxième difficulté a porté sur la distinction entre données simulées et preuves réelles. L'interface n'affiche pas de télémétrie synthétique lorsque la borne ne fournit pas de `MeterValues`. De même, un bouton de commande distante ne signifie pas que l'action a réussi ; le résultat provient de la réponse OCPP. Enfin, WireMock simule le comportement d'une API financière, mais il n'est jamais présenté comme un paiement bancaire certifié.

L'isolation des organisations, les erreurs asynchrones et les différences entre SQLite de test et PostgreSQL ont également nécessité des contrôles spécifiques. Les Form Requests, politiques, services de périmètre, clés d'idempotence, jobs planifiés et tests de non-régression ont été utilisés pour traiter ces risques.

Limites de la version actuelle

La version présentée reste un MVP exécuté dans un environnement local. La borne est simulée, le paiement est externe mais fictif et le déploiement de production n'est pas réalisé. Les mesures de performance actuelles ne constituent pas une campagne de charge représentative d'un grand réseau. La couverture fonctionnelle est importante, mais une recette finale avec l'encadrant, des tests E2E plus larges, une documentation OpenAPI complète et l'observabilité de production restent nécessaires avant une exploitation commerciale.

Perspectives

Les évolutions doivent être introduites selon leur dépendance et leur niveau de risque. La première priorité est l'industrialisation : CI/CD, TLS, gestion centralisée des secrets, sauvegardes, métriques, journaux corrélés, alertes techniques et tests de charge OCPP. Une recette formelle pourra ensuite figer les critères d'acceptation du MVP.

La deuxième priorité consiste à confronter les adaptateurs à des systèmes réels : connecter une borne physique OCPP 1.6J et intégrer, dans un environnement de test, un prestataire compatible avec le marché tunisien. Cette étape devra couvrir la réconciliation, les remboursements, les litiges et les exigences contractuelles du fournisseur.

À moyen terme, la plateforme pourra enrichir la cartographie et les itinéraires, étudier OCPP 2.0.1 et préparer une gestion de firmware signée avec suivi de progression et retour arrière. Les fonctions d'intelligence artificielle, notamment la prédiction de pannes ou l'assistance au diagnostic, ne devront être ajoutées qu'après collecte de données fiables, définition d'indicateurs mesurables et stabilisation du cœur opérationnel.

Ainsi, ChargeTrackr fournit une base cohérente pour superviser un réseau de recharge, coordonner les acteurs et faire évoluer les intégrations sans remettre en cause les règles essentielles du domaine. Le stage a permis de transformer un sujet initial de visualisation en une plateforme démontrable, testée et ouverte à une validation progressive sur des infrastructures réelles.

Bibliographie

- [1] TAC-TIC. Présentation de tac-tic. <https://tn.linkedin.com/company/tac-tic>. Page institutionnelle LinkedIn, consultée le 28 juillet 2026.
- [2] International Energy Agency. Global ev outlook 2026 – electric vehicle charging. <https://www.iea.org/reports/global-ev-outlook-2026/electric-vehicle-charging-chap-6-and-10>, 2026. Consulté le 28 juillet 2026.
- [3] Open Charge Alliance. Open charge point protocol 1.6. <https://openchargealliance.org/protocols/ocpp-protocols/ocpp-1-6/>. Consulté le 28 juillet 2026.
- [4] Ricardo Antunes et al. Open vehicle-to-everything management platform. *Information Systems*, 2024. https://ev4eu.eu/wp-content/uploads/2024/12/2024_OV2XMP_Information-Systems-journal_PPC.pdf, consulté le 28 juillet 2026.
- [5] Open Charge Alliance. Improving uptime monitoring with ocpp. https://openchargealliance.org/wp-content/uploads/2024/08/improving_uptime_with_ocpp_v1_2.pdf, 2024. Livre blanc, consulté le 28 juillet 2026.
- [6] ChargePoint. Chargepoint software. <https://www.chargepoint.com/products/software>. Consulté le 28 juillet 2026.
- [7] Driivz. Ev charging operations management. <https://driivz.com/glossary/operations-management/>. Consulté le 28 juillet 2026.
- [8] Monta. Monta hub – charge point management system. <https://monta.com/en/products/hub-charge-point-management-system/>. Consulté le 28 juillet 2026.
- [9] LF Energy. Citrineos. <https://lfenergy.org/projects/citrineos/>. Consulté le 28 juillet 2026.
- [10] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/scrum-guide.html>, 2020. Consulté le 28 juillet 2026.
- [11] Meta Open Source. React documentation. <https://react.dev/>. Consulté en 2026.
- [12] Leaflet. Leaflet – an open-source javascript library for interactive maps. <https://leafletjs.com/>. Consulté le 28 juillet 2026.
- [13] OpenStreetMap contributors. Openstreetmap. <https://www.openstreetmap.org/>. Consulté le 28 juillet 2026.
- [14] Laravel. Laravel documentation. <https://laravel.com/docs>. Consulté en 2026.

-
- [15] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>. Consulté en 2026.
 - [16] SAP. E-mobility charging stations simulator. <https://github.com/SAP/e-mobility-charging-stations-simulator>. Dépôt GitHub officiel, consulté le 28 juillet 2026.